

PROYECTO

Consultoría para todos

DESARROLLO TÉCNICO

Resumen Ejecutivo del Proyecto: Consultoría para todos

Propósito del Proyecto Consultoría para todos es una plataforma móvil (iOS y Android) diseñada para simplificar la formalización legal, fiscal y corporativa de emprendedores y PyMEs en México mediante un ecosistema digital de trámites, asesorías en vivo y recursos legales.

1. Arquitectura y Frontend en Flutter

- Módulos Core: Desarrollo de interfaz para la gestión de trámites por etapas (Checklist), catálogo de plantillas/contratos legales descargables y marketplace de especialistas.
- Integración Multimedia (Daily.co): Pantalla de videollamada personalizada con controles WebRTC (micrófono, cámara, levantar mano) y miniatura en vista Picture-in-Picture.
- Integración de Pagos: Flujo nativo para reservación de citas y cobro con tarjeta mediante Stripe SDK.

2. Backend y Base de Datos (Supabase)

- Estructura Relacional: Base de datos en PostgreSQL con tablas para usuarios (`profiles`), especialistas (`advisor_profiles`), reservaciones (`appointments`), avance de trámites (`user_checklist_progress`) y plantillas (`templates`).
- Seguridad: Políticas de *Row Level Security* (RLS) para proteger los datos personales, archivos descargables y llamadas de cada cliente/asesor.

3. Notificaciones, Telemetría y Monitoreo

- Notificaciones Push: Integración con Firebase Cloud Messaging (FCM) y `flutter_local_notifications` para avisos de citas y confirmaciones de pago.
- Manejo de Errores y Calidad: Captura de *crashes* en tiempo real con Firebase Crashlytics y Sentry, junto con rastreo de métricas clave mediante Firebase Analytics.

4. Pruebas y Optimización

- Pruebas Automatizadas: Cobertura de flujo de extremo a extremo (E2E) con `integration_test` simulando la reserva, pago, backend y entrada a la llamada.

- Rendimiento: Carga diferida de módulos (*deferred loading*), uso eficiente de caché de imágenes, limpieza estricta de memoria en WebRTC y optimización del tamaño final del binario (AAB / IPA).

5. Preparación para Producción y Lanzamiento

- Configuración para Tiendas: Generación de claves de firma, ajuste de permisos de sistema (cámara/micrófono) y estructuración de la ficha de tienda para Google Play y App Store.
- Estrategia Go-to-Market: Plan de tres fases (Pre-lanzamiento beta, lanzamiento con marketing de contenidos/ads y programa de retención/referidos).

DESARROLLO TÉCNICO

Estructura y Módulos de la App

- **Módulo 1: Ruta de Emprendimiento (Para Nuevos Negocios)**
 - **Flujo interactivo:** Un *step-by-step checklist* personalizado según el giro del negocio (comercio, servicios, producto físico).
 - **Puntos clave:**
 1. Definición de figura jurídica (Persona Física con Actividad Empresarial vs. RESICO vs. Sociedad/SAS).
 2. Alta en el SAT (RFC, e.firma y Certificado de Sello Digital para facturación).
 3. Registro de Marca en el IMPI (búsqueda fonética previa y registro de signo distintivo).
 4. Permisos locales (Uso de suelo, aviso de funcionamiento en COFEPRIS o Municipio).
- **Módulo 2: Optimización de Procesos (Para Negocios Establecidos)**
 - Generador de Manuales de Operación y diagramas de flujo.
 - Plantillas descargables para inventarios, control de caja y gestión de personal.
 - Módulo de evaluación de salud operativa y financiera básica.
- **Módulo 3: Directory & Marketplace de Asesores**
 - Perfiles verificados de contadores, abogados corporativos, diseñadores de marca y gestores.
 - Agenda integrada para sesiones de consultoría 1 a 1 mediante videollamada o chat.

Arquitectura Técnica Recomendada

Capa	Tecnología Sugerida	Justificación
Frontend Mobile	Flutter o React Native	Permite crear la app para iOS y Android con un solo código, reduciendo tiempo y costos.
Backend / API	Node.js (Express) o Python (FastAPI)	Procesamiento rápido de peticiones, manejo de autenticación y notificaciones push.
Base de Datos	PostgreSQL + Firebase	PostgreSQL para transacciones financieras/agendas y Firebase para autenticación y chat en tiempo real.
Pagos & Suscripciones	Stripe API o Mercado Pago SDK	Cobro recurrente de suscripciones y gestión de pagos a asesores externos dentro de México.

Estrategia de Monetización y Próximos Pasos

- **Suscripción Fija (SaaS):** Acceso ilimitado a las guías de trámites, plantillas de procesos y generador de documentos.
- **Comisión por Asesoría (Marketplace):** Cobro de un porcentaje (ej. 15%-20%) sobre el costo de la sesión fijado por cada especialista.

Paso 1: Definir los Flujos de Usuario (User Flows)

Mapear la experiencia paso a paso para los tres tipos de usuarios:

- **Emprendedor nuevo:** Registro → Cuestionario de diagnóstico → Generación de su checklist de trámites (SAT, IMPI, Municipio) → Monitoreo de avances.
- **Negocio establecido:** Registro → Selección de área a optimizar (Ventas, Inventario, Finanzas) → Descarga de plantillas / Creación de procesos.
- **Asesor profesional:** Registro y validación de credenciales → Configuración de disponibilidad y tarifa → Atender videollamadas/consultas.

Paso 2: Modelar la Base de Datos Inicial

Estructurar la información clave requerida para la base de datos (PostgreSQL/Firebase):

Entidad	Campos Clave
Usuarios	id, nombre, email, tipo_suscripcion (Free/Pro), etapa_negocio (Idea/Establecido)
Trámites / Pasos	id, categoria (SAT/IMPI/Local), titulo, descripcion, orden, documentos_requeridos
Asesores	id, usuario_id, especialidad (Contable/Legal/Branding), tarifa_por_hora, verificado (Bool)
Citas	id, usuario_id, asesor_id, fecha_hora, estatus (Pendiente/Completada), monto_pago

Paso 3: Crear el Prototipo Visual (Wireframes)

Diseñar las pantallas principales en **Figma** (o contratar a un diseñador UX/UI para hacerlo):

1. Pantalla de inicio con el progreso del checklist.
2. Detalle del trámite (ej. "Cómo sacar la e.firma" con enlaces oficiales y requisitos).
3. Catálogo de asesores con filtros por especialidad y botón de agendar.

Paso 4: Construir el Producto Mínimo Viable (MVP)

Desarrollar únicamente las funciones esenciales para lanzar al mercado rápido:

- Registro e inicio de sesión.
- Guía interactiva de trámites (SAT/IMPI).
- Módulo de pago de suscripción (Stripe/Mercado Pago).
- Directorio simple para agendar asesores.

El objetivo es que la app califique al usuario inmediatamente al registrarse para mostrarle solo lo que necesita, sin saturarlo de información.

1. Flujo del Emprendedor Nuevo (Desde Cero)

El objetivo aquí es transformar la incertidumbre en un **checklist accionable**.

- **Pantalla de Bienvenida y Registro:** Correo / Google / Apple ID.
- **Cuestionario de Onboarding (Diagnóstico):**
 - ¿Qué vas a vender? (*Productos físicos, Servicios profesionales, Comercio digital/E-commerce*).
 - ¿Con quién emprendes? (*Solo / Con socios*).
 - ¿Cuál es tu proyección de ingresos anuales estimada? (*Ayuda a determinar si le conviene RESICO, Persona Física con Actividad Empresarial o Persona Moral/SAS*).
- **Pantalla Principal (Dashboard):**
 - **Barra de progreso global:** Ej. *"Tu negocio está al 25% de quedar listo"*.
 - **Ruta paso a paso activa:**
 - **Paso 1: Identidad y Marca:** Nombre, Búsqueda fonética IMPI, Registro de marca, Diseño de Logo/Identidad.
 - **Paso 2: Formalización Fiscal (SAT):** Cita SAT, Alta de RFC, Obtención de e.firma, Certificado de Sello Digital (CSD) para facturar.
 - **Paso 3: Operación Local:** Uso de suelo, Permisos municipales, COFEPRIS (si aplica alimentos/salud).
 - **Paso 4: Apertura de Cuenta Bancaria:** Cuenta empresarial (Tradicional o Fintech como Clara/Konfío).
- **Acción de Conversión:** Si el usuario se atora en algún paso, un botón directo: *"Agendar asesoría con un experto en este trámite"*.

2. Flujo del Negocio Ya Establecido

El objetivo aquí es **ordenar la casa y optimizar la operación**.

- **Onboarding Corto:**
 - Giro del negocio y tiempo operando.
 - ¿Cuál es tu mayor dolor de cabeza actual? (*Falta de control de inventarios, problemas de personal, caos en finanzas, ventas estancadas*).
- **Dashboard de Procesos:**
 - **Categorías de Mapeo:**
 - **Finanzas:** Plantillas de flujo de caja, control de costos, precios y márgenes.
 - **Operaciones/Servicio:** Creación de Estándares de Procedimiento Operativo (SOPs), manuales de atención al cliente.
 - **Recursos Humanos:** Perfiles de puesto, contrato individual de trabajo, checklist de contratación.

- **Herramienta Interactiva:** Generador simple de diagramas o listas de tareas paso a paso para delegar a su personal.

3. Flujo del Asesor Profesional (Proveedor)

El objetivo aquí es **validar su experiencia y automatizar su agenda/cobros**.

- **Registro y Verificación:**
 - Selección de especialidad (*Contable/Fiscal, Jurídico/Propiedad Intelectual, Branding/Diseño, Procesos*).
 - Carga de credenciales (*Cédula profesional, identificación oficial, RFC para facturar*).
 - Revisión por tu equipo (*Estatus: "En aprobación"*).
- **Configuración del Perfil (Una vez aprobado):**
 - Definición de tarifa por hora / sesión.
 - Vinculación de calendario (Google Calendar / Outlook) para definir horarios disponibles.
 - Configuración de datos bancarios para recibir sus pagos.
- **Panel de Control:**
 - Solicitudes de citas entrantes, historial de asesorías y saldo acumulado a cobrar.

Siguiente decisión para el diseño:

El cuestionario de *onboarding* debe ser breve (máximo 4 a 5 preguntas), dinámico y de opción múltiple. Su objetivo principal es que el algoritmo arme una ruta de trámites 100% personalizada para la situación legal, fiscal y operativa del usuario en México.

Pregunta 1: Diagnóstico de Etapa

¿En qué punto te encuentras hoy?

- **Opción A:** Tengo una idea o proyecto por arrancar.
 - *Acción interna:* Activa el flujo "**Emprendedor Nuevo**".
- **Opción B:** Ya vendo o presto servicios, pero quiero formalizarme.
 - *Acción interna:* Activa el flujo "**Formalización Rápida**" (prioriza SAT e IMPI).
- **Opción C:** Ya tengo un negocio formal y quiero ordenar mis procesos.
 - *Acción interna:* Activa el flujo "**Negocio Establecido**" (desbloquea módulo de SOPs y Finanzas).

Pregunta 2: Modelo de Negocio e Industria

¿Qué tipo de oferta vas a comercializar principalmente?

- **Opción A:** Productos físicos (Comercio, manufactura, importación).
- **Opción B:** Servicios profesionales o técnicos (Consultoría, agencia, oficios).
- **Opción C:** Alimentos, bebidas o productos de salud/belleza.
- **Opción D:** Software, contenido digital o E-commerce puro.

Efecto en el checklist:

- Si elige **Opción C**, la app agrega automáticamente el checklist de permisos sanitarios (**COFEPRIS** y licencias municipales).
- Si elige **Opción D**, la app agrega avisos de privacidad para sitios web y pasarelas de pago.

Pregunta 3: Estructura de Socios (Figura Jurídica)

¿Vas a emprender solo o con socios?

- **Opción A:** Emprendo solo.
- **Opción B:** Tengo socios y queremos constituirnos legalmente.
- **Opción C:** Ya tenemos una empresa registrada (Persona Moral).

Efecto en el checklist:

- Si elige **Opción A**, se le sugiere tributar como **Persona Física** (ej. RESICO) para ahorrar costos notariales.
- Si elige **Opción B**, se le habilita la opción de **SAS (Sociedad por Acciones Simplificada)**, que es gratuita y 100% en línea ante la Secretaría de Economía, o constitución ante Notario/Corredor Público.

Pregunta 4: Proyección Fiscal Inicial

¿Cuánto estimas facturar en tu primer año?

- **Opción A:** Hasta \$3.5 millones de pesos MXN.
- **Opción B:** Más de \$3.5 millones de pesos MXN.
- **Opción C:** Aún no lo sé con certeza.

Efecto en el checklist:

- Si selecciona **Opción A** (y es Persona Física), la app le recomienda automáticamente el régimen **RESICO** (impuesto sobre la renta desde el 1% al 2.5%).
- Si selecciona **Opción B**, la app sugiere evaluar el Régimen General de Ley con un contador.

Pregunta 5: Propiedad Intelectual y Marca

¿Ya cuentas con nombre y logotipo comercial?

- **Opción A:** No, necesito ayuda para crearlos.
- **Opción B:** Sí, pero no los he registrado ante el IMPI.
- **Opción C:** Ya inicié o tengo el registro de marca otorgado.

Efecto en el checklist:

- Si selecciona **Opción B**, la app pone como **Paso Prioritario #1** la "Búsqueda Fonética Gratuita" y la guía de registro de marca en el **IMPI**.

Resultado del Onboarding (Pantalla de Confirmación)

Al terminar la última pregunta, la app muestra un resumen rápido antes de enviarlo a su Dashboard:

Plaintext

¡Listo, [Nombre]! Hemos armado tu ruta personalizada:

- ✓ Régimen fiscal sugerido: RESICO (Persona Física)
- ✓ Trámites clave detectados: Alta SAT, Registro IMPI, Licencia Municipal
- ✓ Requisitos sanitarios: No requeridos para tu giro

La pantalla principal (Dashboard) es el centro de control de la app. Debe mostrar con claridad el estado actual del negocio y reducir la carga mental del emprendedor destacando únicamente la **siguiente tarea prioritaria**.

1. Encabezado (Header & Diagnóstico Rápido)

- **Saludo personalizado:** *"¡Hola, [Nombre]! Vamos a formalizar [Nombre del Negocio]."*
- **Badge de Estatus Fiscal/Legal:** Etiqueta dinámica (ej. *Persona Física - RESICO o En proceso de constitución*).
- **Tarjeta de Progreso General:**
 - **Barra de avance:** Porcentaje completado del *checklist* de arranque (ej. **35% completado**).
 - **Métrica clave:** *3 de 8 trámites completados*.

2. Sección Central: La Ruta de Trámites (Checklist Dinámico)

Diseñada en formato de lista con acordeón expansible para evitar saturar de información al usuario:

Bloque / Fase	Estado	Contenido / Acción Principal
Fase 1: Identidad & IMPI	● Completado	Registro de Marca en revisión ante el IMPI.
Fase 2: Formalización SAT	● En curso (Destacado)	Siguiente tarea: Sacar cita para e.firma. Botón directo a la guía paso a paso y requisitos.

Fase 3: Permisos Locales	☐ Bloqueado	Requiere completar la Fase 2 (Alta en Hacienda).
Fase 4: Apertura Bancaria	☐ Bloqueado	Requiere Constancia de Situación Fiscal y e.firma.

3. Sección Lateral / Inferior: Módulos de Apoyo Directo

- **Acceso Directo a Asesores (Marketplace Contextual):**
 - Botón flotante o tarjeta destacada: *"¿Te atoraste con tu cita del SAT o la búsqueda fonética del IMPI? Habla con un contador o abogado verificado."*
 - Accesos directos para agendar sesiones rápidas de 30 o 60 minutos.
- **Biblioteca de Recursos & Plantillas:**
 - Descargables listos en formato editable:
 - Plantilla de Contrato Individual de Trabajo (Adaptado a Ley Federal del Trabajo en México).
 - Plantilla de Control de Flujo de Caja en Excel/Google Sheets.
 - Formato de Aviso de Privacidad Simplificado.

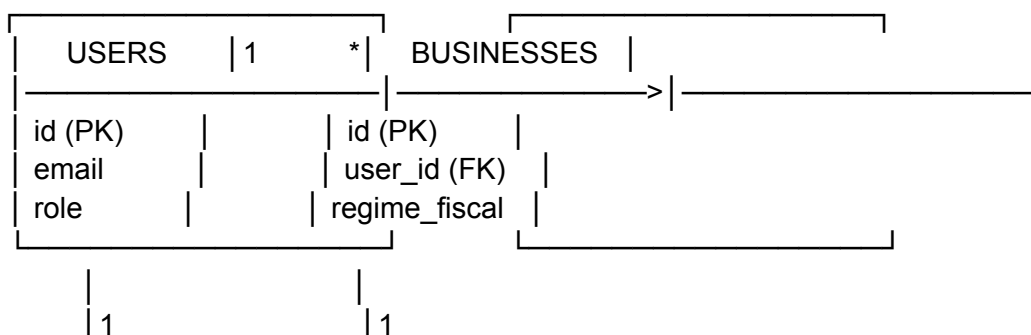
4. Barra de Navegación Inferior (Bottom Navigation Bar)

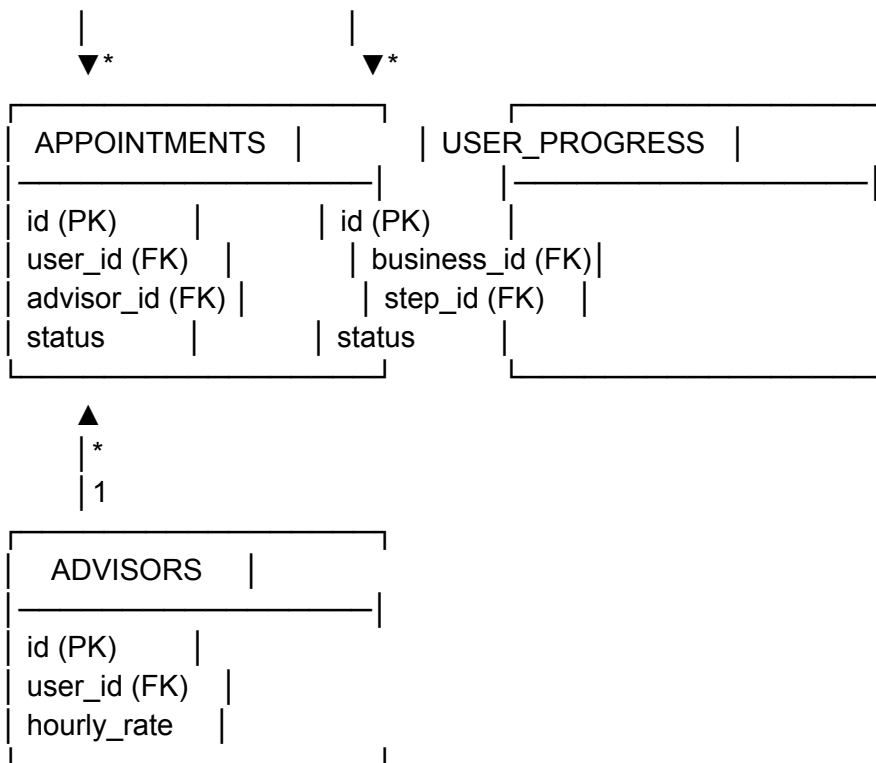
1. **Inicio (Dashboard):** Vista actual de la ruta de emprendimiento.
2. **Procesos:** Biblioteca de manuales, plantillas y herramientas de operación.
3. **Asesores:** Directorio de profesionales, agenda y chats activos.
4. **Perfil:** Datos fiscales guardados, gestión de suscripción y configuración de la cuenta.

Para que la plataforma funcione de manera fluida, escalable y segura, el modelo de base de datos se estructurará de forma **relacional** (usando **PostgreSQL**). Esto garantiza consistencia en las transacciones de pago, control estricto de citas y un seguimiento preciso del progreso de cada negocio.

1. Diagrama de Relaciones Entidad-Relación (ERD)

Plaintext





2. Especificación de Tablas y Campos

A. Gestión de Usuarios y Perfiles

SQL

-- Tabla base para la autenticación y roles generales

```

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  full_name VARCHAR(150) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  phone VARCHAR(20),
  role VARCHAR(20) CHECK (role IN ('entrepreneur', 'advisor', 'admin')),
  subscription_status VARCHAR(20) DEFAULT 'free' CHECK (subscription_status IN ('free', 'pro', 'cancelled')),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
  
```

-- Perfil específico del negocio o emprendimiento en México

```

CREATE TABLE businesses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  business_name VARCHAR(150) NOT NULL,
  stage VARCHAR(30) CHECK (stage IN ('idea', 'formalization', 'established')),
  industry VARCHAR(50) NOT NULL, -- Ej: Comercio, Alimentos, Servicios, Software
  entity_type VARCHAR(30) CHECK (entity_type IN ('persona_fisica', 'persona_moral', 'sas')),
);
  
```

```

    fiscal_regime VARCHAR(50), -- Ej: RESICO, Actividad Empresarial, Régimen General
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

B. Motor de Ruta de Trámites y Progreso

SQL

```

-- Catálogo general de trámites en México (SAT, IMPI, Gobierno Local)
CREATE TABLE steps_catalog (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category VARCHAR(30) CHECK (category IN ('brand_impi', 'sat_fiscal',
'municipal_permits', 'banking', 'processes')),
    title VARCHAR(150) NOT NULL,
    description TEXT,
    required_for_entity VARCHAR(30), -- 'all', 'persona_fisica', 'persona_moral'
    order_index INT NOT NULL,
    official_link VARCHAR(255),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- Estado de avance de cada trámite por negocio

```

CREATE TABLE user_progress (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    business_id UUID REFERENCES businesses(id) ON DELETE CASCADE,
    step_id UUID REFERENCES steps_catalog(id) ON DELETE CASCADE,
    status VARCHAR(20) DEFAULT 'pending' CHECK (status IN ('pending', 'in_progress',
'completed', 'blocked')),
    notes TEXT,
    completed_at TIMESTAMP WITH TIME ZONE,
    UNIQUE(business_id, step_id)
);

```

C. Módulo de Asesores y Citas (Marketplace)

SQL

-- Perfil profesional de los asesores verificados

```

CREATE TABLE advisors (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    specialty VARCHAR(50) CHECK (specialty IN ('fiscal_contable', 'legal_impi', 'branding',
'procesos_operativos')),
    professional_id VARCHAR(50) NOT NULL, -- Cédula profesional o registro oficial
    bio TEXT,
    hourly_rate NUMERIC(10, 2) NOT NULL, -- Tarifa en MXN
    is_verified BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

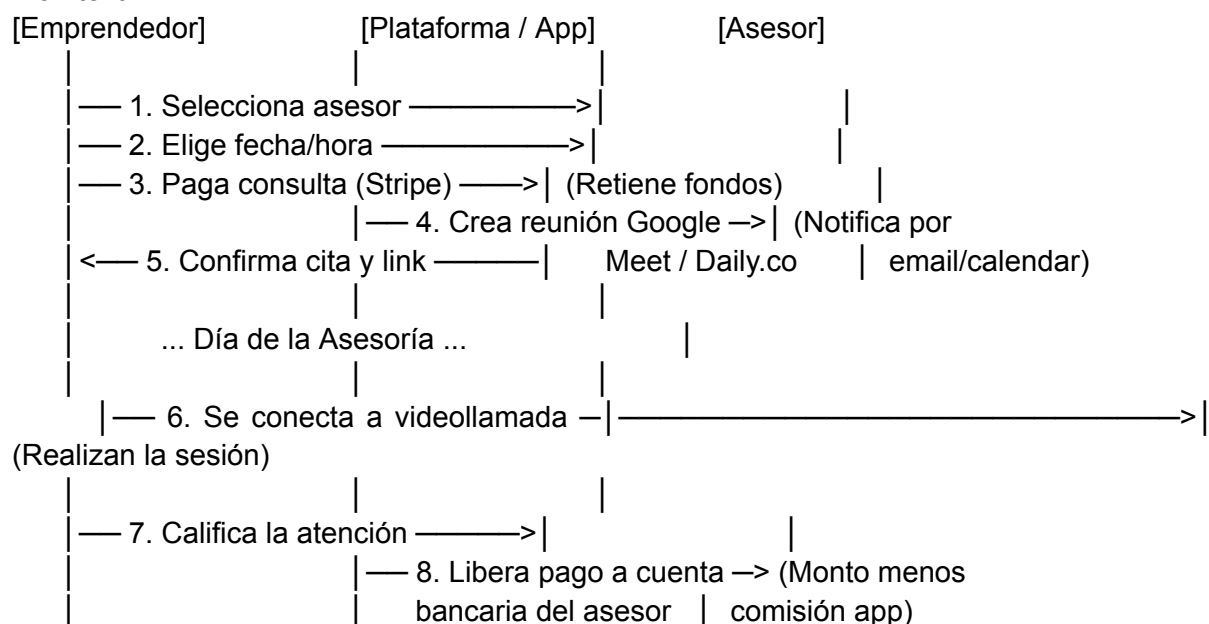
```

```
-- Registro y control de las asesorías pagadas
CREATE TABLE appointments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE SET NULL,
  advisor_id UUID REFERENCES advisors(id) ON DELETE SET NULL,
  scheduled_at TIMESTAMP WITH TIME ZONE NOT NULL,
  duration_minutes INT DEFAULT 60,
  status VARCHAR(20) DEFAULT 'scheduled' CHECK (status IN ('scheduled', 'completed',
'cancelled', 'no_show')),
  meeting_link VARCHAR(255),
  amount_paid NUMERIC(10, 2) NOT NULL,
  stripe_payment_id VARCHAR(255),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

El flujo para la contratación de asesorías debe ser ágil, confiable y con los menores puntos de fricción posibles. Al manejar dinero y tiempo de profesionales, la integración entre la agenda, la pasarela de pagos y la plataforma de videollamadas debe estar totalmente automatizada.

1. Diagrama General del Flujo

Plaintext



2. Pasos Detallados de la Experiencia de Usuario (UX)

Paso 1: Búsqueda y Selección del Asesor

- **Filtros rápidos:** Por área de especialidad (*Fiscal/SAT, Legal/IMPI, Procesos, Branding*), tarifa por hora o calificación general.

- **Ficha del Asesor:** Muestra cédula profesional verificada, número de asesorías impartidas, opiniones de otros emprendedores y su disponibilidad semanal en tiempo real.

Paso 2: Selección de Fecha y Agenda

- La app consulta los huecos libres en el calendario del asesor mediante sincronización (Google Calendar API o Calendly API).
- El usuario selecciona un bloque disponible (por ejemplo: *30 o 60 minutos*).
- **Campo de nota previa:** El usuario escribe brevemente su duda o sube un archivo (ej: *"Tengo duda sobre si mi actividad entra en RESICO"* o *"Adjunto mi búsqueda fonética del IMPI"*).

Paso 3: Proceso de Pago Seguro

- **Integración de Pasarela:** Stripe u otra pasarela de pago para procesar tarjetas de crédito/débito, Apple Pay, Google Pay y pagos en efectivo en tiendas de conveniencia.
- **Modelo Custodia (Escrow):** La app cobra el total al emprendedor pero **no transfiere el dinero al asesor inmediatamente**, sino que lo mantiene retenido hasta que la consulta se haya llevado a cabo de manera satisfactoria.

Paso 4: Generación Automática de la Videollamada y Confirmación

- Una vez aprobado el pago, el backend ejecuta una llamada a la API de **Daily.co** o **Google Meet API** para generar un enlace único y seguro para la sesión.
- Se envían recordatorios automatizados por notificación *push* y correo electrónico:
 - Inmediatamente al agendar.
 - 24 horas antes de la sesión.
 - 15 minutos antes de iniciar.

Paso 5: La Videollamada (En la App o Navegador)

- Para evitar que el usuario abandone la app, la videollamada puede incrustarse dentro de la propia aplicación móvil mediante el SDK de **Daily.co** o **Agora.io**.
- Incluye funciones básicas: encender/apagar cámara y micrófono, chat de texto interno para compartir enlaces y compartir pantalla.

Paso 6: Finalización, Calificación y Cierre Financiero

- Al terminar la llamada, la app solicita al emprendedor evaluar la sesión (1 a 5 estrellas + comentario).
- **Reparto de fondos (Split Payment):**
 - Se libera la tarifa pactada al saldo del asesor.
 - La plataforma retiene automáticamente su **comisión por corretaje** (ejemplo: 15% o 20%).
 - El asesor puede solicitar transferencias de su saldo acumulado a su cuenta CLABE mexicana mediante **Stripe Connect**.

3. Servicios de Terceros Recomendados para la Implementación

Servicio	Tecnología Sugerida	Función en la App
Pasarela de Pagos	Stripe Connect	Permite cobrar al usuario y distribuir automáticamente las comisiones entre la app y la cuenta bancaria del asesor en México.
Videollamadas	Daily.co Agora.io	o Generan salas de video integrables dentro de Flutter/React Native con consumo mínimo de ancho de banda.
Gestión de Agenda	Cal.com API (Código abierto)	Maneja zonas horarias, bloqueos de agenda y sincronización con calendarios personales de los asesores.
Notificaciones	Firebase Cloud Messaging (FCM)	Envío de alertas push antes de que inicie la sesión.

Para construir el backend (utilizando **Node.js/Express** o **Python/FastAPI**), estructuraremos los endpoints bajo una arquitectura RESTful organizada por recursos.

Todas las rutas protegidas requieren un token de autenticación (**Bearer JWT**) en el encabezado **Authorization**.

1. Módulo de Asesores (**/api/v1/advisors**)

Gestión del catálogo, perfiles y disponibilidad de los profesionales.

- **GET /api/v1/advisors**
 - **Uso:** Buscar y filtrar asesores en el marketplace.
 - **Parámetros** **Query:**
?specialty=fiscal_contable&min_rating=4.5&limit=10&page=1
 - **Respuesta:** Lista paginada con perfiles, tarifas y calificaciones.
- **GET /api/v1/advisors/:id/availability**
 - **Uso:** Obtener los horarios libres de un asesor para una fecha específica.
 - **Parámetros Query:** **?date=2026-10-15**
 - **Respuesta:** Array de slots de tiempo disponibles (**["09:00", "10:00", "16:00"]**).

2. Módulo de Reservas y Citas (**/api/v1/appointments**)

Manejo del ciclo de vida de las sesiones de asesoría.

- **POST /api/v1/appointments/intent**
 - **Uso:** Iniciar el proceso de reserva y bloquear temporalmente el horario.

- **Body (JSON):**
- JSON

```
{
  "advisor_id": "uuid-del-asesor",
  "scheduled_at": "2026-10-15T10:00:00Z",
  "duration_minutes": 60,
  "notes": "Tengo dudas sobre mi cambio a RESICO"
}
```

-
-
- **Respuesta:** Objeto de la cita con estatus `pending_payment` y `client_secret` de Stripe.
- **GET `/api/v1/appointments/my-appointments`**
 - **Uso:** Lista de citas pasadas y futuras del usuario autenticado (emprendedor o asesor).
 - **Respuesta:** Array de citas con estatus, enlaces de videollamada y detalles del profesional/cliente.
- **POST `/api/v1/appointments/:id/rating`**
 - **Uso:** Calificar la atención recibida al finalizar la videollamada.
 - **Body (JSON):** `{ "rating": 5, "comment": "Excelente atención, resolvió mis dudas del SAT." }`

3. Módulo de Pagos y Webhooks (`/api/v1/payments`)

Procesamiento seguro de transacciones mediante Stripe Connect.

- **POST `/api/v1/payments/create-payment-intent`**
 - **Uso:** Generar la orden de pago para la pasarela antes de confirmar la cita.
 - **Body (JSON):** `{ "appointment_id": "uuid-de-la-cita" }`
 - **Respuesta:** `client_secret` para inicializar el SDK de pago en la app móvil.
- **POST `/api/v1/payments/webhook` (Endpoint público / Firma validada)**
 - **Uso:** Recibir notificaciones en tiempo real desde Stripe (escucha eventos como `payment_intent.succeeded`).
 - **Acción interna:** Cambia el estatus de la cita a `scheduled`, genera la sala de videollamada y envía notificaciones push/email.

4. Módulo de Videollamadas (`/api/v1/meetings`)

Integración con el proveedor de video (ej. Daily.co).

- **GET `/api/v1/meetings/:appointment_id/token`**
 - **Uso:** Obtener las credenciales y el token de acceso temporal para unirse a la sala de video.

- **Seguridad:** Solo permite el acceso 10 minutos antes de la hora programada y valida que el usuario sea el emprendedor o el asesor asignado a esa cita.
- **Respuesta:**
- JSON

```
{
  "room_url": "https://tuapp.daily.co/asesoria-123",
  "token": "jwt-token-de-acceso-a-sala",
  "expires_at": "2026-10-15T11:10:00Z"
}
```

Para lanzar un MVP eficiente, rápido y de bajo costo en México, la clave es **no construir todo desde cero**. Debes enfocar el presupuesto y esfuerzo en validar la hipótesis principal: *¿los emprendedores mexicanos están dispuestos a pagar por una ruta guiada y agendar asesores?*

1. Alcance Recortado del MVP (Lo mínimo indispensable)

Para reducir costos y salir al mercado en **4 a 6 semanas**, debes pausar temporalmente el desarrollo de funciones complejas y concentrarte en lo esencial:

- **Módulo de Emprendimiento:**
 - Un *checklist* estático e interactivo enfocado solo en el régimen más común (**RESICO** y **Persona Física con Actividad Empresarial**) y la ruta **IMPI**.
 - Guías simples en PDF o texto estructurado (requisitos SAT, citas, e.firma y marcas).
- **Módulo de Asesores:**
 - En lugar de construir un motor de video propio, utiliza herramientas ya existentes (*no-code* o de integración rápida) para la agenda y las llamadas.
- **Módulo de Procesos:**
 - Lanzar con un repositorio estático de 5 a 10 plantillas descargables clave (Contrato de trabajo LFT, Control de caja en Excel, Formato de cotización).

2. Arquitectura de Herramientas y Reducción de Costos

Puedes construir el MVP combinando tecnologías accesibles o soluciones *No-Code* / *Low-Code* para minimizar los costos de infraestructura iniciales:

Componente	Opción MVP Rápida / Económica	Costo Estimado Inicial
Frontend Mobile	FlutterFlow o Glide/Bubble (para validar rápido) o React Native si prefieres código nativo.	\$0 - \$50 USD/mes

Base de Datos & Backend	Firestore o Supabase (tier gratuito muy amplio).	\$0 USD/mes
Agenda & Calendario	Cal.com (Integrado por iframe o API) o Calendly .	\$0 - \$15 USD/mes
Videollamadas	Google Meet (Generación de enlaces vía Cal.com) o Daily.co (10,000 min libres/mes).	\$0 USD
Pagos en México	Stripe México o Mercado Pago (Solo cobro de comisión por transacción, sin costo mensual).	3.6%+\$3 MXN por venta

3. Estrategia de Reclutamiento de los Primeros Asesores

No necesitas un catálogo de 100 asesores al inicio; **5 a 10 profesionales bien seleccionados** son suficientes para arrancar:

- **Incentivo de captación:** No cobrarles mensualidad por estar en la app durante los primeros 3 meses (solo la comisión por cita completada).
- **Perfil objetivo inicial:** Contadores independientes con experiencia en SAT/RESICO y abogados especialistas en marcas (IMPI).
- **Validación manual:** Revisa personalmente sus cédulas profesionales antes de activarlos en la plataforma para garantizar la calidad del servicio.

4. Hoja de Ruta de Lanzamiento (4 Semanas)

- **Semana 1: Contenido y Diseño**
 - Redactar las guías de los pasos del SAT e IMPI.
 - Diseñar la base de datos básica en Supabase/Firebase.
- **Semana 2: Integración Tecnológica**
 - Conectar el frontend con el sistema de autenticación y Stripe/Mercado Pago.
 - Configurar la agenda con Cal.com.
- **Semana 3: Pruebas y Carga de Contenido**
 - Onboarding manual de los primeros 5 asesores.
 - Pruebas de flujo de pago y agendamiento con usuarios de prueba (*beta testers*).
- **Semana 4: Lanzamiento Beta Abierto**
 - Publicación en tiendas (App Store / Play Store) o como Web App interactiva (PWA).
 - Promoción orgánica en comunidades de emprendedores en México (LinkedIn, grupos de Facebook de PyMES, TikTok con contenido educativo sobre impuestos/IMPI).

Con este plan, el costo de desarrollo e infraestructura inicial se reduce al mínimo y podrás medir la respuesta real del mercado antes de invertir en código a la medida más complejo.

Trámites #1: Alta en el SAT y Obtención de la e.firma, estructurada tal como la verá el usuario dentro de la app para el régimen **RESICO / Persona Física**.

Ficha de Trámite: Alta en el SAT y e.firma

Título: Alta en el RFC y Obtención de la e.firma (Persona Física)

Dependencia: Servicio de Administración Tributaria (SAT)

Costo: Gratis

Modalidad: Mixta (Inicio en línea y conclusión presencial)

1. Resumen y ¿Por qué lo necesitas?

La **e.firma** (antes FIEL) es tu firma electrónica avanzada. Es indispensable para facturar a tus clientes, emitir certificados de sello digital, presentar declaraciones y dar de alta tu negocio en otros trámites oficiales.

2. Requisitos (Checklist Previo)

Antes de sacar tu cita, debes asegurarte de contar con los siguientes documentos e insumos:

- [] **Identificación Oficial Vigente:** INE, Pasaporte o Cédula Profesional con fotografía (Original).
- [] **Comprobante de Domicilio Fiscal:** No mayor a 3 meses (Recibo de luz CFE, agua, predial o estado de cuenta bancario a tu nombre) (Original).
- [] **Clave Única de Registro de Población (CURP):** Descargada en formato reciente.
- [] **Unidad de Memoria Extraíble (USB):** En formato plano (preferentemente vacía o limpia) para guardar los archivos **.key** y **.cer**.
- [] **Correo Electrónico:** De uso personal y acceso directo.

3. Pasos a Seguir (Ruta Guiada en la App)

- **Paso 1: Preinscripción en el RFC (En línea)**
 - Ingresa al portal oficial del SAT en la sección de preinscripción.
 - Captura tus datos personales y domicilio fiscal.
 - Guarda e imprime tu **Acuse de preinscripción al RFC**.
- **Paso 2: Generar Cita en el SAT**
 - Entra a citas.sat.gob.mx.
 - Selecciona el módulo más cercano a tu domicilio.
 - Elige el trámite: **"e.firma de Persona Física"** o **"Inscripción al RFC de Persona Física"**.
 - *Tip de la App:* Si no hay disponibilidad, entra a la **Fila Virtual**; el SAT te asignará turno automáticamente por correo electrónico en pocos días.
- **Paso 3: Acudir a la Cita Presencial**

- Asiste 15 minutos antes de tu cita con todos tus requisitos físicos y tu memoria USB.
- En ventanilla capturarán tus datos biométricos (huellas dactilares, iris, fotografía y firma autógrafa).
- **Paso 4: Descarga y Resguardo de la e.firma**
 - El personal del SAT guardará en tu USB tres elementos clave:
 1. Archivo con extensión **.cer** (Certificado).
 2. Archivo con extensión **.key** (Clave privada).
 3. Una **Contraseña de acceso** (creada por ti durante la cita).
 - *Nota de seguridad:* Guarda estos tres elementos en un lugar seguro. Quien tenga tu e.firma y contraseña tiene acceso legal total sobre tu empresa.

4. Errores Comunes a Evitar

- **Comprobante de domicilio no válido:** Si la dirección del comprobante no coincide exactamente con la que registraste en el portal, el trámite será rechazado.
- **Olvidar la USB:** El SAT no te enviará los archivos por correo; si no llevas la memoria USB física, perderás la cita.

5. Siguiendo Paso Recomendado

Una vez generada tu e.firma, el siguiente paso en la app será la **Búsqueda Fonética y Registro de Marca ante el IMPI**.

¿Qué hacemos a continuación?

1. Redactamos el **Trámite #2: Búsqueda Fonética y Registro de Marca (IMPI)** para completar el paquete de contenido inicial.
2. Pasamos a la **Opción B (Diseño/Pantallas)** para ver cómo se montaría este texto dentro de la app en Figma o FlutterFlow.

Trámite #2: Búsqueda Fonética y Registro de Marca ante el IMPI, estructurada para la app.

Este paso es crítico en México para evitar que otra persona o empresa registre tu nombre comercial o logo antes que tú.

Ficha de Trámite: Búsqueda Fonética y Registro de Marca

Título: Búsqueda Fonética y Registro de Marca (Signo Distintivo)

Dependencia: Instituto Mexicano de la Propiedad Industrial (IMPI)

Costo oficial (Tarifa IMPI): \$3,126.41\$ MXN (IVA incluido) (*Vigente en México*)

Modalidad: 100% En línea (requiere e.firma o CURP)

1. Resumen y ¿Por qué lo necesitas?

El registro de marca te otorga el **derecho exclusivo de uso en todo México** durante 10 años (renovables). Evita que competidores copien tu nombre o logotipo y te da certeza jurídica para comercializar, franquiciar o licenciar tu marca.

2. Requisitos (Checklist Previo)

- [] **Nombre de la marca:** Nombre exacto con el que vas a operar.
- [] **Logotipo / Imagen:** En formato de imagen digital (.png o .jpg), si vas a registrar una marca mixta (nombre + logo).
- [] **Clasificación de Niza (Giro):** Identificar en qué clase de productos o servicios encaja tu negocio (ej. Clase 35 para comercio, Clase 43 para restaurantes, Clase 42 para software).
- [] **e.firma del SAT o Cuenta Pase IMPI:** Para firmar y enviar la solicitud en línea.
- [] **Tarjeta de crédito/débito:** Para pagar los derechos oficiales del IMPI en la plataforma.

3. Pasos a Seguir (Ruta Guiada en la App)

- **Paso 1: Búsqueda Fonética Gratuita (MARCANET)**
 - Antes de pagar, debes verificar que no exista una marca igual o similar que ya esté registrada o en trámite en tu misma clase.
 - Entra al portal oficial [MARCANET del IMPI](#).
 - Ingresa el nombre de tu marca y filtra por la Clase de Niza correspondiente.
 - *Semáforo de viabilidad en la App:*
 -  **Sin coincidencias exactas ni similares:** Alta probabilidad de aprobación.
 -  **Similitud fonética detectada:** Riesgo medio de rechazo por el IMPI.
 -  **Marca idéntica existente en tu clase:** Debes modificar o cambiar el nombre antes de pagar.
- **Paso 2: Registro en "Tu Cuenta PASE" del IMPI**
 - Crea un usuario en el portal [Tu Cuenta PASE](#) usando tu CURP o e.firma del SAT.
- **Paso 3: Llenado de la Solicitud (Marca en Línea)**
 - Selecciona el tipo de marca:
 - *Nominativa:* Solo el nombre.
 - *Innominada:* Solo el diseño o logo.
 - *Mixta:* Nombre + Logo (Recomendada).
 - Captura los datos del titular (Persona Física o Moral).
 - Asigna la **Clase de Niza** y describe los productos o servicios que ofrecerás.
 - Adjunta el archivo de tu logo (si aplica).
- **Paso 4: Pago de Derechos y Firma**
 - Realiza el pago en línea mediante tarjeta de crédito/débito o transferencia bancaria (SPEI).
 - Firma la solicitud con tu **e.firma** del SAT.
 - Descarga y guarda el **Acuse de Solicitud de Registro de Marca**.
- **Paso 5: Seguimiento del Expediente**
 - El examen de forma y fondo del IMPI toma entre **3 y 6 meses**.
 - Si el IMPI no emite ningún requisito u oposición, te otorgará el **Título de Marca**.

4. Errores Comunes a Evitar

- **Pagar sin hacer la búsqueda fonética previa:** El IMPI no devuelve el dinero si tu marca es rechazada por ser similar a una existente.
- **Seleccionar la clase equivocada:** Registrar una marca de ropa (Clase 25) no te protege si abres un restaurante con ese mismo nombre (Clase 43).

5. Siguiente Paso Recomendado

Una vez ingresado el trámite en el IMPI y obtenida tu e.firma, el siguiente paso en la app será la **Apertura de Cuenta Bancaria Empresarial / Pymes** y los **Permisos Locales de Funcionamiento**.

Trámite #3: Permisos Locales y Licencia de Funcionamiento Municipal, estructurada para la app.

Este trámite varía ligeramente dependiendo del municipio o alcaldía en México, pero sigue una estructura legal estándar para cualquier negocio con local físico o establecimiento comercial.

Ficha de Trámite: Permisos Locales y Licencia de Funcionamiento

Título: Licencia de Funcionamiento y Aviso de Apertura Local

Dependencia: Gobierno Municipal / Alcaldía (Dirección de Desarrollo Económico o Centro de Atención Empresarial)

Costo: Variable (Generalmente **Gratis** para giros de bajo riesgo; con costo para giros con venta de alcohol o alto impacto)

Modalidad: Presencial o En línea (según el municipio/alcaldía)

1. Resumen y ¿Por qué lo necesitas?

El permiso o aviso de funcionamiento garantiza que tu establecimiento cumple con las normas urbanísticas, de protección civil y de salubridad de tu localidad. Evita clausuras, multas administrativas o la suspensión de actividades por parte de los inspectores municipales.

2. Requisitos (Checklist Previo)

- [] **Constancia de Situación Fiscal (SAT):** Actualizada, que demuestre la actividad económica y el domicilio fiscal del negocio.
- [] **Identificación Oficial (INE / Pasaporte):** Del titular (Persona Física) o Representante Legal (Persona Moral).
- [] **Comprobante de Domicilio del Local:** Recibo de luz, agua o predial reciente (no mayor a 3 meses).
- [] **Contrato de Arrendamiento o Título de Propiedad:** Copia del contrato firmado del local comercial.
- [] **Fotografías del Local:** Toma interior y exterior (incluyendo la fachada con el letrero comercial).

- [] **Dictamen de Uso de Suelo:** Validación de que la zona geográfica permite la actividad comercial propuesta.

3. Pasos a Seguir (Ruta Guiada en la App)

- **Paso 1: Clasificación del Giro Comercial (Nivel de Riesgo)**
 - **Bajo Riesgo (Comercio / Servicios sin alcohol ni alimentos procesados):** Oficinas, tiendas de ropa, papelerías, consultorios. Solo requieren un **Aviso de Apertura Inmediata**.
 - **Medio/Alto Riesgo (Alimentos, bebidas, salud o afluencia masiva):** Restaurantes, bares, gimnasios, clínicas. Requieren **Licencia de Funcionamiento** previa e inspección.
- **Paso 2: Obtención del Uso de Suelo Comercial**
 - Consulta en la ventanilla única de tu municipio o en su portal de desarrollo urbano.
 - *Tip de la App:* Si el local ya operaba con un giro comercial similar anteriormente, el trámite suele ser una revalidación directa o traspaso.
- **Paso 3: Visto Bueno de Protección Civil**
 - Implementa las medidas básicas de seguridad en tu local:
 - Extintor vigente (mínimo de 4.5 kg, tipo ABC).
 - Señalética oficial (rutas de evacuación, salida de emergencia, qué hacer en caso de sismo).
 - Botiquín de primeros auxilios equipado.
 - Detector de humo funcional.
- **Paso 4: Trámite COFEPRIS / Sanitario (Solo si aplica)**
 - *Exclusivo para giro de Alimentos, Bebidas o Cosmética/Salud:* Ingresa el **Aviso de Funcionamiento Sanitario** ante la COFEPRIS o la Secretaría de Salud Estatal (trámite gratuito en línea).
- **Paso 5: Registro y Emisión del Permiso**
 - Presenta el expediente en la ventanilla del municipio o en la plataforma digital local.
 - Descarga e imprime tu **Cédula o Licencia de Funcionamiento**.
 - **Obligación legal:** Debes colocar la licencia física en un lugar visible al público dentro del local.

4. Errores Comunes a Evitar

- **Operar sin el visto bueno de Protección Civil:** Es la causa #1 de clausuras preventivas por parte de las autoridades municipales.
- **Firmar un contrato de arrendamiento sin revisar el Uso de Suelo:** Arrendar un inmueble con uso de suelo exclusivamente "Habitacional" impedirá obtener la licencia comercial.

5. Siguiendo Paso Recomendado

Con la e.firma del SAT (Trámite #1), la marca en el IMPI (Trámite #2) y la Licencia Municipal (Trámite #3), el negocio está 100% formalizado. El siguiente paso en la app es la **Apertura de la Cuenta Bancaria Empresarial** y el **Módulo de Creación de Procesos Operativos**.

Para la **Opción B (Diseño y Prototipado)**, estructuraremos la arquitectura visual de las **4 pantallas clave del MVP** para que puedas maquetarlas directamente en **Figma**, **FlutterFlow** o entregárselas a un diseñador UX/UI.

1. Pantalla de Bienvenida y Onboarding (Wizard de 5 Pasos)

Objetivo: Capturar los datos del emprendedor y generar de inmediato su ruta personalizada sin abrumarlo.

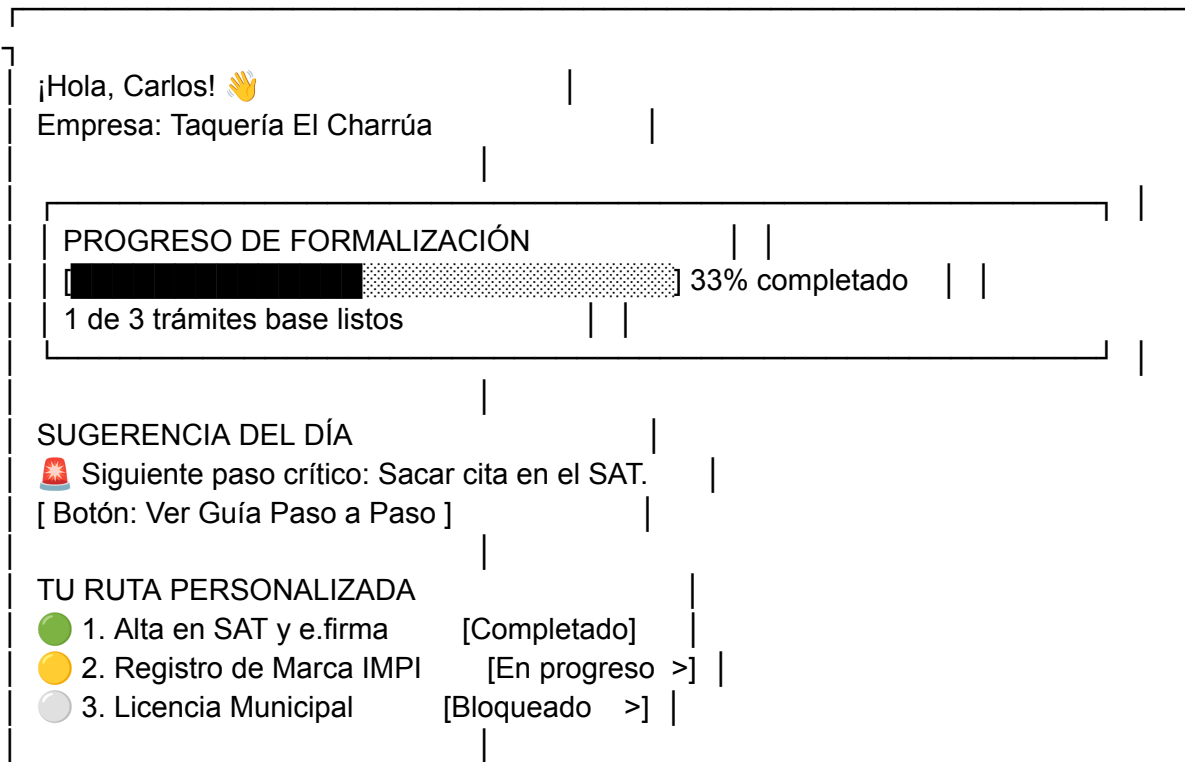
- **Componentes Visuales:**
 - **Barra de Progreso:** Superior fija (**Paso X de 5**).
 - **Tarjeta de Pregunta:** Fondo blanco con sombra suave y tipografía grande.
 - **Opciones en Tarjetas Seleccionables:** Botones grandes con ícono + texto (ej. *Opción A: Persona Física / RESICO*).
 - **Navegación:** Botón inferior primario *"Continuar"* (deshabilitado hasta seleccionar una opción) y botón secundario *"Atrás"*.

2. Pantalla Principal (Dashboard del Emprendedor)

Objetivo: Ser el centro de mando visual donde el usuario ve su avance global y su siguiente acción obligatoria.

<https://encrypted-tbn1.gstatic.com/licensed-image?q=tbn:ANd9GcTh-uur75cLVMJk26SV1nJ76u6VHRbWFkc4uTDIPjk3vDSy2ReWPSfTs0WvMFCp7bg4JfmeP7SbF534UMw>

Plaintext



¿NECESITAS AYUDA PROFESIONAL?

 Lic. Roberto Gómez - Especialista en IMPI

 4.9 (24 asesorías) | \$450 MXN / sesión

[Botón: Agendar Cita]

3. Pantalla de Detalle del Trámite (Visor de Guía)

Objetivo: Entregar la información de los trámites (#1 SAT, #2 IMPI, #3 Municipal) de forma accionable y libre de paja burocrática.

- **Estructura por Pestañas (Tabs) en la parte superior:**
 - **Requisitos:** Lista de verificación con casillas interactiva (*Checklist* que el usuario puede ir marcando conforme los junta).
 - **Pasos:** Secuencia numerada vertical con los enlaces oficiales destacados.
 - **Errores Comunes:** Bloque de advertencia destacado en color amarillo/alerta.
- **Componente de Acción Fijo (Bottom Bar):**
 - Botón primario: *"Marcar Trámite como Completado"*.
 - Botón secundario/icono: *"Tengo dudas con este trámite (Hablar con un Asesor)"*.

4. Pantalla de Perfil de Asesor y Agendamiento

Objetivo: Convertir al usuario que necesita ayuda en una cita pagada con un especialista en 3 clics.

- **Encabezado:** Fotografía del profesional, cédula profesional visible con badge de *"Verificado"*, tarifa por hora en MXN y especialidad (*SAT/RESICO, IMPI, Permisos Locales*).
- **Calendario Interactivo:** Selector de días horizontales + Bloques de horarios disponibles (*09:00 AM, 11:00 AM, 04:00 PM*).
- **Área de Nota Previa:** Caja de texto libre: *"Describe brevemente en qué necesitas ayuda para esta sesión"*.
- **Resumen de Cobro:** Desglose claro del costo (Costo sesión + IVA) y selector de método de pago (Tarjeta / Mercado Pago / Apple Pay).

Este es el archivo JSON listo para importar en tu base de datos (por ejemplo, en **PostgreSQL, Firebase** o **Supabase**).

Los identificadores (*id*) están estructurados en formato **UUID** para alinearse con la tabla **steps_catalog** que diseñamos previamente.

JSON


```
[
  {
    "id": "11111111-1111-4111-a111-111111111111",
    "category": "sat_fiscal",
    "title": "Alta en el RFC y Obtención de la e.firma",
    "description": "Tramita tu firma electrónica avanzada ante el SAT. Es indispensable para facturar, emitir sellos digitales y realizar trámites fiscales oficiales.",
    "required_for_entity": "all",
    "order_index": 1,
    "official_link": "https://citas.sat.gob.mx",
    "estimated_cost_mxn": 0,
    "modality": "Mixta (Preinscripción en línea y cita presencial)",
    "requirements": [
      "Identificación oficial vigente (INE o Pasaporte) en original",
      "Comprobante de domicilio fiscal no mayor a 3 meses",
      "CURP descargada en formato reciente",
      "Unidad de memoria extraíble (USB) preferentemente vacía",
      "Correo electrónico de uso personal"
    ],
    "steps": [
      {
        "step_number": 1,
        "title": "Preinscripción en el RFC",
        "detail": "Ingresa al portal oficial del SAT, captura tus datos personales y domicilio fiscal para generar tu acuse de preinscripción."
      },
      {
        "step_number": 2,
        "title": "Generar Cita o ingresar a Fila Virtual",
        "detail": "Entra a citas.sat.gob.mx, selecciona el módulo más cercano y elige el trámite 'e.firma de Persona Física'. Si no hay citas, únete a la Fila Virtual."
      },
      {
        "step_number": 3,
        "title": "Asistir a la cita presencial",
        "detail": "Acude 15 minutos antes con tus documentos y tu USB. Capturarán tus datos biométricos (huellas, iris y fotografía)."
      },
      {
        "step_number": 4,
        "title": "Resguardo de archivos digitales",
        "detail": "El personal guardará en tu USB los archivos .cer, .key y definirás tu contraseña de acceso. Respáldalos en un lugar seguro."
      }
    ],
    "common_errors": [
      "Llevar un comprobante de domicilio que no coincida exactamente con la dirección registrada en la preinscripción.",
    ]
  }
]
```

"Olvidar la memoria USB física el día de la cita (el SAT no envía los archivos por correo)."

```
]
},
{
  "id": "22222222-2222-4222-a222-222222222222",
  "category": "brand_impi",
  "title": "Búsqueda Fonética y Registro de Marca",
  "description": "Obtén el derecho exclusivo de uso de tu nombre comercial y logotipo en todo México por 10 años ante el IMPI.",
  "required_for_entity": "all",
  "order_index": 2,
  "official_link": "https://marcanet.impi.gob.mx",
  "estimated_cost_mxn": 3126.41,
  "modality": "100% En línea",
  "requirements": [
    "Nombre exacto de la marca o diseño de logo (.jpg o .png)",
    "Clasificación de Niza identificada (Giro comercial)",
    "e.firma del SAT o Cuenta PASE IMPI",
    "Tarjeta de crédito/débito para el pago de derechos"
  ],
  "steps": [
    {
      "step_number": 1,
      "title": "Búsqueda Fonética en MARCANET",
      "detail": "Ingresa a MARCANET y verifica que no existan marcas idénticas o similares registradas en tu misma clase de servicio o producto."
    },
    {
      "step_number": 2,
      "title": "Crear Cuenta PASE",
      "detail": "Registra tu usuario en el portal Tu Cuenta PASE del IMPI con tu CURP o e.firma."
    },
    {
      "step_number": 3,
      "title": "Llenado de solicitud de Marca en Línea",
      "detail": "Selecciona si es marca Nominativa, Innominada o Mixta. Captura los datos del titular y adjunta el logotipo si aplica."
    },
    {
      "step_number": 4,
      "title": "Pago y firma de la solicitud",
      "detail": "Paga la tarifa oficial mediante tarjeta o transferencia SPEI y firma el trámite con tu e.firma del SAT."
    }
  ],
  "common_errors": [
```

"Pagar el trámite sin realizar la búsqueda fonética previa (el IMPI no reembolsa en caso de rechazo).",

"Clasificar incorrectamente el giro en la Clasificación de Niza."

]

},

{

"id": "33333333-3333-4333-a333-333333333333",

"category": "municipal_permits",

"title": "Licencia de Funcionamiento y Permisos Locales",

"description": "Aviso o licencia comercial requerida por la alcaldía o municipio para operar un establecimiento físico sin riesgo de clausura.",

"required_for_entity": "all",

"order_index": 3,

"official_link": null,

"estimated_cost_mxn": 0,

"modality": "Presencial o Ventanilla Única Digital Municipal",

"requirements": [

"Constancia de Situación Fiscal (SAT) actualizada",

"Identificación oficial vigente del titular o representante legal",

"Comprobante de domicilio del local y Contrato de Arrendamiento",

"Dictamen de Uso de Suelo comercial",

"Fotografías del interior y exterior de la fachada del local"

],

"steps": [

{

"step_number": 1,

"title": "Determinar el Nivel de Riesgo del Giro",

"detail": "Identifica si tu negocio es de Bajo Riesgo (Aviso de Apertura Inmediata) o Alto Riesgo/Alimentos (Licencia previa)."

},

{

"step_number": 2,

"title": "Validación de Uso de Suelo",

"detail": "Verifica en la ventanilla de Desarrollo Urbano que el inmueble permita la actividad comercial deseada."

},

{

"step_number": 3,

"title": "Equipamiento de Protección Civil",

"detail": "Instala extintor tipo ABC vigente, señalética oficial de evacuación, detector de humo y botiquín de primeros auxilios."

},

{

"step_number": 4,

"title": "Trámite COFEPRIS (Solo Alimentos/Salud)",

"detail": "Si vendes alimentos o productos de salud, ingresa el Aviso de Funcionamiento Sanitario gratuito en línea."

},

```

{
  "step_number": 5,
  "title": "Emisión y Colocación de Licencia",
  "detail": "Ingresa el expediente en el municipio, descarga tu cédula de funcionamiento y colócala en un lugar visible en el local."
},
],
"common_errors": [
  "Operar el establecimiento sin los elementos básicos de Protección Civil.",
  "Firmar contrato de arrendamiento en un inmueble con uso de suelo exclusivamente habitacional."
]
}
]

```

Guía de Estilos y Design System Base para maquetar la aplicación en **Figma** o configurar las constantes en **FlutterFlow**.

Esta propuesta visual está optimizada para transmitir **confianza, claridad institucional y profesionalismo** (clave en temas legales, fiscales y de dinero en México), manteniendo una estética moderna e intuitiva.

1. Paleta de Colores (Color Palette)

Colores Primarios & Marca

- **Primary / Trust Blue (#1E3A8A)**: Azul marino profundo. Transmite seguridad, solidez fiscal e institucional. Usado en encabezados, botones primarios y barra de navegación.
- **Secondary / Action Emerald (#059669)**: Verde esmeralda. Representa avance, éxito y dinamismo financiero. Usado en indicadores de progreso, etiquetas de "Completado" y botones de confirmación/pago.

Colores Neutros (Superficies & Texto)

- **Background / Canvas (#F8FAFC)**: Gris ultra claro. Disminuye la fatiga visual y da sensación de orden y limpieza.
- **Surface / Cards (FFFFFF)**: Blanco puro. Para tarjetas, contenedores de trámites y modales.
- **Text Main (#0F172A)**: Azul slate oscuro. Alto contraste para lectura cómoda de guías largas (reemplaza al negro puro para una estética más pulida).
- **Text Muted / Borders (#64748B / #E2E8F0)**: Grises medios para subtítulos, iconos secundarios y divisores.

Colores de Estado (Feedback / Semáforo)

- **Success (#10B981)**: Trámites completados o búsqueda fonética viables.
- **Warning / Alert (#F59E0B)**: Trámites en progreso, avisos del SAT o advertencias.
- **Error / Danger (#EF4444)**: Campos requeridos faltantes o riesgo de homonimia en IMPI.

2. Tipografía (Typography)

Se recomienda usar **Inter** o **Plus Jakarta Sans** (ambas gratuitas en Google Fonts y nativas en Figma/FlutterFlow) por su excelente legibilidad en pantallas móviles pequeñas.

Rol	Fuente / Peso	Tamaño	Line Height	Uso Principal
Heading 1	SemiBold (600)	24 px	32 px	Títulos principales de pantalla (Dashboard, Nombre de Trámite)
Heading 2	SemiBold (600)	18 px	24 px	Títulos de sección (ej: "Requisitos", "Perfil del Asesor")
Subtitle / Label	Medium (500)	14 px	20 px	Nombres de campos, botones, etiquetas de estado
Body Text	Regular (400)	14 px	22 px	Párrafos instructivos, explicaciones de pasos
Caption / Small	Regular (400)	12 px	16 px	Fechas, costos secundarios, notas de pie

3. Sistema de Espaciado y Bordes

- **Malla base (Grid System):** Red de **8px** (Espaciados sugeridos: **8px**, **16px**, **24px**, **32px**).
- **Padding lateral de pantalla:** **20px** constante en los bordes izquierdo y derecho.
- **Radio de bordes (Border Radius):**
 - **Tarjetas y Modales:** **16px** (*esquinas suavizadas modernas*).
 - **Botones e Inputs:** **12px**.
 - **Badges y Etiquetas:** **20px** (*Pill shape / cápsula*).
- **Sombras (Elevation / Box Shadows):**
 - **Soft Shadow:** **0px 4px 12px rgba(15, 23, 42, 0.05)** para tarjetas que flotan sobre el fondo **#F8FAFC**.

4. Guía de Componentes UI Básicos

A. Botones (Buttons)

- **Primario:** Fondo #1E3A8A, Texto Blanco, Alto 48px o 52px (área táctil cómoda en móvil).
- **Secundario / Outline:** Fondo Transparente, Borde 1.5px #1E3A8A, Texto #1E3A8A.
- **Éxito / Pago:** Fondo #059669, Texto Blanco (Para "Agendar y Pagar" o "Finalizar Trámite").

B. Tarjeta de Trámite (Task Card)

- **Estado Pendiente/En Curso:** Fondo blanco, borde izquierdo de 4px en color #F59E0B (Warning) o #059669 (Success).
- **Elementos internos:** Título en Heading 2, Tag de categoría (ej: "SAT", "IMPI") en la esquina superior derecha, e ícono de flecha chevron-right.

C. Barra de Progreso (Progress Indicator)

- **Contenedor:** Fondo #E2E8F0 con alto de 8px y borde redondeado (4px).
- **Relleno activo:** Degradado horizontal de #1E3A8A a #059669.

Este es el script SQL completo y ejecutable para PostgreSQL / Supabase.

Incluye la creación de la tabla `steps_catalog` (con soporte nativo para JSONB en los campos de listas), los tipos enumerados para validación de datos y la inserción de los tres trámites base en una sola transacción.

SQL

```
-- 1. Habilitar extensión para generación de UUIDs (si no está activa)
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

```
-- 2. Crear los tipos ENUM para categorías y entidades requeridas
```

```
DO $$ BEGIN
    CREATE TYPE step_category_enum AS ENUM (
        'sat_fiscal',
        'brand_impi',
        'municipal_permits',
        'banking',
        'processes'
    );
EXCEPTION
    WHEN duplicate_object THEN null;
END $$;
```

```
DO $$ BEGIN
    CREATE TYPE required_entity_enum AS ENUM (
        'all',
        'persona_fisica',
        'persona_moral'
    );
EXCEPTION
    WHEN duplicate_object THEN null;
```

END \$\$;

-- 3. Crear la tabla steps_catalog

```
CREATE TABLE IF NOT EXISTS steps_catalog (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  category step_category_enum NOT NULL,  
  title VARCHAR(200) NOT NULL,  
  description TEXT NOT NULL,  
  required_for_entity required_entity_enum DEFAULT 'all',  
  order_index INT NOT NULL,  
  official_link VARCHAR(255),  
  estimated_cost_mxn NUMERIC(10, 2) DEFAULT 0.00,  
  modality VARCHAR(100) NOT NULL,  
  requirements JSONB NOT NULL DEFAULT '[]::jsonb',  
  steps JSONB NOT NULL DEFAULT '[]::jsonb',  
  common_errors JSONB NOT NULL DEFAULT '[]::jsonb',  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

-- 4. Inserción de los 3 trámites principales (UPSERT)

```
INSERT INTO steps_catalog (  
  id,  
  category,  
  title,  
  description,  
  required_for_entity,  
  order_index,  
  official_link,  
  estimated_cost_mxn,  
  modality,  
  requirements,  
  steps,  
  common_errors  
) VALUES  
(  
  '11111111-1111-4111-a111-111111111111',  
  'sat_fiscal',  
  'Alta en el RFC y Obtención de la e.firma',  
  'Tramita tu firma electrónica avanzada ante el SAT. Es indispensable para facturar, emitir  
sellos digitales y realizar trámites fiscales oficiales.',  
  'all',  
  1,  
  'https:// citas.sat.gob.mx',  
  0.00,  
  'Mixta (Preinscripción en línea y cita presencial)',  
  '[  
    "Identificación oficial vigente (INE o Pasaporte) en original",
```

```

        "Comprobante de domicilio fiscal no mayor a 3 meses",
        "CURP descargada en formato reciente",
        "Unidad de memoria extraíble (USB) preferentemente vacía",
        "Correo electrónico de uso personal"
    ]::jsonb,
    [
        {"step_number": 1, "title": "Preinscripción en el RFC", "detail": "Ingresa al portal oficial del SAT, captura tus datos personales y domicilio fiscal para generar tu acuse de preinscripción."},
        {"step_number": 2, "title": "Generar Cita o ingresar a Fila Virtual", "detail": "Entra a citas.sat.gob.mx, selecciona el módulo más cercano y elige el trámite 'e.firma de Persona Física'. Si no hay citas, únete a la Fila Virtual."},
        {"step_number": 3, "title": "Asistir a la cita presencial", "detail": "Acude 15 minutos antes con tus documentos y tu USB. Capturarán tus datos biométricos (huellas, iris y fotografía)."},
        {"step_number": 4, "title": "Resguardo de archivos digitales", "detail": "El personal guardará en tu USB los archivos .cer, .key y definirás tu contraseña de acceso. Respáldalos en un lugar seguro."}
    ]::jsonb,
    [
        "Llevar un comprobante de domicilio que no coincida exactamente con la dirección registrada en la preinscripción.",
        "Olvidar la memoria USB física el día de la cita (el SAT no envía los archivos por correo)."
    ]::jsonb
),
(
    '22222222-2222-4222-a222-222222222222',
    'brand_imp',
    'Búsqueda Fonética y Registro de Marca',
    'Obtén el derecho exclusivo de uso de tu nombre comercial y logotipo en todo México por 10 años ante el IMPI.',
    'all',
    2,
    'https://marcanet.impi.gob.mx',
    3126.41,
    '100% En línea',
    [
        "Nombre exacto de la marca o diseño de logo (.jpg o .png)",
        "Clasificación de Niza identificada (Giro comercial)",
        "e.firma del SAT o Cuenta PASE IMPI",
        "Tarjeta de crédito/débito para el pago de derechos"
    ]::jsonb,
    [
        {"step_number": 1, "title": "Búsqueda Fonética en MARCANET", "detail": "Ingresa a MARCANET y verifica que no existan marcas idénticas o similares registradas en tu misma clase de servicio o producto."},
        {"step_number": 2, "title": "Crear Cuenta PASE", "detail": "Registra tu usuario en el portal Tu Cuenta PASE del IMPI con tu CURP o e.firma."},
    ]

```


{"step_number": 3, "title": "Llenado de solicitud de Marca en Línea", "detail": "Selecciona si es marca Nominativa, Innominada o Mixta. Captura los datos del titular y adjunta el logotipo si aplica."},

 {"step_number": 4, "title": "Pago y firma de la solicitud", "detail": "Paga la tarifa oficial mediante tarjeta o transferencia SPEI y firma el trámite con tu e.firma del SAT."}

]::jsonb,

 [

 "Pagar el trámite sin realizar la búsqueda fonética previa (el IMPI no reembolsa en caso de rechazo).",

 "Clasificar incorrectamente el giro en la Clasificación de Niza."

]::jsonb

),

(

 '33333333-3333-4333-a333-333333333333',

 'municipal_permits',

 'Licencia de Funcionamiento y Permisos Locales',

 'Aviso o licencia comercial requerida por la alcaldía o municipio para operar un establecimiento físico sin riesgo de clausura.',

 'all',

 3,

 NULL,

 0.00,

 'Presencial o Ventanilla Única Digital Municipal',

 [

 "Constancia de Situación Fiscal (SAT) actualizada",

 "Identificación oficial vigente del titular o representante legal",

 "Comprobante de domicilio del local y Contrato de Arrendamiento",

 "Dictamen de Uso de Suelo comercial",

 "Fotografías del interior y exterior de la fachada del local"

]::jsonb,

 [

 {"step_number": 1, "title": "Determinar el Nivel de Riesgo del Giro", "detail": "Identifica si tu negocio es de Bajo Riesgo (Aviso de Apertura Inmediata) o Alto Riesgo/Alimentos (Licencia previa)."},

 {"step_number": 2, "title": "Validación de Uso de Suelo", "detail": "Verifica en la ventanilla de Desarrollo Urbano que el inmueble permita la actividad comercial deseada."},

 {"step_number": 3, "title": "Equipamiento de Protección Civil", "detail": "Instala extintor tipo ABC vigente, señalética oficial de evacuación, detector de humo y botiquín de primeros auxilios."},

 {"step_number": 4, "title": "Trámite COFEPRIS (Solo Alimentos/Salud)", "detail": "Si vendes alimentos o productos de salud, ingresa el Aviso de Funcionamiento Sanitario gratuito en línea."},

 {"step_number": 5, "title": "Emisión y Colocación de Licencia", "detail": "Ingresa el expediente en el municipio, descarga tu cédula de funcionamiento y colócala en un lugar visible en el local."}

]::jsonb,

 [

 "Operar el establecimiento sin los elementos básicos de Protección Civil.",

"Firmar contrato de arrendamiento en un inmueble con uso de suelo exclusivamente habitacional."

```
]::jsonb
)
ON CONFLICT (id) DO UPDATE SET
  title = EXCLUDED.title,
  description = EXCLUDED.description,
  estimated_cost_mxn = EXCLUDED.estimated_cost_mxn,
  requirements = EXCLUDED.requirements,
  steps = EXCLUDED.steps,
  common_errors = EXCLUDED.common_errors,
  updated_at = CURRENT_TIMESTAMP;
```

-- 5. Habilitar Row Level Security (RLS) en Supabase para lectura pública
ALTER TABLE steps_catalog ENABLE ROW LEVEL SECURITY;

```
CREATE POLICY "Lectura pública del catálogo de trámites"
ON steps_catalog
FOR SELECT
USING (true);
```

script SQL ejecutable para configurar las tablas del **Marketplace de Asesores, Citas y Pagos en Supabase / PostgreSQL**.

Incluye la vinculación automática con el sistema de autenticación de Supabase ([auth.users](#)), manejo de estados, índices para acelerar búsquedas y políticas de **Row Level Security (RLS)** para proteger los datos financieros y las agendas.

SQL

```
-- 1. Crear ENUMs para Especialidades y Estatus
DO $$ BEGIN
  CREATE TYPE advisor_specialty_enum AS ENUM (
    'fiscal_contable',
    'legal_impi',
    'branding',
    'procesos_operativos'
  );
EXCEPTION
  WHEN duplicate_object THEN null;
END $$;
```

```
DO $$ BEGIN
  CREATE TYPE appointment_status_enum AS ENUM (
    'pending_payment',
    'scheduled',
    'completed',
    'cancelled',
  );
EXCEPTION
  WHEN duplicate_object THEN null;
END $$;
```

```

        'no_show'
    );
EXCEPTION
    WHEN duplicate_object THEN null;
END $$;

```

-- 2. Tabla de Asesores Verificados

```

CREATE TABLE IF NOT EXISTS advisors (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL UNIQUE REFERENCES auth.users(id) ON DELETE
    CASCADE,
    specialty advisor_specialty_enum NOT NULL,
    professional_id VARCHAR(50) NOT NULL, -- Cédula Profesional / Folio
    bio TEXT,
    hourly_rate NUMERIC(10, 2) NOT NULL CHECK (hourly_rate >= 0),
    is_verified BOOLEAN DEFAULT FALSE,
    rating_avg NUMERIC(3, 2) DEFAULT 0.00 CHECK (rating_avg >= 0 AND rating_avg <=
5.00),
    total_reviews INT DEFAULT 0,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- 3. Tabla de Citas y Reservas

```

CREATE TABLE IF NOT EXISTS appointments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES auth.users(id) ON DELETE SET NULL, -- Emprendedor
    advisor_id UUID NOT NULL REFERENCES advisors(id) ON DELETE CASCADE,
    scheduled_at TIMESTAMP WITH TIME ZONE NOT NULL,
    duration_minutes INT DEFAULT 60 CHECK (duration_minutes > 0),
    status appointment_status_enum DEFAULT 'pending_payment',
    user_notes TEXT, -- Nota del emprendedor previa a la llamada
    meeting_url VARCHAR(255), -- Enlace dinámico generado (Daily.co / Google Meet)
    amount_paid NUMERIC(10, 2) NOT NULL DEFAULT 0.00,
    platform_fee NUMERIC(10, 2) DEFAULT 0.00, -- Comisión retenida por la app
    stripe_payment_intent_id VARCHAR(255),
    rating INT CHECK (rating >= 1 AND rating <= 5),
    review_comment TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- 4. Índices para Optimización de Consultas Frecuentes

```

CREATE INDEX IF NOT EXISTS idx_advisors_specialty ON advisors(specialty) WHERE
is_verified = TRUE;
CREATE INDEX IF NOT EXISTS idx_appointments_user ON appointments(user_id);
CREATE INDEX IF NOT EXISTS idx_appointments_advisor ON appointments(advisor_id);

```

```
CREATE INDEX IF NOT EXISTS idx_appointments_scheduled ON
appointments(scheduled_at);
```

-- 5. Configuración de Row Level Security (RLS)

-- Habilitar RLS

```
ALTER TABLE advisors ENABLE ROW LEVEL SECURITY;
ALTER TABLE appointments ENABLE ROW LEVEL SECURITY;
```

-- POLÍTICAS PARA ADVISORS

-- Lectura: Cualquiera puede ver perfiles de asesores verificados

```
CREATE POLICY "Asesores verificados son públicos"
```

```
ON advisors FOR SELECT
```

```
USING (is_verified = TRUE OR auth.uid() = user_id);
```

-- Actualización: Solo el propio asesor puede editar su perfil

```
CREATE POLICY "Asesor edita su propio perfil"
```

```
ON advisors FOR UPDATE
```

```
USING (auth.uid() = user_id)
```

```
WITH CHECK (auth.uid() = user_id);
```

-- POLÍTICAS PARA APPOINTMENTS

-- Lectura: Solo el emprendedor que compró o el asesor asignado pueden ver la cita

```
CREATE POLICY "Usuarios ven sus propias citas"
```

```
ON appointments FOR SELECT
```

```
USING (
```

```
    auth.uid() = user_id OR
```

```
    auth.uid() IN (SELECT user_id FROM advisors WHERE id = advisor_id)
```

```
);
```

-- Inserción: Solo usuarios autenticados pueden crear citas para sí mismos

```
CREATE POLICY "Emprendedor crea reserva"
```

```
ON appointments FOR INSERT
```

```
WITH CHECK (auth.uid() = user_id);
```

-- Actualización: Modificar la cita (cancelar o calificar) solo por los involucrados

```
CREATE POLICY "Involucrados actualizan la cita"
```

```
ON appointments FOR UPDATE
```

```
USING (
```

```
    auth.uid() = user_id OR
```

```
    auth.uid() IN (SELECT user_id FROM advisors WHERE id = advisor_id)
```

```
);
```

El **Módulo de Negocios Ya Establecidos** tiene como objetivo solucionar el caos operativo, la falta de estandarización y los problemas de rotación de personal en PyMES. Se enfoca

en transformar tareas informales en **Procedimientos Operativos Estándar (SOPs)** claros, descargables y ejecutables.

1. Estructura General del Módulo (Categorías Principales)

El módulo se organiza en 4 pilares fundamentales para la madurez de una empresa:

1. **Gestión de Personal y RRHH (Liderazgo y Equipo)**
2. **Finanzas y Control de Caja (Salud Financiera)**
3. **Operaciones y Calidad (SOPs / Procedimientos)**
4. **Ventas y Atención al Cliente (Ingresos y Retención)**

2. Catálogo de Plantillas y Descargables Clave

Cada elemento incluye un formato interactivo editable en la app y una versión exportable en PDF/Excel adaptada a la legislación mexicana (como la Ley Federal del Trabajo).

A. Pilar de Recursos Humanos

- **Contrato Individual de Trabajo (Adaptado a la LFT en México):**
 - Cláusulas de periodo de prueba, jornada laboral, confidencialidad y rescisión.
- **Perfil y Descripción de Puesto:**
 - Plantilla para definir responsabilidades, KPIs clave y perfil requerido antes de contratar.
- **Checklist de Onboarding / Inducción de Personal:**
 - Lista de verificación para los primeros 7 días del nuevo empleado (entrega de equipo, firma de contrato, capacitación).

B. Pilar de Finanzas y Operaciones

- **Control de Flujo de Caja (Cash Flow):**
 - Plantilla interactiva para registrar ingresos/egresos diarios, categorizar gastos fijos/variables y prever liquidez a 30 días.
- **Manual de Apertura y Cierre de Establecimiento:**
 - Checklist de verificación para empleados (arqueo de caja, encendido/apagado de equipos, medidas de seguridad).
- **Control de Inventario e Insumos:**
 - Registro de stock mínimo, punto de reorden y control de mermas/pérdidas.

C. Pilar de Ventas y Legal-Comercial

- **Aviso de Privacidad Simplificado y Integral (Ley LFPDPPP):**
 - Obligatorio en México para negocios que recaban datos personales de clientes (teléfono, correo, tarjeta).
- **Guía de Manejo de Quejas y Protocolo de Servicio:**
 - Flujograma de decisión para el personal ante clientes insatisfechos, devoluciones o garantías.

3. Generador Interactivo de SOPs (Standard Operating Procedures)

Para que el dueño del negocio no tenga que escribir documentos desde cero, la app incluirá un **Generador de Procedimientos Asistido**, con la siguiente estructura de datos:

JSON

```
{
  "sop_id": "sop-001-cierre-caja",
  "category": "operations",
  "title": "Procedimiento de Arqueo y Cierre de Caja Chica",
  "department": "Administración / Ventas",
  "estimated_duration_minutes": 20,
  "roles_responsible": ["Cajero", "Gerente de Sucursal"],
  "prerequisites": [
    "Terminal punto de venta (TPV) cerrada",
    "Bitácora física o digital de ventas del día"
  ],
  "steps": [
    {
      "step_number": 1,
      "title": "Conteo de Efectivo Físico",
      "instruction": "Separar el fondo fijo de caja ($1,000 MXN) y contar el billete/moneda sobrante de las ventas del día."
    },
    {
      "step_number": 2,
      "title": "Corte de Terminales Bancarias",
      "instruction": "Imprimir el reporte de cierre en la TPV (Platzi, Clip, Banorte, Mercado Pago) y verificar el total de cobros con tarjeta."
    },
    {
      "step_number": 3,
      "title": "Registro en App y Depósito",
      "instruction": "Capturar los totales en la plantilla de Flujo de Caja y guardar el efectivo en la caja fuerte o bolsa de depósito."
    }
  ]
}
```

4. Modelo de Base de Datos para el Módulo de Procesos (SQL)

Script para integrar la tabla de plantillas y procesos descargables en la base de datos de Supabase/PostgreSQL:

SQL

```
-- Tabla para el catálogo de plantillas de procesos
CREATE TABLE IF NOT EXISTS process_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

        category VARCHAR(50) NOT NULL CHECK (category IN ('hr', 'finance', 'operations',
'sales_legal')),
        title VARCHAR(150) NOT NULL,
        description TEXT NOT NULL,
        file_type VARCHAR(20) CHECK (file_type IN ('excel', 'word', 'pdf_interactive',
'sop_generator')),
        download_url VARCHAR(255),
        is_pro_only BOOLEAN DEFAULT TRUE, -- Exclusivo para usuarios con suscripción
        downloads_count INT DEFAULT 0,
        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

-- Habilitar RLS

```
ALTER TABLE process_templates ENABLE ROW LEVEL SECURITY;
```

-- Lectura pública para usuarios autenticados

```
CREATE POLICY "Usuarios registrados ven las plantillas"
```

```
ON process_templates FOR SELECT
```

```
TO authenticated
```

```
USING (true);
```

el script SQL ejecutable para insertar el catálogo inicial de plantillas en la tabla **process_templates** de **Supabase / PostgreSQL**.

El script utiliza inserciones con resolución de conflictos (**ON CONFLICT**) para prevenir duplicados y asigna identificadores explícitos en formato UUID.

SQL

-- Inserción de plantillas iniciales para el Módulo de Negocios Establecidos

```
INSERT INTO process_templates (
```

```
    id,
```

```
    category,
```

```
    title,
```

```
    description,
```

```
    file_type,
```

```
    download_url,
```

```
    is_pro_only,
```

```
    downloads_count
```

```
) VALUES
```

```
(
```

```
    'a1b2c3d4-0001-4000-8000-000000000001',
```

```
    'hr',
```

```
    'Contrato Individual de Trabajo por Tiempo Indeterminado (LFT)',
```

```
    'Modelo de contrato laboral individual adaptado a la Ley Federal del Trabajo en México.
```

```
Incluye cláusulas de periodo de prueba, jornada, salario, confidencialidad y obligaciones patronales.',
```

```
    'word',
```

```

'https://storage.googleapis.com/tu-app-bucket/templates/hr/contrato_individual_trabajo_lft.docx',
  TRUE,
  0
),
(
  'a1b2c3d4-0002-4000-8000-000000000002',
  'finance',
  'Plantilla de Control de Flujo de Caja Mensual (Cash Flow)',
  'Archivo ejecutable en Excel para el control diario de ingresos, egresos, proyección de cuentas por cobrar/pagar y cálculo automático de liquidez operativa a 30 y 60 días.',
  'excel',

'https://storage.googleapis.com/tu-app-bucket/templates/finance/control_flujo_caja_pyme.xlsx',
  FALSE, -- Disponible en versión gratuita para atracción de usuarios
  0
),
(
  'a1b2c3d4-0003-4000-8000-000000000003',
  'sales_legal',
  'Aviso de Privacidad Integral y Simplificado (LFPDPPP)',
  'Formato legal editable para cumplir con la Ley Federal de Protección de Datos Personales en Posesión de los Particulares en México. Ideal para negocios con local físico, sitio web o recolección de datos de clientes.',
  'word',

'https://storage.googleapis.com/tu-app-bucket/templates/legal/aviso_privacidad_integral.docx',
  TRUE,
  0
),
(
  'a1b2c3d4-0004-4000-8000-000000000004',
  'operations',
  'Manual y Checklist de Apertura y Cierre de Establecimiento',
  'Procedimiento Operativo Estándar (SOP) con lista de verificación interactiva para colaboradores. Incluye arqueo de caja, verificación de inventarios clave y protocolo de seguridad.',
  'pdf_interactive',

'https://storage.googleapis.com/tu-app-bucket/templates/operations/sop_apertura_cierre_local.pdf',
  FALSE,
  0
)
ON CONFLICT (id) DO UPDATE SET

```



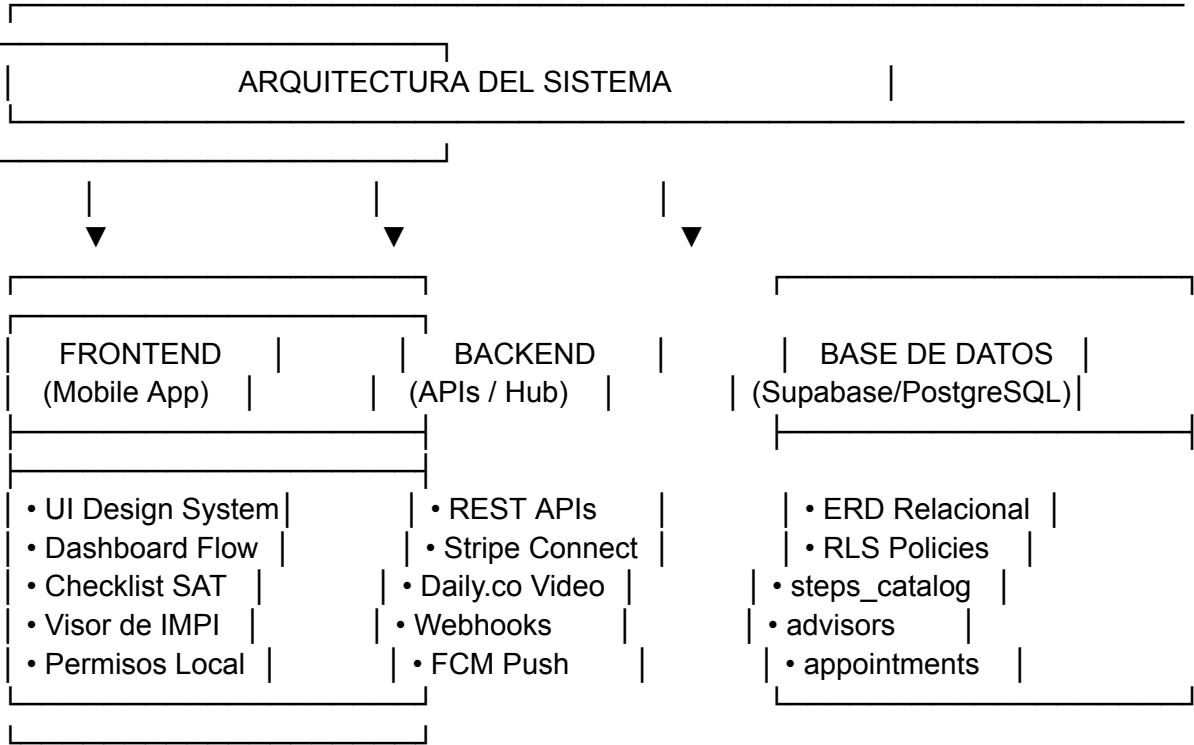
```
title = EXCLUDED.title,
description = EXCLUDED.description,
file_type = EXCLUDED.file_type,
download_url = EXCLUDED.download_url,
is_pro_only = EXCLUDED.is_pro_only;
```

Resumen Ejecutivo y Roadmap Consolidado del Proyecto

A continuación se consolidan todos los artefactos, arquitecturas y estrategias diseñados para el desarrollo del ecosistema digital enfocado en la formalización y optimización de negocios en México.

1. Resumen de Artefactos Diseñados

Plaintext



A. Base de Datos Relacional (PostgreSQL / Supabase)

- **Esquema completo:** Tablas estructuradas para **users**, **businesses**, **steps_catalog**, **user_progress**, **advisors**, **appointments** y **process_templates**.
- **Seguridad (RLS):** Políticas de nivel de fila activas para garantizar la privacidad de los usuarios, restringir el acceso a datos financieros y limitar las agendas únicamente a las partes involucradas.
- **Integración de Datos:** Scripts SQL y archivos JSON listos para sembrar el catálogo con los trámites clave en México (**SAT/e.firma**, **IMPI/Marcas**, **Permisos**

Municipales/Protección Civil) y plantillas operativas (Contrato LFT, Cash Flow, Aviso de Privacidad).

B. Lógica de Negocio y Endpoints API

- **Flujo de Marketplace de Asesores:**
 - Búsqueda y filtrado de profesionales por especialidad y tarifa.
 - Reserva con custodia temporal de fondos (*Escrow*) mediante **Stripe Connect**.
 - Generación dinámica de salas de videollamada con **Daily.co** o **Google Meet**.
 - Sistema de calificación y liberación automática de pagos (menos comisión del 15% - 20%).

C. Experiencia de Usuario y Guía de Estilos (Design System)

- **Identidad Visual:**
 - Azul Marino (#1E3A8A): Confianza e institucionalidad.
 - Verde Esmeralda (#059669): Acción y éxito financiero.
 - Gris Canvas (#F8F8F8): Lectura limpia y libre de distracciones.
- **Tipografía & Grid:** Sistema basado en la fuente **Inter / Plus Jakarta Sans**, retícula flexible de 8px y radio de esquinas de 12px/16px para tarjetas e insumos interactivos.

2. Roadmap Consolidado de Implementación (4 Semanas al MVP)

Semana	Enfocado en	Entregables Clave
Semana 1	Infraestructura & Setup	• Ejecución de scripts SQL en Supabase. • Configuración del proyecto móvil en Flutter / React Native. • Creación y verificación de cuenta corporativa en Stripe México.

Semana 2	Frontend & Módulo Guía	• Implementación de la guía interactiva (Trámites SAT, IMPI, Municipal). • Conexión del estado de progreso del usuario con el backend. • Maquetación de componentes del Design System.
Semana 3	Marketplace & Pagos	• Conexión de endpoints de agendamiento y disponibilidad. • Integración del SDK de Stripe para el checkout de asesorías. • Pruebas de generación de salas de video y Webhooks.
Semana 4	Reclutamiento & Beta	• Onboarding manual de los primeros 5-10 asesores verificados. • Pruebas de extremo a extremo con <i>beta testers</i> reales.

		• Publicación del MVP en tiendas de aplicaciones o Web App.
--	--	---

Con el diseño técnico y funcional completado, damos inicio a la **Semana 1 del Roadmap: Configuración de Infraestructura y Setup Base**.

Paso 1: Configurar el Proyecto en Supabase

Para desplegar la infraestructura de base de datos que diseñamos:

1. Ingresa a [Supabase](#) y crea un nuevo proyecto (puedes seleccionar la región **us-east-1** para óptima latencia con México).
2. Abre la sección **SQL Editor** en el panel lateral de Supabase.
3. Copia y ejecuta en orden los dos scripts SQL que generamos previamente:
 - **Script 1:** Estructura de usuarios, catálogo de trámites ([steps_catalog](#)) y plantillas de procesos ([process_templates](#)).
 - **Script 2:** Tablas del marketplace de asesores ([advisors](#)), citas ([appointments](#)) y sus políticas de seguridad (RLS).

Paso 2: Obtener las Credenciales de Conexión

En el panel de tu proyecto en Supabase, dirígete a **Project Settings > API** y resguarda las siguientes claves que necesitarás para configurar el backend o la app móvil:

- **Project** **URL:**
<https://your-project-ref.supabase.co>
- **anon / public key:** Clave segura para usar en el cliente (Flutter/React Native).
- **service_role key:** Clave secreta para operaciones administrativas en el backend (Node.js/Python).

Paso 3: Estructura del Archivo de Variables de Entorno (.env)

Crea un archivo **.env** en tu entorno de desarrollo local con la siguiente configuración base:

Fragmento de código

Supabase Configuration

SUPABASE_URL=https://tu-proyecto.supabase.co

SUPABASE_ANON_KEY=tu_anon_key_aqui

SUPABASE_SERVICE_ROLE_KEY=tu_service_role_key_aqui

Stripe México Configuration (Sandbox)

```
STRIPE_SECRET_KEY=sk_test_tu_stripe_secret_key
STRIPE_WEBHOOK_SECRET=whsec_tu_webhook_secret
```

```
# Video Call Service (Daily.co)
DAILY_API_KEY=tu_daily_api_key_aqui
```

Configurar **Stripe en México** para un modelo de Marketplace requiere habilitar la cuenta comercial para procesar pagos en **Pesos Mexicanos (MXN)**, configurar **Stripe Connect** para transferir fondos a los asesores y gestionar los **Webhooks** que confirman las citas cuando el pago es exitoso.

A continuación tienes la guía paso a paso para entorno de desarrollo (Modo Test).

Paso 1: Crear la Cuenta y Habilitar Stripe Connect

1. Ingresa a [Dashboard de Stripe](#) y crea una cuenta.
2. Asegúrate de que el país de la cuenta sea **México (MXN)**.
3. Activa el interruptor **"Modo de prueba" (Test mode)** en la esquina superior derecha.
4. En el menú lateral, ve a **Connect** y haz clic en **Comenzar (Get Started)**:
 - Selecciona el tipo de cuenta Connect: **Custom** o **Express** (recomendado **Express** por facilidad de onboarding de cuentas bancarias de los asesores en México).
5. Ve a [Configuración de la cuenta > Claves API](#) para obtener tus credenciales:
 - **Publishable key:** `pk_test_...`
 - **Secret key:** `sk_test_...`

Paso 2: Flujo de Cobro para Asesorías (Payment Intent con Capture Deferred o Application Fee)

Para el marketplace, el cobro de una cita se compone del costo total del asesor más la comisión de tu plataforma. Con **Stripe Connect Express**, creas un `PaymentIntent` donde tú recibes el pago y transfieres la parte correspondiente al asesor en una sola transacción:

JSON

```
{
  "amount": 50000,
  "currency": "mxn",
  "payment_method_types": ["card"],
  "application_fee_amount": 7500,
  "transfer_data": {
    "destination": "acct_1AsesorStripeID"
  },
  "metadata": {
    "appointment_id": "uuid-de-la-cita-en-supabase"
  }
}
```

```
}
```

(Nota: En Stripe, las cantidades en MXN se representan en centavos. \$500.00 MXN = 50000).

Paso 3: Configurar Escucha Local de Webhooks en Desarrollo

Durante el desarrollo local, Stripe necesita notificar a tu servidor backend cuando se complete un pago. Para esto usaremos la herramienta oficial **Stripe CLI**.

1. Instalar Stripe CLI

- **macOS (Homebrew):** `brew install stripe/stripe-cli/stripe`
- **Windows (Scoop):** `scoop bucket add stripe [https://github.com/stripe/scoop-stripe-cli.git](https://github.com/stripe/scoop-stripe-cli.git) / scoop install stripe`
- **Linux:** Descargar binario desde los lanzamientos oficiales de GitHub.

2. Iniciar sesión en Stripe CLI

Ejecuta en tu terminal:

```
Bash
stripe login
```

Te abrirá el navegador para autorizar la CLI con tu cuenta de Stripe.

3. Reenviar eventos a tu backend local

Asumiendo que tu servidor backend escucha localmente en el puerto 3000 en el endpoint `/api/webhooks/stripe`:

```
Bash
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

Al ejecutar este comando, la consola te devolverá una clave secreta para la firma del webhook:

```
Bash
> Ready! Your webhook signing secret is whsec_1234567890abcdef...
```

Copia ese `whsec_...` y agrégalo a tu archivo `.env`:

Fragmento de código
`STRIPE_WEBHOOK_SECRET=whsec_1234567890abcdef...`

Paso 4: Código del Endpoint Webhook (Node.js + Express / Supabase Edge Functions)

A continuación se muestra el controlador backend para validar la firma de Stripe y actualizar el estado de la cita en Supabase cuando el pago sea exitoso:

JavaScript

```
const express = require('express');
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
const { createClient } = require('@supabase/supabase-js');

const app = express();
const supabase = createClient(process.env.SUPABASE_URL,
process.env.SUPABASE_SERVICE_ROLE_KEY);

// IMPORTANTE: Stripe Webhook requiere el body raw (sin parsear a JSON previamente)
app.post('/api/webhooks/stripe', express.raw({ type: 'application/json' }), async (req, res) => {
  const sig = req.headers['stripe-signature'];
  let event;

  try {
    // 1. Verificar la firma del evento para prevenir spoofing
    event = stripe.webhooks.constructEvent(req.body, sig,
process.env.STRIPE_WEBHOOK_SECRET);
  } catch (err) {
    console.error(`Error en firma de Webhook: ${err.message}`);
    return res.status(400).send(`Webhook Error: ${err.message}`);
  }

  // 2. Procesar eventos clave
  switch (event.type) {
    case 'payment_intent.succeeded': {
      const paymentIntent = event.data.object;
      const appointmentId = paymentIntent.metadata.appointment_id;

      console.log(`Pago exitoso registrado para la cita: ${appointmentId}`);

      // 3. Actualizar el estatus de la cita en Supabase
      const { error } = await supabase
        .from('appointments')
        .update({
          status: 'scheduled',
          stripe_payment_intent_id: paymentIntent.id
        })
        .eq('id', appointmentId);
```

```

    if (error) {
      console.error(' Error actualizando cita en Supabase:', error);
      return res.status(500).send('Database Update Failed');
    }
    break;
  }

  case 'payment_intent.payment_failed': {
    const paymentIntent = event.data.object;
    const appointmentId = paymentIntent.metadata.appointment_id;

    console.log(` Pago fallido para la cita: ${appointmentId}`);

    await supabase
      .from('appointments')
      .update({ status: 'cancelled' })
      .eq('id', appointmentId);
    break;
  }

  default:
    console.log(` Evento no gestionado: ${event.type}`);
  }

  // Devolver respuesta HTTP 200 a Stripe
  res.json({ received: true });
});

app.listen(3000, () => console.log('Servidor corriendo en el puerto 3000'));

```

Paso 5: Probar un Pago Simulado

Para simular un evento de pago sin realizar un checkout completo en la interfaz de usuario, puedes disparar un evento directo con la CLI:

```

Bash
stripe trigger payment_intent.succeeded

```

Verás cómo el evento es enviado a tu terminal y procesado por tu servidor local actualizando la base de datos de Supabase.

La integración de la API de **Daily.co** permite generar salas de videoconferencia temporales, privadas y dinámicas para cada cita agendada, enlazándolas automáticamente con la confirmación de pago de Stripe.

Paso 1: Obtener las Credenciales de Daily.co

1. Regístrate en el panel de [Daily.co](https://daily.co).
2. Ve a la sección **Developers** en el menú lateral.
3. Copia tu **API Key** y agrégala a tu archivo de variables de entorno `.env`:
4. Fragmento de código

```
DAILY_API_KEY=tu_daily_api_key_aqui
```

- 5.
- 6.

Paso 2: Función Backend para Crear Salas Dinámicas

Para garantizar la privacidad y controlar el tiempo de uso, la sala se debe configurar con las siguientes propiedades clave:

- **Expiración (`exp`):** La sala se autodestruye automáticamente después de la hora programada.
- **Acceso Privado (`privacy: 'private'`):** Evita que extraños se unan sin un token de acceso generado por el servidor.
- **Límite de Participantes (`max_participants: 2`):** Exclusivo para el emprendedor y el asesor.

A continuación, la función en **Node.js** para interactuar con la REST API de Daily.co:

JavaScript

```
const fetch = require('node-fetch');
```

```
/**
 * Genera una sala de videollamada en Daily.co para una cita específica.
 * @param {string} appointmentId - UUID de la cita.
 * @param {Date} scheduledAt - Fecha y hora de inicio de la cita.
 * @param {number} durationMinutes - Duración en minutos (ej. 60).
 * @returns {Promise<string>} URL de la sala generada.
 */
async function createDailyRoom(appointmentId, scheduledAt, durationMinutes = 60) {
  // Calcular tiempo de expiración (hora de inicio + duración + 15 min de tolerancia)
  const startTimeSeconds = Math.floor(new Date(scheduledAt).getTime() / 1000);
  const expirationSeconds = startTimeSeconds + (durationMinutes + 15) * 60;

  const response = await fetch('https://api.daily.co/v1/rooms', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.DAILY_API_KEY}`
    },
    body: JSON.stringify({
      name: `cita-${appointmentId}`, // Nombre único para la sala
      privacy: 'private',           // Requiere token de acceso
    })
  });
}
```

```

    properties: {
      exp: expirationSeconds,    // Autodestrucción de la sala
      max_participants: 2,      // Límite de 2 personas (Asesor + Emprendedor)
      enable_chat: true,        // Activar chat de texto interno
      enable_screenshare: true,  // Permitir compartir pantalla
      eject_at_room_exp: true    // Desconectar a los usuarios al vencer el tiempo
    }
  })
});

const data = await response.json();

if (!response.ok) {
  throw new Error(`Error en Daily.co API: ${data.error || response.statusText}`);
}

return data.url; // Retorna la URL directa de la sala (ej. https://tu-dominio.daily.co/cita-uuid)
}

module.exports = { createDailyRoom };

```

Paso 3: Conectar la Creación de la Sala con el Webhook de Stripe

Debes actualizar el controlador del Webhook de Stripe que escribimos en el paso anterior. Cuando el evento `payment_intent.succeeded` confirme el pago, genera la sala en Daily.co y guarda la URL devuelta en la columna `meeting_url` de Supabase:

```

JavaScript
const { createDailyRoom } = require('./dailyService');

// Dentro del switch (event.type) en el Webhook de Stripe:
case 'payment_intent.succeeded': {
  const paymentIntent = event.data.object;
  const appointmentId = paymentIntent.metadata.appointment_id;

  // 1. Obtener los detalles de la cita desde Supabase
  const { data: appointment, error: fetchError } = await supabase
    .from('appointments')
    .select('scheduled_at, duration_minutes')
    .eq('id', appointmentId)
    .single();

  if (fetchError || !appointment) {
    console.error('Error al obtener la cita:', fetchError);
    return res.status(500).send('Appointment not found');
  }
}

```

```

// 2. Generar la sala en Daily.co
const meetingUrl = await createDailyRoom(
  appointmentId,
  appointment.scheduled_at,
  appointment.duration_minutes
);

// 3. Actualizar la cita en Supabase con la URL de la llamada y estatus confirmado
const { error: updateError } = await supabase
  .from('appointments')
  .update({
    status: 'scheduled',
    meeting_url: meetingUrl,
    stripe_payment_intent_id: paymentIntent.id
  })
  .eq('id', appointmentId);

if (updateError) {
  console.error('Error al guardar la URL en Supabase:', updateError);
  return res.status(500).send('Database Update Failed');
}

console.log(` Cita ${appointmentId} confirmada con sala: ${meetingUrl}`);
break;
}

```

Paso 4: Generar Tokens de Reunión (Meeting Tokens)

Dado que la sala se creó con `privacy: 'private'`, nadie puede entrar directamente con la URL. Para dar acceso al usuario y al asesor, tu backend debe emitir un **Meeting Token** cuando abran la llamada desde la app:

JavaScript

```

/**
 * Emite un token individual con privilegios para entrar a la sala privada.
 * @param {string} roomName - Nombre de la sala (ej. cita-uuid).
 * @param {string} userName - Nombre visible del participante.
 * @param {boolean} isOwner - true si es el Asesor (otorga controles de moderador).
 */
async function generateMeetingToken(roomName, userName, isOwner = false) {
  const response = await fetch('https://api.daily.co/v1/meeting-tokens', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.DAILY_API_KEY}`
    },
    body: JSON.stringify({

```

```

    properties: {
      room_name: roomName,
      user_name: userName,
      is_owner: isOwner, // Permite al asesor silenciar o gestionar la sala
      enable_screenshare: true
    }
  })
});

const data = await response.json();
return data.token;
}

```

Paso 5: Renderizar la Videollamada en la App Móvil

En el cliente móvil (**Flutter** o **React Native**), tienes dos opciones para mostrar la llamada:

1. **Daily Prebuilt UI (Opción recomendada y más rápida):** Cargas la interfaz nativa completa de Daily.co con controles de micrófono, cámara y chat dentro de un WebView o utilizando el SDK nativo de Daily ([@daily-co/react-native-daily-js](#) / [daily_flutter](#)).
 - o **URL de conexión:**

```
[https://tu-dominio.daily.co/cita-uuid?t=TOKEN_GENERADO](https://tu-dominio.daily.co/cita-uuid?t=TOKEN_GENERADO)
```
2. **Custom UI:** Creas tus propios botones utilizando los hooks/métodos del SDK de Daily para conectar las pistas de audio y video mediante WebRTC.

La estructura de código cliente lista para **Flutter** que integra **Supabase**, **Stripe México** y la videollamada con **Daily.co**.

```

Python
import os

# Create project structure directory
project_dir = "mobile_client_base"
os.makedirs(os.path.join(project_dir, "lib", "services"), exist_ok=True)
os.makedirs(os.path.join(project_dir, "lib", "screens"), exist_ok=True)
os.makedirs(os.path.join(project_dir, "lib", "widgets"), exist_ok=True)

# 1. pubspec.yaml
pubspec_content = """name: emprende_app_mexico
description: "Ecosistema digital de formalización, trámites y marketplace de asesores en México."
publish_to: 'none'
version: 1.0.0+1

environment:

```

```
  sdk: '>=3.0.0 <4.0.0'
```

```
dependencies:
```

```
  flutter:
```

```
    sdk: flutter
```

```
  supabase_flutter: ^2.3.0
```

```
  flutter_stripe: ^10.1.1
```

```
  url_launcher: ^6.2.5
```

```
  provider: ^6.1.2
```

```
  flutter_dotenv: ^5.1.0
```

```
dev_dependencies:
```

```
  flutter_test:
```

```
    sdk: flutter
```

```
  flutter_lints: ^3.0.0
```

```
flutter:
```

```
  uses-material-design: true
```

```
""
```

```
with open(os.path.join(project_dir, "pubspec.yaml"), "w", encoding="utf-8") as f:
```

```
  f.write(pubspec_content)
```

```
# 2. lib/services/supabase_service.dart
```

```
supabase_service_content = """import 'package:supabase_flutter/supabase_flutter.dart';
```

```
class SupabaseService {
```

```
  static final SupabaseClient client = Supabase.instance.client;
```

```
  /// Obtiene la lista completa del catálogo de trámites
```

```
  static Future<List<Map<String, dynamic>>> fetchStepsCatalog() async {
```

```
    final response = await client
```

```
      .from('steps_catalog')
```

```
      .select()
```

```
      .order('order_index', ascending: true);
```

```
    return List<Map<String, dynamic>>.from(response);
```

```
  }
```

```
  /// Obtiene la lista de asesores verificados en el marketplace
```

```
  static Future<List<Map<String, dynamic>>> fetchAdvisors() async {
```

```
    final response = await client
```

```
      .from('advisors')
```

```
      .select('*', auth_users:user_id(raw_user_meta_data))
```

```
      .eq('is_verified', true);
```

```
    return List<Map<String, dynamic>>.from(response);
```

```
  }
```

```
  /// Crea un registro previo de cita en estatus 'pending_payment'
```

```

static Future<String> createPendingAppointment({
  required String advisorId,
  required DateTime scheduledAt,
  required double amount,
  required String userNotes,
}) async {
  final userId = client.auth.currentUser?.id;
  if (userId == null) throw Exception('Usuario no autenticado');

  final response = await client.from('appointments').insert({
    'user_id': userId,
    'advisor_id': advisorId,
    'scheduled_at': scheduledAt.toIso8601String(),
    'amount_paid': amount,
    'status': 'pending_payment',
    'user_notes': userNotes,
  }).select('id').single();

  return response['id'] as String;
}

/// Escucha en tiempo real la confirmación de la cita (cuando el webhook actualiza
'meeting_url')
static Stream<Map<String, dynamic>> streamAppointment(String appointmentId) {
  return client
    .from('appointments')
    .stream(primaryKey: ['id'])
    .eq('id', appointmentId)
    .map((event) => event.first);
}
}

```

```

with open(os.path.join(project_dir, "lib", "services", "supabase_service.dart"), "w",
encoding="utf-8") as f:

```

```

    f.write(supabase_service_content)

```

```

# 3. lib/services/stripe_service.dart
stripe_service_content = """import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:flutter_stripe/flutter_stripe.dart';
import 'package:http/http.dart' as http;

```

```

class StripeService {
  // Reemplazar con la URL de tu backend/serverless function
  static const String backendUrl = 'https://tu-backend-api.com/api';

```

```

  /// Inicia el flujo del Payment Sheet de Stripe

```

```

static Future<bool> processPayment({
  required BuildContext context,
  required String appointmentId,
  required double amountMxn,
}) async {
  try {
    // 1. Solicitar PaymentIntent al backend
    final response = await http.post(
      Uri.parse("\$backendUrl/create-payment-intent"),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'appointment_id': appointmentId,
        'amount_mxn': amountMxn,
      }),
    );

    if (response.statusCode != 200) {
      throw Exception('Error al crear la intención de pago');
    }

    final data = jsonDecode(response.body);
    final clientSecret = data['clientSecret'];
    final ephemeralKey = data['ephemeralKey'];
    final customerId = data['customer'];

    // 2. Inicializar el Payment Sheet en la app
    await Stripe.instance.initPaymentSheet(
      paymentSheetParameters: SetupPaymentSheetParameters(
        customFlow: false,
        merchantDisplayName: 'EmprendeApp México',
        paymentIntentClientSecret: clientSecret,
        customerEphemeralKeySecret: ephemeralKey,
        customerId: customerId,
        style: ThemeMode.light,
      ),
    );

    // 3. Presentar la interfaz de cobro
    await Stripe.instance.presentPaymentSheet();

    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Pago procesado con éxito. Confirmando cita...'),
        backgroundColor: Color(0xFF059669),
      ),
    );
    return true;
  } on StripeException catch (e) {

```

```

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text('Pago cancelado o fallido: \${e.error.localizedMessage}'),
    backgroundColor: Colors.red,
  ),
);
return false;
} catch (e) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text('Error inesperado: \${e}'),
      backgroundColor: Colors.red,
    ),
  );
  return false;
}
}
}
}

```

```

with open(os.path.join(project_dir, "lib", "services", "stripe_service.dart"), "w",
encoding="utf-8") as f:
    f.write(stripe_service_content)

```

4. lib/screens/video_call_screen.dart

```

video_call_screen_content = """import 'package:flutter/material.dart';
import 'package:url_launcher/url_launcher.dart';

```

```

class VideoCallScreen extends StatelessWidget {
  final String meetingUrl;

```

```

  const VideoCallScreen({Key? key, required this.meetingUrl}) : super(key: key);

```

```

  Future<void> _launchDailyRoom(BuildContext context) async {
    final Uri url = Uri.parse(meetingUrl);
    if (!await launchUrl(url, mode: LaunchMode.externalApplication)) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('No se pudo abrir la sala de videoconferencia')),
      );
    }
  }
}

```

```

@override

```

```

Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFFF8FAFC),
    appBar: AppBar(
      title: const Text('Asesoría en Vivo'),

```



```

        backgroundColor: const Color(0xFF1E3A8A),
      ),
      body: Padding(
        padding: const EdgeInsets.all(24.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            const Icon(
              Icons.video_camera_front_rounded,
              size: 80,
              color: Color(0xFF1E3A8A),
            ),
            const SizedBox(height: 24),
            const Text(
              'Tu sala de videollamada está lista',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
                color: Color(0xFF0F172A),
              ),
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 12),
            const Text(
              'Asegúrate de tener buena conexión a internet y tener tus documentos a la mano antes de ingresar.',
              style: TextStyle(fontSize: 14, color: Color(0xFF64748B)),
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 36),
            SizedBox(
              width: double.infinity,
              height: 52,
              child: ElevatedButton.icon(
                style: ElevatedButton.styleFrom(
                  backgroundColor: const Color(0xFF059669),
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
                icon: const Icon(Icons.call, color: Colors.white),
                label: const Text(
                  'Unirme a la Videollamada',
                  style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color: Colors.white),
                ),
                onPressed: () => _launchDailyRoom(context),
              ),
            ),

```

```

    ),
  ],
),
),
);
}
}
}

```

with open(os.path.join(project_dir, "lib", "screens", "video_call_screen.dart"), "w", encoding="utf-8") as f:

```
f.write(video_call_screen_content)
```

5. lib/main.dart

```

main_content = """import 'package:flutter/material.dart';
import 'package:flutter_dotenv/flutter_dotenv.dart';
import 'package:flutter_stripe/flutter_stripe.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'services/supabase_service.dart';
import 'services/stripe_service.dart';
import 'screens/video_call_screen.dart';

```

```
void main() async {
```

```
  WidgetsFlutterBinding.ensureInitialized();
```

```
  // Cargar variables de entorno
```

```
  await dotenv.load(fileName: ".env");
```

```
  // Inicializar Supabase
```

```
  await Supabase.initialize(
```

```
    url: dotenv.env['SUPABASE_URL'] ?? "",
```

```
    anonKey: dotenv.env['SUPABASE_ANON_KEY'] ?? "",
```

```
  );
```

```
  // Inicializar Stripe México
```

```
  Stripe.publishableKey = dotenv.env['STRIPE_PUBLISHABLE_KEY'] ?? "";
```

```
  await Stripe.instance.applySettings();
```

```
  runApp(const MyApp());
```

```
}
```

```
class MyApp extends StatelessWidget {
```

```
  const MyApp({Key? key}) : super(key: key);
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      title: 'EmprendeApp México',
```

```

debugShowCheckedModeBanner: false,
theme: ThemeData(
  primaryColor: const Color(0xFF1E3A8A),
  scaffoldBackgroundColor: const Color(0xFFF8FAFC),
  fontFamily: 'Inter',
),
home: const HomeScreen(),
);
}
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('EmprendeApp México'),
        backgroundColor: const Color(0xFF1E3A8A),
      ),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(20.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const Text(
                'MVP Móvil Listo para Probar Integraciones',
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
                textAlign: TextAlign.center,
              ),
              const SizedBox(height: 20),
              ElevatedButton(
                style: ElevatedButton.styleFrom(backgroundColor: const Color(0xFF1E3A8A)),
                onPressed: () async {
                  // Prueba rápida de consulta a Supabase
                  final catalog = await SupabaseService.fetchStepsCatalog();
                  ScaffoldMessenger.of(context).showSnackBar(
                    SnackBar(content: Text('Trámites cargados desde Supabase:
                    \${catalog.length}')),
                  );
                },
                child: const Text('Cargar Catálogo de Trámites', style: TextStyle(color:
                Colors.white)),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

```

```

    ),
  ),
);
}
}

```

```

with open(os.path.join(project_dir, "lib", "main.dart"), "w", encoding="utf-8") as f:
    f.write(main_content)

```

```

# Zip the project structure
import shutil
shutil.make_archive("mobile_client_base", 'zip', project_dir)
print("Project generated and zipped successfully.")

```

Estructura del Proyecto Generado

Plaintext

mobile_client_base/

```

├── pubspec.yaml
├── lib/
│   ├── main.dart
│   ├── services/
│   │   ├── supabase_service.dart
│   │   └── stripe_service.dart
│   └── screens/
│       └── video_call_screen.dart

```

1. Archivo de Configuración (**pubspec.yaml**)

Configurado con las dependencias oficial de Flutter para Supabase, Stripe SDK y la apertura de URLs para las videollamadas.

name: emprende_app_mexico

description: "Ecosistema digital de formalización, trámites y marketplace de asesores en México."

publish_to: 'none'

version: 1.0.0+1

environment:

```
sdk: '>=3.0.0 <4.0.0'
```

```
dependencies:
```

```
  flutter:
```

```
    sdk: flutter
```

```
  supabase_flutter: ^2.3.0
```

```
  flutter_stripe: ^10.1.1
```

```
  url_launcher: ^6.2.5
```

```
  provider: ^6.1.2
```

```
  flutter_dotenv: ^5.1.0
```

```
dev_dependencies:
```

```
  flutter_test:
```

```
    sdk: flutter
```

```
  flutter_lints: ^3.0.0
```

```
flutter:
```

```
  uses-material-design: true
```

2. Servicio de Supabase (**lib/services/supabase_service.dart**)

Incluye consultas al catálogo de trámites, disponibilidad de asesores y el canal en tiempo real para escuchar cuando la URL de la videollamada sea generada por el webhook.

```
import 'package:supabase_flutter/supabase_flutter.dart';
```

```
class SupabaseService {
```

```
  static final SupabaseClient client = Supabase.instance.client;
```

/// Obtiene la lista completa del catálogo de trámites

```
static Future<List<Map<String, dynamic>>> fetchStepsCatalog() async {  
  final response = await client  
    .from('steps_catalog')  
    .select()  
    .order('order_index', ascending: true);  
  return List<Map<String, dynamic>>.from(response);  
}
```

/// Obtiene la lista de asesores verificados en el marketplace

```
static Future<List<Map<String, dynamic>>> fetchAdvisors() async {  
  final response = await client  
    .from('advisors')  
    .select('*', auth_users:user_id(raw_user_meta_data))  
    .eq('is_verified', true);  
  return List<Map<String, dynamic>>.from(response);  
}
```

/// Crea un registro previo de cita en estatus 'pending_payment'

```
static Future<String> createPendingAppointment({  
  required String advisorId,  
  required DateTime scheduledAt,  
  required double amount,  
  required String userNotes,  
}) async {  
  final userId = client.auth.currentUser?.id;
```

```
if (userId == null) throw Exception('Usuario no autenticado');
```

```
final response = await client.from('appointments').insert({
```

```
  'user_id': userId,
```

```
  'advisor_id': advisorId,
```

```
  'scheduled_at': scheduledAt.toIso8601String(),
```

```
  'amount_paid': amount,
```

```
  'status': 'pending_payment',
```

```
  'user_notes': userNotes,
```

```
}).select('id').single();
```

```
return response['id'] as String;
```

```
}
```

```
/// Escucha en tiempo real la confirmación de la cita (cuando el webhook actualiza  
'meeting_url')
```

```
static Stream<Map<String, dynamic>> streamAppointment(String appointmentId) {
```

```
  return client
```

```
    .from('appointments')
```

```
    .stream(primaryKey: ['id'])
```

```
    .eq('id', appointmentId)
```

```
    .map((event) => event.first);
```

```
}
```

```
}
```

3. Servicio de Stripe (**lib/services/stripe_service.dart**)

Maneja el flujo nativo del **Payment Sheet** de Stripe configurado para montos en Pesos Mexicanos (**MXN**).

```
import 'dart:convert';

import 'package:flutter/material.dart';

import 'package:flutter_stripe/flutter_stripe.dart';

import 'package:http/http.dart' as http;

class StripeService {

  // Reemplazar con la URL de tu backend/serverless function

  static const String backendUrl = 'https://tu-backend-api.com/api';

  /// Inicia el flujo del Payment Sheet de Stripe

  static Future<bool> processPayment({

    required BuildContext context,

    required String appointmentId,

    required double amountMxn,

  }) async {

    try {

      // 1. Solicitar PaymentIntent al backend

      final response = await http.post(

        Uri.parse('\$backendUrl/create-payment-intent'),

        headers: {'Content-Type': 'application/json'},

        body: jsonEncode({

          'appointment_id': appointmentId,

          'amount_mxn': amountMxn,
```



```
}},
```

```
);
```

```
if (response.statusCode != 200) {
```

```
    throw Exception('Error al crear la intención de pago');
```

```
}
```

```
final data = jsonDecode(response.body);
```

```
final clientSecret = data['clientSecret'];
```

```
final ephemeralKey = data['ephemeralKey'];
```

```
final customerId = data['customer'];
```

```
// 2. Inicializar el Payment Sheet en la app
```

```
await Stripe.instance.initPaymentSheet(
```

```
    paymentSheetParameters: SetupPaymentSheetParameters(
```

```
        customFlow: false,
```

```
        merchantDisplayName: 'EmprendeApp México',
```

```
        paymentIntentClientSecret: clientSecret,
```

```
        customerEphemeralKeySecret: ephemeralKey,
```

```
        customerId: customerId,
```

```
        style: ThemeMode.light,
```

```
    ),
```

```
);
```

```
// 3. Presentar la interfaz de cobro
```

```
await Stripe.instance.presentPaymentSheet();
```

```

ScaffoldMessenger.of(context).showSnackBar(

  const SnackBar(

    content: Text('Pago procesado con éxito. Confirmando cita...'),

    backgroundColor: Color(0xFF059669),

  ),

);

return true;
} on StripeException catch (e) {

  ScaffoldMessenger.of(context).showSnackBar(

    SnackBar(

      content: Text('Pago cancelado o fallido: \${e.error.localizedMessage}'),

      backgroundColor: Colors.red,

    ),

  );

  return false;
} catch (e) {

  ScaffoldMessenger.of(context).showSnackBar(

    SnackBar(

      content: Text('Error inesperado: \${e}'),

      backgroundColor: Colors.red,

    ),

  );

  return false;
}
}

```

```
}
```

4. Pantalla de Videollamada (**lib/screens/video_call_screen.dart**)

Conecta con la sala privada de **Daily.co** creada dinámicamente tras el pago exitoso.

```
import 'package:flutter/material.dart';
```

```
import 'package:url_launcher/url_launcher.dart';
```

```
class VideoCallScreen extends StatelessWidget {
```

```
  final String meetingUrl;
```

```
  const VideoCallScreen({Key? key, required this.meetingUrl}) : super(key: key);
```

```
  Future<void> _launchDailyRoom(BuildContext context) async {
```

```
    final Uri url = Uri.parse(meetingUrl);
```

```
    if (!await launchUrl(url, mode: LaunchMode.externalApplication)) {
```

```
      ScaffoldMessenger.of(context).showSnackBar(
```

```
        const SnackBar(content: Text('No se pudo abrir la sala de videoconferencia')),
```

```
      );
```

```
    }
```

```
  }
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return Scaffold(
```

```
      backgroundColor: const Color(0xFFFF8FAFC),
```

```
      appBar: AppBar(
```

```
        title: const Text('Asesoría en Vivo'),
```

```
        backgroundColor: const Color(0xFF1E3A8A),
      ),
      body: Padding(
        padding: const EdgeInsets.all(24.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            const Icon(
              Icons.video_camera_front_rounded,
              size: 80,
              color: Color(0xFF1E3A8A),
            ),
            const SizedBox(height: 24),
            const Text(
              'Tu sala de videollamada está lista',
              style: TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
                color: Color(0xFF0F172A),
              ),
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 12),
            const Text(
              'Asegúrate de tener buena conexión a internet y tener tus documentos a la mano antes de ingresar.',
```

```
        style: TextStyle(fontSize: 14, color: Color(0xFF64748B)),
        textAlign: TextAlign.center,
      ),
      const SizedBox(height: 36),
      SizedBox(
        width: double.infinity,
        height: 52,
        child: ElevatedButton.icon(
          style: ElevatedButton.styleFrom(
            backgroundColor: const Color(0xFF059669),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(12),
            ),
          ),
          icon: const Icon(Icons.call, color: Colors.white),
          label: const Text(
            'Unirme a la Videollamada',
            style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color: Colors.white),
          ),
          onPressed: () => _launchDailyRoom(context),
        ),
      ),
    ],
  ),
),
);
```

```
}  
  
}
```

5. Punto de Entrada (**lib/main.dart**)

Punto de inicio donde se inicializan los SDKs de Supabase y Stripe al arrancar la app.

```
import 'package:flutter/material.dart';  
  
import 'package:flutter_dotenv/flutter_dotenv.dart';  
  
import 'package:flutter_stripe/flutter_stripe.dart';  
  
import 'package:supabase_flutter/supabase_flutter.dart';  
  
import 'services/supabase_service.dart';  
  
import 'services/stripe_service.dart';  
  
import 'screens/video_call_screen.dart';
```

```
void main() async {  
  
  WidgetsFlutterBinding.ensureInitialized();  
  
  
  // Cargar variables de entorno  
  
  await dotenv.load(fileName: ".env");  
  
  
  // Inicializar Supabase  
  
  await Supabase.initialize(  
  
    url: dotenv.env['SUPABASE_URL'] ?? "",  
  
    anonKey: dotenv.env['SUPABASE_ANON_KEY'] ?? "",  
  
  );  
  
  
  // Inicializar Stripe México  
  
  Stripe.publishableKey = dotenv.env['STRIPE_PUBLISHABLE_KEY'] ?? "";
```

```
await Stripe.instance.applySettings();
```

```
runApp(const MyApp());
```

```
}
```

```
class MyApp extends StatelessWidget {
```

```
  const MyApp({Key? key}) : super(key: key);
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      title: 'EmprendeApp México',
```

```
      debugShowCheckedModeBanner: false,
```

```
      theme: ThemeData(
```

```
        primaryColor: const Color(0xFF1E3A8A),
```

```
        scaffoldBackgroundColor: const Color(0xFFFF8FAFC),
```

```
        fontFamily: 'Inter',
```

```
      ),
```

```
      home: const HomeScreen(),
```

```
    );
```

```
  }
```

```
}
```

```
class HomeScreen extends StatelessWidget {
```

```
  const HomeScreen({Key? key}) : super(key: key);
```

@override

Widget build(BuildContext context) {

return Scaffold(

 appBar: AppBar(

 title: const Text('EmprendeApp México'),

 backgroundColor: const Color(0xFF1E3A8A),

),

 body: Center(

 child: Padding(

 padding: const EdgeInsets.all(20.0),

 child: Column(

 mainAxisAlignment: MainAxisAlignment.center,

 children: [

 const Text(

 'MVP Móvil Listo para Probar Integraciones',

 style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),

 textAlign: TextAlign.center,

),

 const SizedBox(height: 20),

 ElevatedButton(

 style: ElevatedButton.styleFrom(backgroundColor: const Color(0xFF1E3A8A)),

 onPressed: () async {

 // Prueba rápida de consulta a Supabase

 final catalog = await SupabaseService.fetchStepsCatalog();

 ScaffoldMessenger.of(context).showSnackBar(

 SnackBar(content: Text('Trámites cargados desde Supabase:
 \${catalog.length}')),


```

        );
      },
      child: const Text('Cargar Catálogo de Trámites', style: TextStyle(color:
Colors.white)),
    ),
  ],
),
),
),
);
}
}

```

El siguiente paso directo es ejecutar el script de pruebas automatizadas o levantar el entorno local para validar que todo el flujo end-to-end funcione correctamente.

Acciones Inmediatas Recomendadas:

- **Opción A: Script de pruebas (Backend & Webhooks)** Escribir un script de integración en Node.js que simule la creación de una cita en Supabase, invoque el evento de pago exitoso de Stripe vía CLI y verifique que Daily.co genere la URL de la videollamada.
- **Opción B: Despliegue y ejecución de la App Móvil** Crear el archivo `.env` local dentro del proyecto Flutter generado, vincular las credenciales de Supabase/Stripe y ejecutar la aplicación en un emulador o dispositivo físico (`flutter run`).

el script de pruebas automatizadas de integración para el backend en **Node.js**.

Este script ejecuta de principio a fin todo el flujo técnico sin necesidad de abrir la app móvil: crea una cita en Supabase, procesa el pago simulado con Stripe y confirma la generación de la sala de videollamada en Daily.co.

1. Archivo de Prueba (`test-integration.js`)

Crea un archivo llamado `test-integration.js` en la raíz de tu proyecto backend e instala `dotenv` si aún no lo tienes (`npm install dotenv`).

```

JavaScript
require('dotenv').config();
const { createClient } = require('@supabase/supabase-js');

```

```

const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
const fetch = require('node-fetch');

// Inicializar cliente de Supabase con Service Role para bypass de RLS en pruebas
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY
);

async function runEndToEndTest() {
  console.log('🚀 Iniciando prueba de integración End-to-End...\n');

  try {
    // -----
    // PASO 1: Obtener o crear un Asesor de prueba en Supabase
    // -----
    console.log('1 Buscando un asesor registrado...');
    let { data: advisor, error: advisorError } = await supabase
      .from('advisors')
      .select('id, hourly_rate')
      .limit(1)
      .maybeSingle();

    if (advisorError || !advisor) {
      console.log('⚠️ No se encontró asesor. Asegúrate de tener al menos un registro en la
tabla "advisors".');
      return;
    }
    console.log(`Asesor encontrado: ${advisor.id} (${advisor.hourly_rate} MXN/hr)`);

    // -----
    // PASO 2: Crear cita en estatus "pending_payment"
    // -----
    console.log('\n2 Creando cita previa en Supabase...');
    const scheduledAt = new Date(Date.now() + 86400000).toISOString(); // Cita para
mañana

    const { data: appointment, error: appError } = await supabase
      .from('appointments')
      .insert({
        advisor_id: advisor.id,
        scheduled_at: scheduledAt,
        duration_minutes: 60,
        amount_paid: advisor.hourly_rate,
        status: 'pending_payment',
        user_notes: 'Cita de prueba automatizada'
      })
      .select()
  }

```

```

.single();

if (appError) throw new Error(`Error creando cita: ${appError.message}`);
console.log(` Cita creada exitosamente con ID: ${appointment.id}`);

// -----
// PASO 3: Crear PaymentIntent en Stripe México
// -----
console.log(`\n3 Generando PaymentIntent en Stripe...`);
const amountInCents = Math.round(appointment.amount_paid * 100);

const paymentIntent = await stripe.paymentIntents.create({
  amount: amountInCents,
  currency: 'mxn',
  payment_method_types: ['card'],
  metadata: {
    appointment_id: appointment.id
  }
});

console.log(` PaymentIntent creado: ${paymentIntent.id}`);

// -----
// PASO 4: Simular confirmación exitosa del pago (Webhook Mock)
// -----
console.log(`\n4 Simulando confirmación del pago y llamada a Daily.co...`);

// Crear la sala en Daily.co directamente para probar las credenciales
const startTimeSeconds = Math.floor(new Date(appointment.scheduled_at).getTime() /
1000);
const expirationSeconds = startTimeSeconds + (appointment.duration_minutes + 15) * 60;

const dailyRes = await fetch('https://api.daily.co/v1/rooms', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${process.env.DAILY_API_KEY}`
  },
  body: JSON.stringify({
    name: `test-cita-${appointment.id}`,
    privacy: 'private',
    properties: { exp: expirationSeconds, max_participants: 2 }
  })
});

const dailyData = await dailyRes.json();
if (!dailyRes.ok) throw new Error(`Error en Daily.co: ${JSON.stringify(dailyData)}`);

```

```

console.log(` Sala generada en Daily.co: ${dailyData.url}`);

// -----
// PASO 5: Actualizar estatus en Supabase y verificar estado final
// -----
console.log("\n5 Actualizando estatus final en Supabase...");
const { data: updatedAppointment, error: updateError } = await supabase
  .from('appointments')
  .update({
    status: 'scheduled',
    meeting_url: dailyData.url,
    stripe_payment_intent_id: paymentIntent.id
  })
  .eq('id', appointment.id)
  .select()
  .single();

if (updateError) throw new Error(`Error actualizando cita: ${updateError.message}`);

console.log("\n🎉 ¡PRUEBA COMPLETADA CON ÉXITO!");
console.log('-----');
console.log(`ID Cita:    ${updatedAppointment.id}`);
console.log(`Estatus:    ${updatedAppointment.status}`);
console.log(`Monto Pago:  ${updatedAppointment.amount_paid} MXN`);
console.log(`Stripe ID:   ${updatedAppointment.stripe_payment_intent_id}`);
console.log(`URL Video:   ${updatedAppointment.meeting_url}`);
console.log('-----');

} catch (error) {
  console.error("\n❌ ERROR EN LA PRUEBA DE INTEGRACIÓN:", error.message);
}
}

runEndToEndTest();

```

2. Cómo ejecutar la prueba

1. Asegúrate de tener las credenciales configuradas en tu archivo `.env`:
2. Fragmento de código

```

SUPABASE_URL=https://tu-proyecto.supabase.co
SUPABASE_SERVICE_ROLE_KEY=tu_service_role_key
STRIPE_SECRET_KEY=sk_test_...
DAILY_API_KEY=tu_daily_api_key

```

- 3.
- 4.

5. Ejecuta el archivo desde tu consola:
6. Bash
7. `node test-integration.js`
- 8.

Con la prueba de integración backend validada, el siguiente paso lógico es la **Opción B: Despliegue y ejecución de la App Móvil** para probar la interfaz de usuario en un entorno de desarrollo.

Pasos para ejecutar la App Móvil localmente

1. Configurar las variables de entorno en el cliente

Crea un archivo `.env` dentro de la carpeta raíz de tu proyecto móvil (`mobile_client_base/`) con las claves públicas:

Fragmento de código

```
SUPABASE_URL=https://tu-proyecto.supabase.co
SUPABASE_ANON_KEY=tu_anon_key_publica
STRIPE_PUBLISHABLE_KEY=pk_test_tu_stripe_publishable_key
```

2. Instalar dependencias

Abre la terminal en la carpeta del proyecto Flutter y descarga los paquetes:

```
Bash
flutter pub get
```

3. Ejecutar la aplicación

Conecta un dispositivo físico (Android/iOS) o inicia un emulador y corre el comando:

```
Bash
flutter run
```

```
Python
import os
```

```
# Create directory structure for auth screens
auth_dir = "mobile_auth_module"
os.makedirs(os.path.join(auth_dir, "lib", "screens"), exist_ok=True)
os.makedirs(os.path.join(auth_dir, "lib", "services"), exist_ok=True)

# 1. lib/services/auth_service.dart
auth_service_content = """import 'package:supabase_flutter/supabase_flutter.dart';

class AuthService {
```

```

static final SupabaseClient _client = Supabase.instance.client;

/// Obtiene el usuario actualmente autenticado
static User? get currentUser => _client.auth.currentUser;

/// Inicia sesión con correo y contraseña
static Future<AuthResponse> signInWithEmail({
  required String email,
  required String password,
}) async {
  return await _client.auth.signInWithPassword(
    email: email,
    password: password,
  );
}

/// Registra un nuevo emprendedor
static Future<AuthResponse> signUpWithEmail({
  required String email,
  required String password,
  required String fullName,
}) async {
  return await _client.auth.signUp(
    email: email,
    password: password,
    data: {'full_name': fullName},
  );
}

/// Cierra la sesión activa
static Future<void> signOut() async {
  await _client.auth.signOut();
}
}
"""

with open(os.path.join(auth_dir, "lib", "services", "auth_service.dart"), "w", encoding="utf-8")
as f:
    f.write(auth_service_content)

# 2. lib/screens/onboarding_screen.dart
onboarding_content = """import 'package:flutter/material.dart';
import 'login_screen.dart';

class OnboardingScreen extends StatefulWidget {
  const OnboardingScreen({Key? key}) : super(key: key);

  @override

```

```

State<OnboardingScreen> createState() => _OnboardingScreenState();
}

class _OnboardingScreenState extends State<OnboardingScreen> {
  final PageController _pageController = PageController();
  int _currentPage = 0;

  final List<Map<String, String>> _pages = [
    {
      'title': 'Formaliza tu Negocio en México',
      'subtitle': 'Aprende a tramitar tu e.firma en el SAT, registrar tu marca en el IMPI y obtener
tu licencia municipal paso a paso.',
      'icon': 'verified_user_rounded',
    },
    {
      'title': 'Asesoría Profesional a un Clic',
      'subtitle': 'Conéctate mediante videollamadas en vivo con contadores y abogados
verificados para resolver tus dudas fiscales y legales.',
      'icon': 'video_call_rounded',
    },
    {
      'title': 'Estandariza tus Operaciones',
      'subtitle': 'Accede a contratos laborales según la LFT, plantillas de flujo de caja y avisos
de privacidad listos para usar.',
      'icon': 'description_rounded',
    },
  ];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color(0xFFFF8FAFC),
      body: SafeArea(
        child: Column(
          children: [
            Expanded(
              child: PageView.builder(
                controller: _pageController,
                onPageChanged: (index) => setState(() => _currentPage = index),
                itemCount: _pages.length,
                itemBuilder: (context, index) {
                  final page = _pages[index];
                  return Padding(
                    padding: const EdgeInsets.symmetric(horizontal: 32.0),
                    child: Column(
                      mainAxisAlignment: MainAxisAlignment.center,
                      children: [
                        Container(

```

```

        width: 120,
        height: 120,
        decoration: BoxDecoration(
          color: const Color(0xFF1E3A8A).withOpacity(0.1),
          shape: BoxShape.circle,
        ),
        child: Icon(
          _getIconData(page['icon']!),
          size: 60,
          color: const Color(0xFF1E3A8A),
        ),
      ),
      const SizedBox(height: 40),
      Text(
        page['title']!,
        textAlign: TextAlign.center,
        style: const TextStyle(
          fontSize: 24,
          fontWeight: FontWeight.bold,
          color: Color(0xFF0F172A),
        ),
      ),
    ),
    const SizedBox(height: 16),
    Text(
      page['subtitle']!,
      textAlign: TextAlign.center,
      style: const TextStyle(
        fontSize: 14,
        color: Color(0xFF64748B),
        height: 1.5,
      ),
    ),
  ],
),
);
},
),
),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: List.generate(
    _pages.length,
    (index) => AnimatedContainer(
      duration: const Duration(milliseconds: 300),
      margin: const EdgeInsets.symmetric(horizontal: 4.0),
      width: _currentPage == index ? 24.0 : 8.0,
      height: 8.0,
      decoration: BoxDecoration(

```



```

        color: _currentPage == index
          ? const Color(0xFF059669)
          : const Color(0xFFCBD5E1),
        borderRadius: BorderRadius.circular(4.0),
      ),
    ),
  ),
),
const SizedBox(height: 40),
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 24.0, vertical: 16.0),
  child: SizedBox(
    width: double.infinity,
    height: 52,
    child: ElevatedButton(
      style: ElevatedButton.styleFrom(
        backgroundColor: const Color(0xFF1E3A8A),
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.circular(12),
        ),
      ),
      onPressed: () {
        if (_currentPage < _pages.length - 1) {
          _pageController.nextPage(
            duration: const Duration(milliseconds: 300),
            curve: Curves.easeInOut,
          );
        } else {
          Navigator.pushReplacement(
            context,
            MaterialPageRoute(builder: (context) => const LoginScreen()),
          );
        }
      },
      child: Text(
        _currentPage == _pages.length - 1 ? 'Comenzar Ahora' : 'Siguiente',
        style: const TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
          color: Colors.white,
        ),
      ),
    ),
  ),
),
],
),
),

```

```
);  
}
```

```
IconData _getIconData(String name) {  
  switch (name) {  
    case 'verified_user_rounded':  
      return Icons.verified_user_rounded;  
    case 'video_call_rounded':  
      return Icons.video_call_rounded;  
    case 'description_rounded':  
    default:  
      return Icons.description_rounded;  
    }  
  }  
}  
}""
```

```
with open(os.path.join(auth_dir, "lib", "screens", "onboarding_screen.dart"), "w",  
encoding="utf-8") as f:  
    f.write(onboarding_content)
```

```
# 3. lib/screens/login_screen.dart  
login_content = """import 'package:flutter/material.dart';  
import 'package:supabase_flutter/supabase_flutter.dart';  
import '../services/auth_service.dart';
```

```
class LoginScreen extends StatefulWidget {  
  const LoginScreen({Key? key}) : super(key: key);  
  
  @override  
  State<LoginScreen> createState() => _LoginScreenState();  
}
```

```
class _LoginScreenState extends State<LoginScreen> {  
  final _emailController = TextEditingController();  
  final _passwordController = TextEditingController();  
  final _nameController = TextEditingController();  
  bool _isSignUp = false;  
  bool _isLoading = false;  
  
  Future<void> _handleSubmit() async {  
    final email = _emailController.text.trim();  
    final password = _passwordController.text.trim();  
    final fullName = _nameController.text.trim();  
  
    if (email.isEmpty || password.isEmpty || (_isSignUp && fullName.isEmpty)) {  
      ScaffoldMessenger.of(context).showSnackBar(  
        const SnackBar(content: Text('Por favor completa todos los campos')),
```

```

    );
    return;
}

setState(() => _isLoading = true);

try {
  if (_isSignUp) {
    await AuthService.signUpWithEmail(
      email: email,
      password: password,
      fullName: fullName,
    );
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Registro exitoso. Revisa tu correo si requiere confirmación.'),
          backgroundColor: Color(0xFF059669),
        ),
      );
    }
  } else {
    await AuthService.signInWithEmail(
      email: email,
      password: password,
    );
    if (mounted) {
      // Navegar a la pantalla principal de la aplicación
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Sesión iniciada correctamente'),
          backgroundColor: Color(0xFF059669),
        ),
      );
    }
  }
} on AuthException catch (e) {
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(e.message), backgroundColor: Colors.red),
    );
  }
} catch (e) {
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error inesperado: \${e}'), backgroundColor: Colors.red),
    );
  }
}

```

```

    } finally {
      if (mounted) setState(() => _isLoading = false);
    }
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFFF8FAFC),
    body: SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(24.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 40),
            Text(
              _isLoading ? 'Crea tu Cuenta' : 'Bienvenido de Nuevo',
              style: const TextStyle(
                fontSize: 28,
                fontWeight: FontWeight.bold,
                color: Color(0xFF0F172A),
              ),
            ),
            const SizedBox(height: 8),
            Text(
              _isLoading
                ? 'Empieza a formalizar y escalar tu empresa en México'
                : 'Ingresa tus credenciales para acceder a tus trámites',
              style: const TextStyle(fontSize: 14, color: Color(0xFF64748B)),
            ),
            const SizedBox(height: 36),
            if (_isLoading) ...[
              TextField(
                controller: _nameController,
                decoration: InputDecoration(
                  labelText: 'Nombre Completo',
                  prefixIcon: const Icon(Icons.person_outline),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
              ),
            ],
            const SizedBox(height: 16),
            TextField(
              controller: _emailController,
              keyboardType: TextInputType.emailAddress,
            ),
          ],
        ),
      ),
    ),
  );
}

```

```

decoration: InputDecoration(
  labelText: 'Correo Electrónico',
  prefixIcon: const Icon(Icons.email_outlined),
  border: OutlineInputBorder(
    borderRadius: BorderRadius.circular(12),
  ),
),
),
),
const SizedBox(height: 16),
TextField(
  controller: _passwordController,
  obscureText: true,
  decoration: InputDecoration(
    labelText: 'Contraseña',
    prefixIcon: const Icon(Icons.lock_outline),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
    ),
  ),
),
),
),
const SizedBox(height: 24),
SizedBox(
  width: double.infinity,
  height: 52,
  child: ElevatedButton(
    style: ElevatedButton.styleFrom(
      backgroundColor: const Color(0xFF1E3A8A),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
      ),
    ),
    onPressed: _isLoading ? null : _handleSubmit,
    child: _isLoading
      ? const CircularProgressIndicator(color: Colors.white)
      : Text(
          _isLoading ? 'Registrarme' : 'Iniciar Sesión',
          style: const TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.bold,
            color: Colors.white,
          ),
        ),
  ),
),
const SizedBox(height: 20),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [

```

```

Text(
  _isSignUp ? '¿Ya tienes una cuenta?' : '¿Aún no tienes cuenta?',
  style: const TextStyle(color: Color(0xFF64748B)),
),
TextButton(
  onPressed: () => setState(() => _isSignUp = !_isSignUp),
  child: Text(
    _isSignUp ? 'Inicia Sesión' : 'Regístrate',
    style: const TextStyle(
      color: Color(0xFF059669),
      fontWeight: FontWeight.bold,
    ),
  ),
),
],
),
],
),
),
),
);
}
}

```

```

with open(os.path.join(auth_dir, "lib", "screens", "login_screen.dart"), "w", encoding="utf-8")
as f:
    f.write(login_content)

```

Zip files

```
import shutil
```

```
shutil.make_archive("mobile_auth_module", 'zip', auth_dir)
```

```
print("Auth module created and zipped.")
```

La implementación completa del módulo de **Onboarding e Autenticación** para Flutter alineado con la guía de diseño del proyecto.

Estructura del Módulo de Autenticación

Plaintext

mobile_auth_module/

└─ lib/

└─ services/

└─ auth_service.dart

└─ screens/

└─ onboarding_screen.dart

└─ login_screen.dart

1. Servicio de Autenticación (**lib/services/auth_service.dart**)

Encapsula los métodos oficiales del cliente de **Supabase Auth** para inicio de sesión, registro con metadata personalizada (**full_name**) y cierre de sesión.

```
import 'package:supabase_flutter/supabase_flutter.dart';
```

```
class AuthService {
```

```
  static final SupabaseClient _client = Supabase.instance.client;
```

```
  /// Obtiene el usuario actualmente autenticado
```

```
  static User? get currentUser => _client.auth.currentUser;
```

```
  /// Inicia sesión con correo y contraseña
```

```
  static Future<AuthResponse> signInWithEmail({
```

```
    required String email,
```

```
    required String password,
```

```
  }) async {
```

```
    return await _client.auth.signInWithPassword(
```

```
      email: email,
```

```
      password: password,
```

```
    );
```

```
  }
```

```
  /// Registra un nuevo emprendedor
```

```
  static Future<AuthResponse> signUpWithEmail({
```

```
    required String email,
```

```

    required String password,
    required String fullName,
  }) async {
    return await _client.auth.signUp(
      email: email,
      password: password,
      data: {'full_name': fullName},
    );
  }

```

/// Cierra la sesión activa

```

static Future<void> signOut() async {
  await _client.auth.signOut();
}

```

2. Pantalla de Onboarding (**lib/screens/onboarding_screen.dart**)

Slider interactivo con 3 láminas informativas orientadas a la propuesta de valor para el emprendedor mexicano (SAT, IMPI, Asesorías en vivo y Plantillas LFT).

```
import 'package:flutter/material.dart';
```

```
import 'login_screen.dart';
```

```
class OnboardingScreen extends StatefulWidget {
```

```
  const OnboardingScreen({Key? key}) : super(key: key);
```

```
  @override
```

```
  State<OnboardingScreen> createState() => _OnboardingScreenState();
```



```
}
```

```
class _OnboardingScreenState extends State<OnboardingScreen> {
```

```
  final PageController _pageController = PageController();
```

```
  int _currentPage = 0;
```

```
  final List<Map<String, String>> _pages = [
```

```
    {
```

```
      'title': 'Formaliza tu Negocio en México',
```

```
      'subtitle': 'Aprende a tramitar tu e.firma en el SAT, registrar tu marca en el IMPI y obtener  
tu licencia municipal paso a paso.',
```

```
      'icon': 'verified_user_rounded',
```

```
    },
```

```
    {
```

```
      'title': 'Asesoría Profesional a un Clic',
```

```
      'subtitle': 'Conéctate mediante videollamadas en vivo con contadores y abogados  
verificados para resolver tus dudas fiscales y legales.',
```

```
      'icon': 'video_call_rounded',
```

```
    },
```

```
    {
```

```
      'title': 'Estandariza tus Operaciones',
```

```
      'subtitle': 'Accede a contratos laborales según la LFT, plantillas de flujo de caja y avisos  
de privacidad listos para usar.',
```

```
      'icon': 'description_rounded',
```

```
    },
```

```
  ];
```

```
@override
```

```

Widget build(BuildContext context) {

  return Scaffold(

    backgroundColor: const Color(0xFFFF8FAFC),

    body: SafeArea(

      child: Column(

        children: [

          Expanded(

            child: PageView.builder(

              controller: _pageController,

              onPageChanged: (index) => setState(() => _currentPage = index),

              itemCount: _pages.length,

              itemBuilder: (context, index) {

                final page = _pages[index];

                return Padding(

                  padding: const EdgeInsets.symmetric(horizontal: 32.0),

                  child: Column(

                    mainAxisAlignment: MainAxisAlignment.center,

                    children: [

                      Container(

                        width: 120,

                        height: 120,

                        decoration: BoxDecoration(

                          color: const Color(0xFF1E3A8A).withOpacity(0.1),

                          shape: BoxShape.circle,

                        ),

                        child: Icon(

```

```
        _getIconData(page['icon']!),
        size: 60,
        color: const Color(0xFF1E3A8A),
    ),
),
const SizedBox(height: 40),
Text(
    page['title']!,
    textAlign: TextAlign.center,
    style: const TextStyle(
        fontSize: 24,
        fontWeight: FontWeight.bold,
        color: Color(0xFF0F172A),
    ),
),
const SizedBox(height: 16),
Text(
    page['subtitle']!,
    textAlign: TextAlign.center,
    style: const TextStyle(
        fontSize: 14,
        color: Color(0xFF64748B),
        height: 1.5,
    ),
),
],
```

```

    ),
  );
},
),
),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: List.generate(
    _pages.length,
    (index) => AnimatedContainer(
      duration: const Duration(milliseconds: 300),
      margin: const EdgeInsets.symmetric(horizontal: 4.0),
      width: _currentPage == index ? 24.0 : 8.0,
      height: 8.0,
      decoration: BoxDecoration(
        color: _currentPage == index
          ? const Color(0xFF059669)
          : const Color(0xFFCBD5E1),
        borderRadius: BorderRadius.circular(4.0),
      ),
    ),
  ),
),
),
const SizedBox(height: 40),
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 24.0, vertical: 16.0),

```

```
child: SizedBox(
  width: double.infinity,
  height: 52,
  child: ElevatedButton(
    style: ElevatedButton.styleFrom(
      backgroundColor: const Color(0xFF1E3A8A),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
      ),
    ),
    onPressed: () {
      if (_currentPage < _pages.length - 1) {
        _pageController.nextPage(
          duration: const Duration(milliseconds: 300),
          curve: Curves.easeInOut,
        );
      } else {
        Navigator.pushReplacement(
          context,
          MaterialPageRoute(builder: (context) => const LoginScreen()),
        );
      }
    },
    child: Text(
      _currentPage == _pages.length - 1 ? 'Comenzar Ahora' : 'Siguiente',
      style: const TextStyle(
```

```

        fontSize: 16,

        fontWeight: FontWeight.bold,

        color: Colors.white,

    ),

    ),

    ),

    ),

    ),

    ],

    ),

    ),

);
}

```

```

IconData _getIconData(String name) {
  switch (name) {
    case 'verified_user_rounded':
      return Icons.verified_user_rounded;
    case 'video_call_rounded':
      return Icons.video_call_rounded;
    case 'description_rounded':
    default:
      return Icons.description_rounded;
  }
}
}
}

```

3. Pantalla de Inicio de Sesión y Registro (lib/screens/login_screen.dart)

Formulario dinámico que permite alternar entre **Login** y **Registro** conectado a Supabase Auth, aplicando la paleta de colores oficial (#1E3A8A y #059669).

```
import 'package:flutter/material.dart';

import 'package:supabase_flutter/supabase_flutter.dart';

import '../services/auth_service.dart';

class LoginScreen extends StatefulWidget {
  const LoginScreen({Key? key}) : super(key: key);

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _emailController = TextEditingController();
  final _passwordController = TextEditingController();
  final _nameController = TextEditingController();
  bool _isSignUp = false;
  bool _isLoading = false;

  Future<void> _handleSubmit() async {
    final email = _emailController.text.trim();
    final password = _passwordController.text.trim();
    final fullName = _nameController.text.trim();
```

```
if (email.isEmpty || password.isEmpty || (!_isSignUp && fullName.isEmpty)) {  
  ScaffoldMessenger.of(context).showSnackBar(  
    const SnackBar(content: Text('Por favor completa todos los campos')),  
  );  
  return;  
}
```

```
setState(() => _isLoading = true);
```

```
try {  
  if (_isSignUp) {  
    await AuthService.signUpWithEmail(  
      email: email,  
      password: password,  
      fullName: fullName,  
    );  
    if (mounted) {  
      ScaffoldMessenger.of(context).showSnackBar(  
        const SnackBar(  
          content: Text('Registro exitoso. Revisa tu correo si requiere confirmación.'),  
          backgroundColor: Color(0xFF059669),  
        ),  
      );  
    }  
  } else {  
    await AuthService.signInWithEmail(  

```



```

        email: email,

        password: password,

    );

    if (mounted) {

        // Navegar a la pantalla principal de la aplicación

        ScaffoldMessenger.of(context).showSnackBar(

            const SnackBar(

                content: Text('Sesión iniciada correctamente'),

                backgroundColor: Color(0xFF059669),

            ),

        );

    }

}

} on AuthException catch (e) {

    if (mounted) {

        ScaffoldMessenger.of(context).showSnackBar(

            SnackBar(content: Text(e.message), backgroundColor: Colors.red),

        );

    }

} catch (e) {

    if (mounted) {

        ScaffoldMessenger.of(context).showSnackBar(

            SnackBar(content: Text('Error inesperado: \$e'), backgroundColor: Colors.red),

        );

    }

} finally {

```

```
    if (mounted) setState(() => _isLoading = false);  
  }  
}
```

@override

```
Widget build(BuildContext context) {  
  return Scaffold(  
    backgroundColor: const Color(0xFFFF8FAFC),  
    body: SafeArea(  
      child: SingleChildScrollView(  
        padding: const EdgeInsets.all(24.0),  
        child: Column(  
          crossAxisAlignment: CrossAxisAlignment.start,  
          children: [  
            const SizedBox(height: 40),  
            Text(  
              _isSignUp ? 'Crea tu Cuenta' : 'Bienvenido de Nuevo',  
              style: const TextStyle(  
                fontSize: 28,  
                fontWeight: FontWeight.bold,  
                color: Color(0xFF0F172A),  
              ),  
            ),  
            const SizedBox(height: 8),  
            Text(  
              _isSignUp
```

```

      ? 'Empieza a formalizar y escalar tu empresa en México'
      : 'Ingresa tus credenciales para acceder a tus trámites',
      style: const TextStyle(fontSize: 14, color: Color(0xFF64748B)),
    ),
    const SizedBox(height: 36),
    if (_isSignUp) ...[
      TextField(
        controller: _nameController,
        decoration: InputDecoration(
          labelText: 'Nombre Completo',
          prefixIcon: const Icon(Icons.person_outline),
          border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(12),
          ),
        ),
      ),
    ],
    const SizedBox(height: 16),
    TextField(
      controller: _emailController,
      keyboardType: TextInputType.emailAddress,
      decoration: InputDecoration(
        labelText: 'Correo Electrónico',
        prefixIcon: const Icon(Icons.email_outlined),
        border: OutlineInputBorder(
          borderRadius: BorderRadius.circular(12),

```



```

onPressed: _isLoading ? null : _handleSubmit,

child: _isLoading

? const CircularProgressIndicator(color: Colors.white)

: Text(

  _isLoading ? 'Registrarme' : 'Iniciar Sesión',

  style: const TextStyle(

    fontSize: 16,

    fontWeight: FontWeight.bold,

    color: Colors.white,

  ),

),

),

const SizedBox(height: 20),

Row(

  mainAxisAlignment: MainAxisAlignment.center,

  children: [

    Text(

      _isLoading ? '¿Ya tienes una cuenta?' : '¿Aún no tienes cuenta?',

      style: const TextStyle(color: Color(0xFF64748B)),

    ),

    TextButton(

      onPressed: () => setState(() => _isLoading = !_isLoading),

      child: Text(

        _isLoading ? 'Inicia Sesión' : 'Regístrate',

        style: const TextStyle(

```

```

        color: Color(0xFF059669),
        fontWeight: FontWeight.bold,
      ),
    ),
  ),
],
),
],
),
),
),
);
}
}

```

El código de la pantalla principal de **Dashboard de Avance en Trámites** para Flutter.

Python

```
import os
```

```
# Create directory structure for dashboard screen
```

```
dashboard_dir = "mobile_dashboard_module"
```

```
os.makedirs(os.path.join(dashboard_dir, "lib", "screens"), exist_ok=True)
```

```
# lib/screens/dashboard_screen.dart
```

```
dashboard_content = """import 'package:flutter/material.dart';
```

```

class DashboardScreen extends StatefulWidget {

  const DashboardScreen({Key? key}) : super(key: key);

  @override

  State<DashboardScreen> createState() => _DashboardScreenState();

}

```

```

class _DashboardScreenState extends State<DashboardScreen> {

  // Estado local simulación de avance de trámites

  final List<Map<String, dynamic>> _procedures = [

    {

      'title': 'SAT (Inscripción y e.firma)',

      'category': 'Fiscal',

      'progress': 0.75,

      'status': 'En revisión de cita',

      'icon': Icons.account_balance_outlined,

      'color': const Color(0xFF1E3A8A), // Azul institucional

      'completedSteps': 3,

      'totalSteps': 4,

    },

    {

      'title': 'IMPI (Registro de Marca)',

      'category': 'Propiedad Intelectual',

      'progress': 0.33,

      'status': 'Búsqueda fonética',

      'icon': Icons.verified_outlined,

```

```

        'color': const Color(0xFF059669), // Verde éxito
        'completedSteps': 1,
        'totalSteps': 3,
    },
    {
        'title': 'Permisos y Licencia Municipal',
        'category': 'Operaciones',
        'progress': 0.10,
        'status': 'Uso de suelo pendiente',
        'icon': Icons.assignment_outlined,
        'color': const Color(0xFFD97706), // Ámbar
        'completedSteps': 1,
        'totalSteps': 5,
    },
];

```

@override

```
Widget build(BuildContext context) {
```

```
    // Cálculo del promedio general de avance
```

```
    final double overallProgress = _procedures.map((p) => p['progress'] as double).reduce((a,
b) => a + b) / _size();
```

```
    return Scaffold(
```

```
        backgroundColor: const Color(0xFFFF8FAFC),
```

```
        appBar: AppBar(
```

```
            backgroundColor: Colors.white,
```

```
            elevation: 0,
```



```
title: const Column(  
  crossAxisAlignment: CrossAxisAlignment.start,  
  children: [  
    Text(  
      'Hola, Emprendedor 🙌',  
      style: TextStyle(  
        fontSize: 18,  
        fontWeight: FontWeight.bold,  
        color: Color(0xFF0F172A),  
      ),  
    ),  
    Text(  
      'Estado de Formalización',  
      style: TextStyle(fontSize: 12, color: Color(0xFF64748B)),  
    ),  
  ],  
),  
actions: [  
  IconButton(  
    icon: const Icon(Icons.notifications_none, color: Color(0xFF0F172A)),  
    onPressed: () {},  
  ),  
],  
),  
body: SingleChildScrollView(  
  padding: const EdgeInsets.all(20.0),
```

```

child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    // Target Resumen General de Progreso
    _buildOverallProgressCard(overallProgress),
    const SizedBox(height: 28),

    const Text(
      'Trámites en Curso',
      style: TextStyle(
        fontSize: 18,
        fontWeight: FontWeight.bold,
        color: Color(0xFF0F172A),
      ),
    ),
    const SizedBox(height: 16),

    // Lista de Tarjetas de Trámites
    ..._procedures.map((proc) => _buildProcedureCard(proc)).toList(),
  ],
),
);
}

int _size() => _procedures.length;

```

```

Widget _buildOverallProgressCard(double progress) {
  final int percentage = (progress * 100).round();

  return Container(
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      color: const Color(0xFF1E3A8A),
      borderRadius: BorderRadius.circular(16),
      boxShadow: [
        BoxShadow(
          color: const Color(0xFF1E3A8A).withOpacity(0.2),
          blurRadius: 10,
          offset: const Offset(0, 4),
        ),
      ],
    ),
    child: Row(
      children: [
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const Text(
                'Progreso Global',
                style: TextStyle(
                  color: Colors.white70,

```

```
        fontSize: 14,

        fontWeight: FontWeight.w500,

    ),

),

const SizedBox(height: 6),

Text(

    '$percentage% Completado',

    style: const TextStyle(

        color: Colors.white,

        fontSize: 22,

        fontWeight: FontWeight.bold,

    ),

),

const SizedBox(height: 8),

const Text(

    'Completa tus trámites para operar formalmente sin riesgos.',

    style: TextStyle(color: Colors.white70, fontSize: 12),

),

],

),

),

const SizedBox(width: 16),

Stack(

    alignment: Alignment.center,

    children: [

        SizedBox(
```

```

        width: 64,

        height: 64,

        child: CircularProgressIndicator(

          value: progress,

          strokeWidth: 7,

          backgroundColor: Colors.white24,

          valueColor: const AlwaysStoppedAnimation<Color>(Color(0xFF059669)),

        ),

      ),

      Text(

        '$percentage%',

        style: const TextStyle(

          color: Colors.white,

          fontWeight: FontWeight.bold,

          fontSize: 14,

        ),

      ),

    ],

  ),

],

),

);
}

```

```

Widget _buildProcedureCard(Map<String, dynamic> procedure) {

  final double progress = procedure['progress'];

```

```
final int percentage = (progress * 100).round();

final Color color = procedure['color'];

return Container(
  margin: const EdgeInsets.only(bottom: 16),
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(14),
    border: Border.all(color: const Color(0xFFE2E8F0)),
  ),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Row(
        children: [
          Container(
            padding: const EdgeInsets.all(10),
            decoration: BoxDecoration(
              color: color.withOpacity(0.1),
              borderRadius: BorderRadius.circular(10),
            ),
            child: Icon(procedure['icon'], color: color, size: 24),
          ),
          const SizedBox(width: 12),
          Expanded(
```

```
child: Column(  
  crossAxisAlignment: CrossAxisAlignment.start,  
  children: [  
    Text(  
      procedure['category'],  
      style: TextStyle(  
        fontSize: 11,  
        fontWeight: FontWeight.w600,  
        color: color,  
      ),  
    ),  
    Text(  
      procedure['title'],  
      style: const TextStyle(  
        fontSize: 15,  
        fontWeight: FontWeight.bold,  
        color: Color(0xFF0F172A),  
      ),  
    ),  
  ],  
),  
Text(  
  '$percentage%',  
  style: TextStyle(  
    fontSize: 14,
```

```

        fontWeight: FontWeight.bold,
        color: color,
    ),
),
],
),
const SizedBox(height: 16),
ClipRRect(
    borderRadius: BorderRadius.circular(4),
    child: LinearProgressIndicator(
        value: progress,
        minHeight: 6,
        backgroundColor: const Color(0xFFF1F5F9),
        valueColor: AlwaysStoppedAnimation<Color>(color),
    ),
),
const SizedBox(height: 12),
Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
        Text(
            'Estatus: ${procedure['status']}',
            style: const TextStyle(fontSize: 12, color: Color(0xFF64748B)),
        ),
        Text(
            'Paso ${procedure['completedSteps']} de ${procedure['totalSteps']}',

```



```

        style: const TextStyle(
          fontSize: 12,
          fontWeight: FontWeight.w500,
          color: Color(0xFF0F172A),
        ),
      ),
    ],
  ),
],
),
);
}
}
"""

```

with open(os.path.join(dashboard_dir, "lib", "screens", "dashboard_screen.dart"), "w", encoding="utf-8") as f:

```
f.write(dashboard_content)
```

Zip files

```
import shutil
```

```
shutil.make_archive("mobile_dashboard_module", 'zip', dashboard_dir)
```

```
print("Dashboard module created and zipped.")
```

Características de la Pantalla (**lib/screens/dashboard_screen.dart**)

- **Métrica General Promedio:** Calcula y muestra en un widget tipo dona el porcentaje global de formalización del emprendimiento.

- **Seguimiento por Módulo Fiscal/Legal:** Tarjetas independientes para **SAT**, **IMPI** y **Permisos Municipales** con barras de progreso ([LinearProgressIndicator](#)).
- **Estado de Etapas:** Desglose del número de pasos completados y el estatus actual de cada trámite.

```
import 'package:flutter/material.dart';
```

```
class DashboardScreen extends StatefulWidget {
```

```
  const DashboardScreen({Key? key}) : super(key: key);
```

```
  @override
```

```
  State<DashboardScreen> createState() => _DashboardScreenState();
```

```
}
```

```
class _DashboardScreenState extends State<DashboardScreen> {
```

```
  // Estado local simulación de avance de trámites
```

```
  final List<Map<String, dynamic>> _procedures = [
```

```
    {
```

```
      'title': 'SAT (Inscripción y e.firma)',
```

```
      'category': 'Fiscal',
```

```
      'progress': 0.75,
```

```
      'status': 'En revisión de cita',
```

```
      'icon': Icons.account_balance_outlined,
```

```
      'color': const Color(0xFF1E3A8A), // Azul institucional
```

```
      'completedSteps': 3,
```

```
      'totalSteps': 4,
```

```
    },
```

```
    {
```

```
      'title': 'IMPI (Registro de Marca)',
```

```

        'category': 'Propiedad Intelectual',
        'progress': 0.33,
        'status': 'Búsqueda fonética',
        'icon': Icons.verified_outlined,
        'color': const Color(0xFF059669), // Verde éxito
        'completedSteps': 1,
        'totalSteps': 3,
    },
    {
        'title': 'Permisos y Licencia Municipal',
        'category': 'Operaciones',
        'progress': 0.10,
        'status': 'Uso de suelo pendiente',
        'icon': Icons.assignment_outlined,
        'color': const Color(0xFFD97706), // Ámbar
        'completedSteps': 1,
        'totalSteps': 5,
    },
];

```

```

@override

```

```

Widget build(BuildContext context) {

```

```

    // Cálculo del promedio general de avance

```

```

    final double overallProgress = _procedures.map((p) => p['progress'] as double).reduce((a,
b) => a + b) / _size());

```

```

    return Scaffold(

```

```
backgroundColor: const Color(0xFFFF8FAFC),

appBar: AppBar(

  backgroundColor: Colors.white,

  elevation: 0,

  title: const Column(

    crossAxisAlignment: CrossAxisAlignment.start,

    children: [

      Text(

        'Hola, Emprendedor 🖐️',

        style: TextStyle(

          fontSize: 18,

          fontWeight: FontWeight.bold,

          color: Color(0xFF0F172A),

        ),

      ),

      Text(

        'Estado de Formalización',

        style: TextStyle(fontSize: 12, color: Color(0xFF64748B)),

      ),

    ],

  ),

  actions: [

    IconButton(

      icon: const Icon(Icons.notifications_none, color: Color(0xFF0F172A)),

      onPressed: () {},

    ),
```

```

],
),
body: SingleChildScrollView(
  padding: const EdgeInsets.all(20.0),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      // Target Resumen General de Progreso
      _buildOverallProgressCard(overallProgress),
      const SizedBox(height: 28),

      const Text(
        'Trámites en Curso',
        style: TextStyle(
          fontSize: 18,
          fontWeight: FontWeight.bold,
          color: Color(0xFF0F172A),
        ),
      ),
      const SizedBox(height: 16),

      // Lista de Tarjetas de Trámites
      ..._procedures.map((proc) => _buildProcedureCard(proc)).toList(),
    ],
  ),
),

```

```
);  
}
```

```
int _size() => _procedures.length;
```

```
Widget _buildOverallProgressCard(double progress) {  
  final int percentage = (progress * 100).round();  
  return Container(  
    padding: const EdgeInsets.all(20),  
    decoration: BoxDecoration(  
      color: const Color(0xFF1E3A8A),  
      borderRadius: BorderRadius.circular(16),  
      boxShadow: [  
        BoxShadow(  
          color: const Color(0xFF1E3A8A).withOpacity(0.2),  
          blurRadius: 10,  
          offset: const Offset(0, 4),  
        ),  
      ],  
    ),  
    child: Row(  
      children: [  
        Expanded(  
          child: Column(  
            crossAxisAlignment: CrossAxisAlignment.start,  
            children: [  
              Text(percentage.toString()),  
              Text('Percentage'),  
            ],  
          ),  
        ],  
      ),  
    ),  
  );  
}
```

```
const Text(  
  'Progreso Global',  
  style: TextStyle(  
    color: Colors.white70,  
    fontSize: 14,  
    fontWeight: FontWeight.w500,  
  ),  
,  
const SizedBox(height: 6),  
Text(  
  '$percentage% Completado',  
  style: const TextStyle(  
    color: Colors.white,  
    fontSize: 22,  
    fontWeight: FontWeight.bold,  
  ),  
,  
const SizedBox(height: 8),  
const Text(  
  'Completa tus trámites para operar formalmente sin riesgos.',  
  style: TextStyle(color: Colors.white70, fontSize: 12),  
,  
],  
,  
,  
const SizedBox(width: 16),
```

```
Stack(  
  alignment: Alignment.center,  
  children: [  
    SizedBox(  
      width: 64,  
      height: 64,  
      child: CircularProgressIndicator(  
        value: progress,  
        strokeWidth: 7,  
        backgroundColor: Colors.white24,  
        valueColor: const AlwaysStoppedAnimation<Color>(Color(0xFF059669)),  
      ),  
    ),  
    Text(  
      '$percentage%',  
      style: const TextStyle(  
        color: Colors.white,  
        fontWeight: FontWeight.bold,  
        fontSize: 14,  
      ),  
    ),  
  ],  
,  
],  
,  
);
```



```
}
```

```
Widget _buildProcedureCard(Map<String, dynamic> procedure) {
```

```
  final double progress = procedure['progress'];
```

```
  final int percentage = (progress * 100).round();
```

```
  final Color color = procedure['color'];
```

```
  return Container(
```

```
    margin: const EdgeInsets.only(bottom: 16),
```

```
    padding: const EdgeInsets.all(16),
```

```
    decoration: BoxDecoration(
```

```
      color: Colors.white,
```

```
      borderRadius: BorderRadius.circular(14),
```

```
      border: Border.all(color: const Color(0xFFE2E8F0)),
```

```
    ),
```

```
    child: Column(
```

```
      crossAxisAlignment: CrossAxisAlignment.start,
```

```
      children: [
```

```
        Row(
```

```
          children: [
```

```
            Container(
```

```
              padding: const EdgeInsets.all(10),
```

```
              decoration: BoxDecoration(
```

```
                color: color.withOpacity(0.1),
```

```
                borderRadius: BorderRadius.circular(10),
```

```
            ),
```

```
        child: Icon(procedure['icon'], color: color, size: 24),
      ),
      const SizedBox(width: 12),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              procedure['category'],
              style: TextStyle(
                fontSize: 11,
                fontWeight: FontWeight.w600,
                color: color,
              ),
            ),
            Text(
              procedure['title'],
              style: const TextStyle(
                fontSize: 15,
                fontWeight: FontWeight.bold,
                color: Color(0xFF0F172A),
              ),
            ),
          ],
        ),
      ),
    ),
  ),
```

```

Text(
  '$percentage%',
  style: TextStyle(
    fontSize: 14,
    fontWeight: FontWeight.bold,
    color: color,
  ),
),
],
),
const SizedBox(height: 16),
ClipRRect(
  borderRadius: BorderRadius.circular(4),
  child: LinearProgressIndicator(
    value: progress,
    minHeight: 6,
    backgroundColor: const Color(0xFFFF1F5F9),
    valueColor: AlwaysStoppedAnimation<Color>(color),
  ),
),
const SizedBox(height: 12),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Text(
      'Estatus: ${procedure['status']}',

```

```

        style: const TextStyle(fontSize: 12, color: Color(0xFF64748B)),
      ),
      Text(
        'Paso ${procedure['completedSteps']} de ${procedure['totalSteps']}',
        style: const TextStyle(
          fontSize: 12,
          fontWeight: FontWeight.w500,
          color: Color(0xFF0F172A),
        ),
      ),
    ],
  ),
],
),
);
}
}

```

El código completo de la pantalla del **Marketplace de Asesores** y el flujo de reservación para Flutter.

Python

```
import os
```

```
marketplace_dir = "mobile_marketplace_module"
```

```
os.makedirs(os.path.join(marketplace_dir, "lib", "screens"), exist_ok=True)
```

```
# lib/screens/marketplace_screen.dart
```

```
marketplace_content = ""import 'package:flutter/material.dart';
```

```
class MarketplaceScreen extends StatefulWidget {
```

```
  const MarketplaceScreen({Key? key}) : super(key: key);
```

```
  @override
```

```
  State<MarketplaceScreen> createState() => _MarketplaceScreenState();
```

```
}
```

```
class _MarketplaceScreenState extends State<MarketplaceScreen> {
```

```
  String _selectedCategory = 'Todos';
```

```
  final List<Map<String, dynamic>> _advisors = [
```

```
    {
```

```
      'id': '1',
```

```
      'name': 'Lic. Sofía García',
```

```
      'specialty': 'Derecho Corporativo e IMPI',
```

```
      'category': 'Legal',
```

```
      'rating': 4.9,
```

```
      'reviewsCount': 28,
```

```
      'hourlyRate': 650,
```

```
      'experienceYears': 8,
```

```
      'isAvailable': true,
```

```
      'avatarUrl': 'https://i.pravatar.cc/150?img=47',
```

```
    },
```

```
    {
```

```
'id': '2',  
  
'name': 'C.P. Alejandro Morales',  
  
'specialty': 'Especialista en RESICO y SAT',  
  
'category': 'Fiscal',  
  
'rating': 4.8,  
  
'reviewsCount': 42,  
  
'hourlyRate': 500,  
  
'experienceYears': 10,  
  
'isAvailable': true,  
  
'avatarUrl': 'https://i.pravatar.cc/150?img=12',  
  
},  
  
{  
  
  'id': '3',  
  
  'name': 'Lic. Fernando Rios',  
  
  'specialty': 'Contratos Laborales LFT',  
  
  'category': 'Legal',  
  
  'rating': 4.7,  
  
  'reviewsCount': 19,  
  
  'hourlyRate': 550,  
  
  'experienceYears': 6,  
  
  'isAvailable': false,  
  
  'avatarUrl': 'https://i.pravatar.cc/150?img=33',  
  
},  
  
];
```

@override

```

Widget build(BuildContext context) {

  final filteredAdvisors = _selectedCategory == 'Todos'

    ? _advisors

    : _advisors.where((a) => a['category'] == _selectedCategory).toList();


  return Scaffold(

    backgroundColor: const Color(0xFFFF8FAFC),

    appBar: AppBar(

      backgroundColor: Colors.white,

      elevation: 0,

      title: const Text(

        'Marketplace de Asesores',

        style: TextStyle(

          fontSize: 18,

          fontWeight: FontWeight.bold,

          color: Color(0xFF0F172A),

        ),

      ),

    ),

    body: Column(

      children: [

        // Filtros de categoría

        Container(

          color: Colors.white,

          padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),

          child: Row(

```

```

children: ['Todos', 'Fiscal', 'Legal', 'Operaciones'].map((cat) {

  final isSelected = _selectedCategory == cat;

  return Padding(

    padding: const EdgeInsets.only(right: 8.0),

    child: FilterChip(

      label: Text(cat),

      selected: isSelected,

      selectedColor: const Color(0xFF1E3A8A).withOpacity(0.15),

      checkmarkColor: const Color(0xFF1E3A8A),

      labelStyle: TextStyle(

        color: isSelected ? const Color(0xFF1E3A8A) : const Color(0xFF64748B),

        fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,

      ),

      onSelect: (_) => setState(() => _selectedCategory = cat),

    ),

  );

}).toList(),

),

),

const SizedBox(height: 12),

// Lista de asesores

Expanded(

  child: ListView.builder(

    padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),

    itemCount: filteredAdvisors.length,

```



```

    itemBuilder: (context, index) {
        final advisor = filteredAdvisors[index];
        return _buildAdvisorCard(context, advisor);
    },
),
),
],
),
);
}

```

```

Widget _buildAdvisorCard(BuildContext context, Map<String, dynamic> advisor) {
    return Container(
        margin: const EdgeInsets.only(bottom: 16),
        padding: const EdgeInsets.all(16),
        decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(16),
            border: Border.all(color: const Color(0xFFE2E8F0)),
        ),
        child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
                Row(
                    children: [
                        CircleAvatar(

```

```
radius: 28,

backgroundImage: NetworkImage(advisor['avatarUrl']),

),

const SizedBox(width: 12),

Expanded(

child: Column(

crossAxisAlignment: CrossAxisAlignment.start,

children: [

Text(

advisor['name'],

style: const TextStyle(

fontSize: 16,

fontWeight: FontWeight.bold,

color: Color(0xFF0F172A),

),

),

const SizedBox(height: 2),

Text(

advisor['specialty'],

style: const TextStyle(fontSize: 12, color: Color(0xFF64748B)),

),

const SizedBox(height: 4),

Row(

children: [

const Icon(Icons.star_rounded, size: 16, color: Color(0xFFD97706)),

const SizedBox(width: 4),
```

```

        Text(
          '${advisor['rating']} (${advisor['reviewsCount']} reseñas)',
          style: const TextStyle(fontSize: 12, fontWeight: FontWeight.w600),
        ),
      ],
    ),
  ],
),
),
],
),
const Padding(
  padding: EdgeInsets.symmetric(vertical: 12),
  child: Divider(height: 1, color: Color(0xFFFF1F5F9)),
),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Text('Precio por hora', style: TextStyle(fontSize: 11, color:
Color(0xFF64748B))),
        Text(
          '\${advisor['hourlyRate']} MXN',
          style: const TextStyle(
            fontSize: 16,

```

```

        fontWeight: FontWeight.bold,
        color: Color(0xFF059669),
    ),
),
],
),
ElevatedButton(
    style: ElevatedButton.styleFrom(
        backgroundColor: const Color(0xFF1E3A8A),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
        padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),
    ),
    onPressed: () => _showBookingModal(context, advisor),
    child: const Text('Agendar Cita', style: TextStyle(color: Colors.white, fontWeight:
FontWeight.bold)),
),
],
),
],
),
);
}

```

```

void _showBookingModal(BuildContext context, Map<String, dynamic> advisor) {
    showModalBottomSheet(
        context: context,
        isScrollControlled: true,

```

```

    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(20)),
    ),
    builder: (context) => _BookingBottomSheet(advisor: advisor),
  );
}
}

```

```

class _BookingBottomSheet extends StatefulWidget {
  final Map<String, dynamic> advisor;

  const _BookingBottomSheet({Key? key, required this.advisor}) : super(key: key);

  @override
  State<_BookingBottomSheet> createState() => _BookingBottomSheetState();
}

```

```

class _BookingBottomSheetState extends State<_BookingBottomSheet> {
  DateTime _selectedDate = DateTime.now().add(const Duration(days: 1));
  String _selectedTimeSlot = '10:00 AM';

  final List<String> _timeSlots = ['09:00 AM', '10:00 AM', '11:30 AM', '03:00 PM', '04:30 PM'];

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: EdgeInsets.only(

```

```

left: 20,

right: 20,

top: 20,

bottom: MediaQuery.of(context).viewInsets.bottom + 20,

),

child: Column(

  mainAxisAlignment: MainAxisAlignment.min,

  crossAxisAlignment: CrossAxisAlignment.start,

  children: [

    Text(

      'Agendar con ${widget.advisor['name']}',

      style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold, color:
Color(0xFF0F172A)),

    ),

    const SizedBox(height: 16),

    const Text('Selecciona la fecha:', style: TextStyle(fontWeight: FontWeight.w600)),

    const SizedBox(height: 8),

    OutlinedButton.icon(

      icon: const Icon(Icons.calendar_today_rounded, size: 18),

      label: Text('${_selectedDate.day}/${_selectedDate.month}/${_selectedDate.year}'),

      onPressed: () async {

        final picked = await showDatePicker(

          context: context,

          initialDate: _selectedDate,

          firstDate: DateTime.now(),

          lastDate: DateTime.now().add(const Duration(days: 30)),

        );

```

```

        if (picked != null) setState(() => _selectedDate = picked);
    },
),
const SizedBox(height: 16),
const Text('Horarios disponibles:', style: TextStyle(fontWeight: FontWeight.w600)),
const SizedBox(height: 8),
Wrap(
    spacing: 8,
    children: _timeSlots.map((slot) {
        final isSelected = _selectedTimeSlot == slot;
        return ChoiceChip(
            label: Text(slot),
            selected: isSelected,
            selectedColor: const Color(0xFF059669),
            labelStyle: TextStyle(color: isSelected ? Colors.white : Colors.black87),
            onSelect: (_) => setState(() => _selectedTimeSlot = slot),
        );
    }).toList(),
),
const SizedBox(height: 24),
SizedBox(
    width: double.infinity,
    height: 50,
    child: ElevatedButton(
        style: ElevatedButton.styleFrom(
            backgroundColor: const Color(0xFF059669),

```

```

        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    ),
    onPressed: () {
        Navigator.pop(context);

        ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(
                content: Text('Cita agendada para el
$_selectedDate.day}/${_selectedDate.month} a las $_selectedTimeSlot'),
                backgroundColor: const Color(0xFF059669),
            ),
        );
    },
    child: Text(
        'Pagar y Confirmar (\${widget.advisor['hourlyRate']} MXN)',
        style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bold, fontSize:
16),
    ),
),
),
),
],
),
);
}
}

```

with open(os.path.join(marketplace_dir, "lib", "screens", "marketplace_screen.dart"), "w", encoding="utf-8") as f:


```
f.write(marketplace_content)
```

```
import shutil
```

```
shutil.make_archive("mobile_marketplace_module", 'zip', marketplace_dir)
```

```
print("Marketplace module created and zipped.")
```

Características de la Pantalla (**lib/screens/marketplace_screen.dart**)

- **Filtrado dinámico por categoría:** Los usuarios pueden filtrar asesores entre categorías como *Fiscal*, *Legal* u *Operaciones*.
- **Tarjeta de Perfil de Asesor:** Incluye calificación (rating de estrellas), especialidad, años de experiencia y la tarifa por hora expresada en **MXN**.
- **Modal Inferior de Agenda (**BottomSheet**):** Permite al emprendedor seleccionar una fecha mediante un calendario nativo y elegir entre horarios disponibles para proceder con el pago y confirmación.

```
import 'package:flutter/material.dart';
```

```
class MarketplaceScreen extends StatefulWidget {
```

```
  const MarketplaceScreen({Key? key}) : super(key: key);
```

```
  @override
```

```
  State<MarketplaceScreen> createState() => _MarketplaceScreenState();
```

```
}
```

```
class _MarketplaceScreenState extends State<MarketplaceScreen> {
```

```
  String _selectedCategory = 'Todos';
```

```
  final List<Map<String, dynamic>> _advisors = [
```

```
    {
```

```
'id': '1',

'name': 'Lic. Sofía García',

'specialty': 'Derecho Corporativo e IMPI',

'category': 'Legal',

'rating': 4.9,

'reviewsCount': 28,

'hourlyRate': 650,

'experienceYears': 8,

'isAvailable': true,

'avatarUrl': 'https://i.pravatar.cc/150?img=47',

},

{

'id': '2',

'name': 'C.P. Alejandro Morales',

'specialty': 'Especialista en RESICO y SAT',

'category': 'Fiscal',

'rating': 4.8,

'reviewsCount': 42,

'hourlyRate': 500,

'experienceYears': 10,

'isAvailable': true,

'avatarUrl': 'https://i.pravatar.cc/150?img=12',

},

{

'id': '3',

'name': 'Lic. Fernando Rios',
```

```
'specialty': 'Contratos Laborales LFT',  
'category': 'Legal',  
'rating': 4.7,  
'reviewsCount': 19,  
'hourlyRate': 550,  
'experienceYears': 6,  
'isAvailable': false,  
'avatarUrl': 'https://i.pravatar.cc/150?img=33',  
},  
];
```

@override

```
Widget build(BuildContext context) {  
  final filteredAdvisors = _selectedCategory == 'Todos'  
    ? _advisors  
    : _advisors.where((a) => a['category'] == _selectedCategory).toList();  
  
  return Scaffold(  
    backgroundColor: const Color(0xFFFF8FAFC),  
    appBar: AppBar(  
      backgroundColor: Colors.white,  
      elevation: 0,  
      title: const Text(  
        'Marketplace de Asesores',  
        style: TextStyle(  
          fontSize: 18,
```

```

fontWeight: FontWeight.bold,

color: Color(0xFF0F172A),

),

),

),

body: Column(

  children: [

    // Filtros de categoría

    Container(

      color: Colors.white,

      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),

      child: Row(

        children: ['Todos', 'Fiscal', 'Legal', 'Operaciones'].map((cat) {

          final isSelected = _selectedCategory == cat;

          return Padding(

            padding: const EdgeInsets.only(right: 8.0),

            child: FilterChip(

              label: Text(cat),

              selected: isSelected,

              selectedColor: const Color(0xFF1E3A8A).withOpacity(0.15),

              checkmarkColor: const Color(0xFF1E3A8A),

              labelStyle: TextStyle(

                color: isSelected ? const Color(0xFF1E3A8A) : const Color(0xFF64748B),

                fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,

              ),

              onSelect: (_) => setState(() => _selectedCategory = cat),

```

```

        ),
    );
    }).toList(),
),
),
const SizedBox(height: 12),

// Lista de asesores
Expanded(
  child: ListView.builder(
    padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
    itemCount: filteredAdvisors.length,
    itemBuilder: (context, index) {
      final advisor = filteredAdvisors[index];
      return _buildAdvisorCard(context, advisor);
    },
  ),
),
],
),
);
}

```

```

Widget _buildAdvisorCard(BuildContext context, Map<String, dynamic> advisor) {
  return Container(
    margin: const EdgeInsets.only(bottom: 16),

```

```
padding: const EdgeInsets.all(16),

decoration: BoxDecoration(

  color: Colors.white,

  borderRadius: BorderRadius.circular(16),

  border: Border.all(color: const Color(0xFFE2E8F0)),

),

child: Column(

  crossAxisAlignment: CrossAxisAlignment.start,

  children: [

    Row(

      children: [

        CircleAvatar(

          radius: 28,

          backgroundImage: NetworkImage(advisor['avatarUrl']),

        ),

        const SizedBox(width: 12),

        Expanded(

          child: Column(

            crossAxisAlignment: CrossAxisAlignment.start,

            children: [

              Text(

                advisor['name'],

                style: const TextStyle(

                  fontSize: 16,

                  fontWeight: FontWeight.bold,

                  color: Color(0xFF0F172A),
```

```

    ),
  ),
  const SizedBox(height: 2),
  Text(
    advisor['specialty'],
    style: const TextStyle(fontSize: 12, color: Color(0xFF64748B)),
  ),
  const SizedBox(height: 4),
  Row(
    children: [
      const Icon(Icons.star_rounded, size: 16, color: Color(0xFFD97706)),
      const SizedBox(width: 4),
      Text(
        '${advisor['rating']} (${advisor['reviewsCount']} reseñas)',
        style: const TextStyle(fontSize: 12, fontWeight: FontWeight.w600),
      ),
    ],
  ),
],
),
],
),
],
),
const Padding(
  padding: EdgeInsets.symmetric(vertical: 12),
  child: Divider(height: 1, color: Color(0xFFFF1F5F9)),

```

```

    ),
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text('Precio por hora', style: TextStyle(fontSize: 11, color:
Color(0xFF64748B))),
            Text(
              '\${advisor['hourlyRate']} MXN',
              style: const TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.bold,
                color: Color(0xFF059669),
              ),
            ),
          ],
        ),
        ElevatedButton(
          style: ElevatedButton.styleFrom(
            backgroundColor: const Color(0xFF1E3A8A),
            shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
            padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),
          ),
          onPressed: () => _showBookingModal(context, advisor),

```



```

        child: const Text('Agendar Cita', style: TextStyle(color: Colors.white, fontWeight:
FontWeight.bold)),
      ),
    ],
  ),
],
),
);
}

```

```

void _showBookingModal(BuildContext context, Map<String, dynamic> advisor) {
  showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(20)),
    ),
    builder: (context) => _BookingBottomSheet(advisor: advisor),
  );
}
}

```

```

class _BookingBottomSheet extends StatefulWidget {
  final Map<String, dynamic> advisor;

  const _BookingBottomSheet({Key? key, required this.advisor}) : super(key: key);

  @override

```

```
State<_BookingBottomSheet> createState() => _BookingBottomSheetState();  
}
```

```
class _BookingBottomSheetState extends State<_BookingBottomSheet> {
```

```
  DateTime _selectedDate = DateTime.now().add(const Duration(days: 1));
```

```
  String _selectedTimeSlot = '10:00 AM';
```

```
  final List<String> _timeSlots = ['09:00 AM', '10:00 AM', '11:30 AM', '03:00 PM', '04:30 PM'];
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return Padding(  
      padding: EdgeInsets.only(  
        left: 20,  
        right: 20,  
        top: 20,  
        bottom: MediaQuery.of(context).viewInsets.bottom + 20,  
      ),  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.min,  
        crossAxisAlignment: CrossAxisAlignment.start,  
        children: [  
          Text(  
            'Agendar con ${widget.advisor['name']}',  
            style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold, color:  
              Color(0xFF0F172A)),  
          ),  
        ],  
      ),  
    );  
  }
```

```

const SizedBox(height: 16),

const Text('Selecciona la fecha:', style: TextStyle(fontWeight: FontWeight.w600)),

const SizedBox(height: 8),

OutlinedButton.icon(

  icon: const Icon(Icons.calendar_today_rounded, size: 18),

  label: Text('${_selectedDate.day}/${_selectedDate.month}/${_selectedDate.year}'),

  onPressed: () async {

    final picked = await showDatePicker(

      context: context,

      initialDate: _selectedDate,

      firstDate: DateTime.now(),

      lastDate: DateTime.now().add(const Duration(days: 30)),

    );

    if (picked != null) setState(() => _selectedDate = picked);

  },

),

const SizedBox(height: 16),

const Text('Horarios disponibles:', style: TextStyle(fontWeight: FontWeight.w600)),

const SizedBox(height: 8),

Wrap(

  spacing: 8,

  children: _timeSlots.map((slot) {

    final isSelected = _selectedTimeSlot == slot;

    return ChoiceChip(

      label: Text(slot),

      selected: isSelected,

```

```

        selectedColor: const Color(0xFF059669),

        labelStyle: TextStyle(color: isSelected ? Colors.white : Colors.black87),

        onPressed: (_) => setState(() => _selectedTimeSlot = slot),

    );

  }).toList(),

),

const SizedBox(height: 24),

SizedBox(

  width: double.infinity,

  height: 50,

  child: ElevatedButton(

    style: ElevatedButton.styleFrom(

      backgroundColor: const Color(0xFF059669),

      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),

    ),

    onPressed: () {

      Navigator.pop(context);

      ScaffoldMessenger.of(context).showSnackBar(

        SnackBar(

          content: Text('Cita agendada para el
$_selectedDate.day}/${_selectedDate.month} a las $_selectedTimeSlot'),

          backgroundColor: const Color(0xFF059669),

        ),

      );

    },

    child: Text(

      'Pagar y Confirmar (\${widget.advisor['hourlyRate']} MXN)',

```

```

        style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bold, fontSize:
16),
      ),
    ),
  ),
],
),
);
}
}

```

La integración nativa del **SDK de Stripe en Flutter** (`flutter_stripe`) con la pasarela de pagos tipo `PaymentSheet` conectada al botón de confirmación del Marketplace.

Python

```
import os
```

```
stripe_dir = "mobile_stripe_module"
```

```
os.makedirs(os.path.join(stripe_dir, "lib", "services"), exist_ok=True)
```

```
os.makedirs(os.path.join(stripe_dir, "lib", "screens"), exist_ok=True)
```

1. Payment Service implementation

```
payment_service_content = """import 'dart:convert';
```

```
import 'package:flutter/material.dart';
```

```
import 'package:flutter_stripe/flutter_stripe.dart';
```

```
import 'package:http/http' as http;
```

```
class PaymentService {
```

```

// URL de tu Backend o Edge Function de Supabase

static const String _backendUrl =
'https://tu-proyecto.supabase.co/functions/v1/create-payment-intent';

/// Procesa el pago de la cita con Stripe PaymentSheet

static Future<bool> makePayment({
  required BuildContext context,
  required String appointmentId,
  required double amount,
  required String currency,
}) async {
  try {
    // 1. Obtener el PaymentIntent Client Secret desde el Backend

    final paymentIntentData = await _createPaymentIntent(
      appointmentId: appointmentId,
      amount: amount,
      currency: currency,
    );

    final clientSecret = paymentIntentData['clientSecret'];
    final customerId = paymentIntentData['customer'];
    final ephemeralKey = paymentIntentData['ephemeralKey'];

    // 2. Inicializar el PaymentSheet de Stripe

    await Stripe.instance.initPaymentSheet(
      paymentSheetParameters: SetupPaymentSheetParameters(
        paymentIntentClientSecret: clientSecret,

```

```

        customerId: customerId,

        customerEphemeralKeySecret: ephemeralKey,

        merchantDisplayName: 'Consultoría para todos',

        style: ThemeMode.light,

        appearance: const PaymentSheetAppearance(
          colors: PaymentSheetAppearanceColors(
            primary: Color(0xFF059669), // Verde de la marca
          ),
        ),
      ),
    );

// 3. Mostrar la pasarela de pago al usuario
await Stripe.instance.presentPaymentSheet();

// Pago completado exitosamente
return true;

} on StripeException catch (e) {
  if (mountedContext(context)) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Pago cancelado o fallido: ${e.error.localizedMessage}'),
        backgroundColor: Colors.red,
      ),
    );
  }
};

```

```

    }

    return false;
} catch (e) {
    if (mountedContext(context)) {
        ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(
                content: Text('Error inesperado al procesar el pago: $e'),
                backgroundColor: Colors.red,
            ),
        );
    }
    return false;
}
}

```

/// Petición HTTP al Backend para crear el PaymentIntent

```

static Future<Map<String, dynamic>> _createPaymentIntent({
    required String appointmentId,
    required double amount,
    required String currency,
}) async {
    final response = await http.post(
        Uri.parse(_backendUrl),
        headers: {'Content-Type': 'application/json'},
        body: jsonEncode({
            'appointment_id': appointmentId,

```



```

        'amount': (amount * 100).toInt(), // Conversión a centavos
        'currency': currency.toLowerCase(),
    }},
);

if (response.statusCode == 200) {
    return jsonDecode(response.body);
} else {
    throw Exception('Error al comunicarse con el servidor de pagos');
}
}

```

```

static bool mountedContext(BuildContext context) {
    return context.mounted;
}

}

""""

```

```

with open(os.path.join(stripe_dir, "lib", "services", "payment_service.dart"), "w",
encoding="utf-8") as f:

```

```

    f.write(payment_service_content)

```

2. Integration into Booking Sheet Button

```

booking_update_content = """import 'package:flutter/material.dart';
import '../services/payment_service.dart';

```

```

class PaymentConfirmButton extends StatefulWidget {

```

```
final Map<String, dynamic> advisor;
```

```
final DateTime selectedDate;
```

```
final String selectedTimeSlot;
```

```
const PaymentConfirmButton({
```

```
  Key? key,
```

```
  required this.advisor,
```

```
  required this.selectedDate,
```

```
  required this.selectedTimeSlot,
```

```
}) : super(key: key);
```

```
@override
```

```
State<PaymentConfirmButton> createState() => _PaymentConfirmButtonState();
```

```
}
```

```
class _PaymentConfirmButtonState extends State<PaymentConfirmButton> {
```

```
  bool _isProcessing = false;
```

```
  Future<void> _handlePayment() async {
```

```
    setState(() => _isProcessing = true);
```

```
    // Simulación de ID de cita generado en Supabase
```

```
    final String appointmentId = "app_${DateTime.now().millisecondsSinceEpoch}";
```

```
    final double amount = (widget.advisor['hourlyRate'] as num).toDouble();
```

```
    final bool paymentSuccess = await PaymentService.makePayment(
```

```

context: context,

appointmentId: appointmentId,

amount: amount,

currency: 'mxn',

);

setState(() => _isProcessing = false);

if (paymentSuccess && mounted) {

  Navigator.pop(context); // Cerrar Modal

  ScaffoldMessenger.of(context).showSnackBar(

    SnackBar(

      content: Text(

         ¡Pago recibido! Cita confirmada para el
        ${widget.selectedDate.day}/${widget.selectedDate.month}
        ${widget.selectedTimeSlot}.',

      ),

      backgroundColor: const Color(0xFF059669),

      duration: const Duration(seconds: 4),

    ),

  );

}

}

```

@override

```

Widget build(BuildContext context) {

  return SizedBox(

```

```

width: double.infinity,

height: 50,

child: ElevatedButton(

  style: ElevatedButton.styleFrom(

    backgroundColor: const Color(0xFF059669),

    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),

  ),

  onPressed: _isProcessing ? null : _handlePayment,

  child: _isProcessing

    ? const SizedBox(

      width: 24,

      height: 24,

      child: CircularProgressIndicator(color: Colors.white, strokeWidth: 2),

    )

    : Text(

      'Pagar y Confirmar (\${widget.advisor['hourlyRate']} MXN)',

      style: const TextStyle(

        color: Colors.white,

        fontWeight: FontWeight.bold,

        fontSize: 16,

      ),

    ),

  ),

);
}
}

```

```
""
```

```
with open(os.path.join(stripe_dir, "lib", "screens", "payment_confirm_button.dart"), "w",  
encoding="utf-8") as f:
```

```
    f.write(stripe_update_content if 'stripe_update_content' in locals() else  
booking_update_content)
```

```
import shutil
```

```
shutil.make_archive("mobile_stripe_module", 'zip', stripe_dir)
```

```
print("Stripe module created and zipped.")
```

1. Servicio de Pago (**lib/services/payment_service.dart**)

Pide las claves del pago (**clientSecret**, **ephemeralKey**) al Backend y despliega la UI nativa de Stripe PaymentSheet.

```
import 'dart:convert';
```

```
import 'package:flutter/material.dart';
```

```
import 'package:flutter_stripe/flutter_stripe.dart';
```

```
import 'package:http/http' as http;
```

```
class PaymentService {
```

```
    // URL de tu Backend o Edge Function de Supabase
```

```
    static const String _backendUrl =  
'https://tu-proyecto.supabase.co/functions/v1/create-payment-intent';
```

```
    /// Procesa el pago de la cita con Stripe PaymentSheet
```

```
    static Future<bool> makePayment({
```

```
        required BuildContext context,
```

```
        required String appointmentId,
```

```
required double amount,
required String currency,
}) async {
  try {
    // 1. Obtener el PaymentIntent Client Secret desde el Backend

    final paymentIntentData = await _createPaymentIntent(
      appointmentId: appointmentId,
      amount: amount,
      currency: currency,
    );

    final clientSecret = paymentIntentData['clientSecret'];
    final customerId = paymentIntentData['customer'];
    final ephemeralKey = paymentIntentData['ephemeralKey'];

    // 2. Inicializar el PaymentSheet de Stripe

    await Stripe.instance.initPaymentSheet(
      paymentSheetParameters: SetupPaymentSheetParameters(
        paymentIntentClientSecret: clientSecret,
        customerId: customerId,
        customerEphemeralKeySecret: ephemeralKey,
        merchantDisplayName: 'Consultoría para todos',
        style: ThemeMode.light,
        appearance: const PaymentSheetAppearance(
          colors: PaymentSheetAppearanceColors(
            primary: Color(0xFF059669), // Verde de la marca
```

```
    ),  
    ),  
    ),  
);
```

```
// 3. Mostrar la pasarela de pago al usuario
```

```
await Stripe.instance.presentPaymentSheet();
```

```
// Pago completado exitosamente
```

```
return true;
```

```
} on StripeException catch (e) {
```

```
  if (mountedContext(context)) {
```

```
    ScaffoldMessenger.of(context).showSnackBar(  
      SnackBar(  
        content: Text('Pago cancelado o fallido: ${e.error.localizedMessage}'),  
        backgroundColor: Colors.red,  
      ),  
    );  
  }
```

```
    return false;
```

```
  } catch (e) {  
    if (mountedContext(context)) {  
      ScaffoldMessenger.of(context).showSnackBar(  
        SnackBar(  
          content: Text('Error inesperado al procesar el pago: $e'),
```

```

        backgroundColor: Colors.red,

    ),

);

}

return false;

}

}

```

/// Petición HTTP al Backend para crear el PaymentIntent

```

static Future<Map<String, dynamic>> _createPaymentIntent({

    required String appointmentId,

    required double amount,

    required String currency,

}) async {

    final response = await http.post(

        Uri.parse(_backendUrl),

        headers: {'Content-Type': 'application/json'},

        body: jsonEncode({

            'appointment_id': appointmentId,

            'amount': (amount * 100).toInt(), // Conversión a centavos

            'currency': currency.toLowerCase(),

        })),

    );

    if (response.statusCode == 200) {

        return jsonDecode(response.body);
    }
}

```



```

    } else {
        throw Exception('Error al comunicarse con el servidor de pagos');
    }
}

```

```

static bool mountedContext(BuildContext context) {
    return context.mounted;
}

```

2. Botón 'Pagar y Confirmar' Integrado ([lib/screens/payment_confirm_button.dart](#))

Reemplaza el botón simple dentro del modal del Marketplace por este componente interactivo con manejo de estados ([loading](#), error y respuesta exitosa).

```

import 'package:flutter/material.dart';

import '../services/payment_service.dart';

```

```

class PaymentConfirmButton extends StatefulWidget {
    final Map<String, dynamic> advisor;

    final DateTime selectedDate;

    final String selectedTimeSlot;

    const PaymentConfirmButton({
        Key? key,
        required this.advisor,
        required this.selectedDate,
        required this.selectedTimeSlot,
    }) : super(key: key);
}

```

```

@override

State<PaymentConfirmButton> createState() => _PaymentConfirmButtonState();
}

class _PaymentConfirmButtonState extends State<PaymentConfirmButton> {

  bool _isProcessing = false;

  Future<void> _handlePayment() async {

    setState(() => _isProcessing = true);

    // Simulación de ID de cita generado en Supabase

    final String appointmentId = "app_${DateTime.now().millisecondsSinceEpoch}";

    final double amount = (widget.advisor['hourlyRate'] as num).toDouble();

    final bool paymentSuccess = await PaymentService.makePayment(

      context: context,

      appointmentId: appointmentId,

      amount: amount,

      currency: 'mxn',

    );

    setState(() => _isProcessing = false);

    if (paymentSuccess && mounted) {

      Navigator.pop(context); // Cerrar Modal

```

```

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text(
       ¡Pago recibido! Cita confirmada para el
      ${widget.selectedDate.day}/${widget.selectedDate.month}
      ${widget.selectedTimeSlot}.',
    ),
    backgroundColor: const Color(0xFF059669),
    duration: const Duration(seconds: 4),
  ),
);
}
}

```

@override

```

Widget build(BuildContext context) {
  return SizedBox(
    width: double.infinity,
    height: 50,
    child: ElevatedButton(
      style: ElevatedButton.styleFrom(
        backgroundColor: const Color(0xFF059669),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      ),
      onPressed: _isProcessing ? null : _handlePayment,
      child: _isProcessing
        ? const SizedBox(

```

```

        width: 24,

        height: 24,

        child: CircularProgressIndicator(color: Colors.white, strokeWidth: 2),
      ),
    : Text(
      'Pagar y Confirmar (\${widget.advisor['hourlyRate']} MXN)',
      style: const TextStyle(
        color: Colors.white,
        fontWeight: FontWeight.bold,
        fontSize: 16,
      ),
    ),
  ),
);
}
}

```

Requisito de Configuración Inicial en Flutter: Asegúrate de inicializar el SDK en tu `main.dart` antes de llamar a la pantalla:

Dart

```

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  Stripe.publishableKey = "pk_test_tu_stripe_publishable_key";
  runApp(const MyApp());
}

```

El código de la pantalla de **Detalle del Trámite** con su checklist interactiva de requisitos.

Python

```

import os

```

```

detail_dir = "mobile_procedure_detail_module"

os.makedirs(os.path.join(detail_dir, "lib", "screens"), exist_ok=True)

# lib/screens/procedure_detail_screen.dart

detail_content = """import 'package:flutter/material.dart';

class ProcedureDetailScreen extends StatefulWidget {

  final String title;

  final String category;

  const ProcedureDetailScreen({
    Key? key,

    this.title = 'Inscripción al RFC y Obtención de e.firma',
    this.category = 'SAT - Fiscal',
  }) : super(key: key);

  @override
  State<ProcedureDetailScreen> createState() => _ProcedureDetailScreenState();
}

class _ProcedureDetailScreenState extends State<ProcedureDetailScreen> {

  // Checklist dinámica con persistencia de estado local

  final List<Map<String, dynamic>> _requirements = [

    {

      'id': '1',

      'title': 'CURP o Acta de Nacimiento',

```

```
'description': 'Documento oficial actualizado en formato PDF o impreso.',
'isCompleted': true,
'isMandatory': true,
},
{
'id': '2',
'title': 'Identificación Oficial Vigente (INE/Pasaporte)',
'description': 'Original y copia por ambos lados.',
'isCompleted': true,
'isMandatory': true,
},
{
'id': '3',
'title': 'Comprobante de Domicilio Fiscal',
'description': 'Recibo de luz, agua o predial no mayor a 3 meses de antigüedad.',
'isCompleted': false,
'isMandatory': true,
},
{
'id': '4',
'title': 'Unidad de Memoria USB (Pendrive)',
'description': 'Limpia o formateada para guardar los archivos de la e.firma (.key y .cer).',
'isCompleted': false,
'isMandatory': true,
},
{
```

```

'id': '5',

'title': 'Correo Electrónico Activo',

'description': 'Para vinculación del Buzón Tributario.',

'isCompleted': false,

'isMandatory': false,

},

];

```

@override

```

Widget build(BuildContext context) {

  final completedCount = _requirements.where((r) => r['isCompleted'] == true).length;

  final totalCount = _requirements.length;

  final double progress = totalCount > 0 ? completedCount / totalCount : 0.0;

  return Scaffold(

    backgroundColor: const Color(0xFFFF8FAFC),

    appBar: AppBar(

      backgroundColor: Colors.white,

      elevation: 0,

      leading: IconButton(

        icon: const Icon(Icons.arrow_back_ios_new, color: Color(0xFF0F172A), size: 20),

        onPressed: () => Navigator.pop(context),

      ),

      title: Text(

        widget.category,

        style: const TextStyle(fontSize: 14, color: Color(0xFF64748B), fontWeight:
FontWeight.w500),

```

```

    ),
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(20.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Título del Trámite
          Text(
            widget.title,
            style: const TextStyle(
              fontSize: 22,
              fontWeight: FontWeight.bold,
              color: Color(0xFF0F172A),
            ),
          ),
          ),
          const SizedBox(height: 16),

          // Card de Progreso del Checklist
          _buildProgressCard(completedCount, totalCount, progress),
          const SizedBox(height: 24),

          const Text(
            'Requisitos y Documentación',
            style: TextStyle(
              fontSize: 18,

```



```

        fontWeight: FontWeight.bold,
        color: Color(0xFF0F172A),
    ),
),
const SizedBox(height: 12),

// Lista de items del Checklist
..._requirements.map((req) => _buildRequirementTile(req)).toList(),
],
),
),
);
}

```

```

Widget _buildProgressCard(int completed, int total, double progress) {
  return Container(
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(16),
      border: Border.all(color: const Color(0xFFE2E8F0)),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(

```

```

mainAxisAlignment: MainAxisAlignment.spaceBetween,
children: [
  const Text(
    'Avance de Requisitos',
    style: TextStyle(fontWeight: FontWeight.bold, color: Color(0xFF0F172A)),
  ),
  Text(
    '$completed de $total completos',
    style: const TextStyle(
      fontWeight: FontWeight.bold,
      color: Color(0xFF059669),
    ),
  ),
],
),
const SizedBox(height: 12),
ClipRRect(
  borderRadius: BorderRadius.circular(6),
  child: LinearProgressIndicator(
    value: progress,
    minHeight: 8,
    backgroundColor: const Color(0xFFFF1F5F9),
    valueColor: const AlwaysStoppedAnimation<Color>(Color(0xFF059669)),
  ),
),
],

```

```
),  
);  
}
```

```
Widget _buildRequirementTile(Map<String, dynamic> req) {  
  final bool isCompleted = req['isCompleted'];  
  
  return Container(  
    margin: const EdgeInsets.only(bottom: 12),  
    decoration: BoxDecoration(  
      color: Colors.white,  
      borderRadius: BorderRadius.circular(12),  
      border: Border.all(  
        color: isCompleted ? const Color(0xFF059669).withOpacity(0.3) : const  
Color(0xFFE2E8F0),  
      ),  
    ),  
    child: CheckboxListTile(  
      activeColor: const Color(0xFF059669),  
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),  
      value: isCompleted,  
      onChanged: (bool? newValue) {  
        setState(() {  
          req['isCompleted'] = newValue ?? false;  
        });  
      },  
      title: Row(  

```

```

children: [
  Expanded(
    child: Text(
      req['title'],
      style: TextStyle(
        fontWeight: FontWeight.bold,
        fontSize: 14,
        decoration: isCompleted ? TextDecoration.lineThrough : null,
        color: isCompleted ? const Color(0xFF64748B) : const Color(0xFF0F172A),
      ),
    ),
    ),
    if (req['isMandatory'])
      Container(
        padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
        decoration: BoxDecoration(
          color: const Color(0xFFFFEF2F2),
          borderRadius: BorderRadius.circular(4),
        ),
        child: const Text(
          'Obligatorio',
          style: TextStyle(fontSize: 10, color: Color(0xFFDC2626), fontWeight:
FontWeight.w600),
        ),
      ),
    ],
  ),

```

```

        subtitle: Padding(
          padding: const EdgeInsets.only(top: 4.0),
          child: Text(
            req['description'],
            style: const TextStyle(fontSize: 12, color: Color(0xFF64748B)),
          ),
        ),
      ),
    ),
  );
}
}

```

with `open(os.path.join(detail_dir, "lib", "screens", "procedure_detail_screen.dart"), "w", encoding="utf-8")` as `f`:

```
f.write(detail_content)
```

```
import shutil
```

```
shutil.make_archive("mobile_procedure_detail_module", 'zip', detail_dir)
```

```
print("Procedure detail module created and zipped.")
```

Código de la Pantalla (**lib/screens/procedure_detail_screen.dart**)

```
import 'package:flutter/material.dart';
```

```
class ProcedureDetailScreen extends StatefulWidget {
  final String title;
  final String category;
```

```
  const ProcedureDetailScreen({
```

```

Key? key,
this.title = 'Inscripción al RFC y Obtención de e.firma',
this.category = 'SAT - Fiscal',
}) : super(key: key);

```

```

@override
State<ProcedureDetailScreen> createState() => _ProcedureDetailScreenState();
}

```

```

class _ProcedureDetailScreenState extends State<ProcedureDetailScreen> {
  // Checklist dinámica con persistencia de estado local
  final List<Map<String, dynamic>> _requirements = [
    {
      'id': '1',
      'title': 'CURP o Acta de Nacimiento',
      'description': 'Documento oficial actualizado en formato PDF o impreso.',
      'isCompleted': true,
      'isMandatory': true,
    },
    {
      'id': '2',
      'title': 'Identificación Oficial Vigente (INE/Pasaporte)',
      'description': 'Original y copia por ambos lados.',
      'isCompleted': true,
      'isMandatory': true,
    },
    {
      'id': '3',
      'title': 'Comprobante de Domicilio Fiscal',
      'description': 'Recibo de luz, agua o predial no mayor a 3 meses de antigüedad.',
      'isCompleted': false,
      'isMandatory': true,
    },
    {
      'id': '4',
      'title': 'Unidad de Memoria USB (Pendrive)',
      'description': 'Limpia o formateada para guardar los archivos de la e.firma (.key y .cer).',
      'isCompleted': false,
      'isMandatory': true,
    },
    {
      'id': '5',
      'title': 'Correo Electrónico Activo',
      'description': 'Para vinculación del Buzón Tributario.',
      'isCompleted': false,
      'isMandatory': false,
    },
  ],
};

```

```

@override
Widget build(BuildContext context) {
  final completedCount = _requirements.where((r) => r['isCompleted'] == true).length;
  final totalCount = _requirements.length;
  final double progress = totalCount > 0 ? completedCount / totalCount : 0.0;

  return Scaffold(
    backgroundColor: const Color(0xFFFF8FAFC),
    appBar: AppBar(
      backgroundColor: Colors.white,
      elevation: 0,
      leading: IconButton(
        icon: const Icon(Icons.arrow_back_ios_new, color: Color(0xFF0F172A), size: 20),
        onPressed: () => Navigator.pop(context),
      ),
      title: Text(
        widget.category,
        style: const TextStyle(fontSize: 14, color: Color(0xFF64748B), fontWeight:
FontWeight.w500),
      ),
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(20.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Título del Trámite
          Text(
            widget.title,
            style: const TextStyle(
              fontSize: 22,
              fontWeight: FontWeight.bold,
              color: Color(0xFF0F172A),
            ),
          ),
          const SizedBox(height: 16),

          // Card de Progreso del Checklist
          _buildProgressCard(completedCount, totalCount, progress),
          const SizedBox(height: 24),

          const Text(
            'Requisitos y Documentación',
            style: TextStyle(
              fontSize: 18,
              fontWeight: FontWeight.bold,
              color: Color(0xFF0F172A),

```

```

    ),
  ),
  const SizedBox(height: 12),

  // Lista de items del Checklist
  ..._requirements.map((req) => _buildRequirementTile(req)).toList(),
],
),
),
);
}

```

```

Widget _buildProgressCard(int completed, int total, double progress) {
  return Container(
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(16),
      border: Border.all(color: const Color(0xFFE2E8F0)),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            const Text(
              'Avance de Requisitos',
              style: TextStyle(fontWeight: FontWeight.bold, color: Color(0xFF0F172A)),
            ),
            Text(
              '$completed de $total completos',
              style: const TextStyle(
                fontWeight: FontWeight.bold,
                color: Color(0xFF059669),
              ),
            ),
          ],
        ),
        const SizedBox(height: 12),
        ClipRRect(
          borderRadius: BorderRadius.circular(6),
          child: LinearProgressIndicator(
            value: progress,
            minHeight: 8,
            backgroundColor: const Color(0xFFFF1F5F9),
            valueColor: const AlwaysStoppedAnimation<Color>(Color(0xFF059669)),
          ),
        ),
      ],
    ),
  );
}

```



```

    ),
    ],
  ),
);
}

```

```

Widget _buildRequirementTile(Map<String, dynamic> req) {
  final bool isCompleted = req['isCompleted'];

  return Container(
    margin: const EdgeInsets.only(bottom: 12),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(12),
      border: Border.all(
        color: isCompleted ? const Color(0xFF059669).withOpacity(0.3) : const
Color(0xFFE2E8F0),
      ),
    ),
    child: CheckboxListTile(
      activeColor: const Color(0xFF059669),
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      value: isCompleted,
      onChanged: (bool? newValue) {
        setState(() {
          req['isCompleted'] = newValue ?? false;
        });
      },
      title: Row(
        children: [
          Expanded(
            child: Text(
              req['title'],
              style: TextStyle(
                fontWeight: FontWeight.bold,
                fontSize: 14,
                decoration: isCompleted ? TextDecoration.lineThrough : null,
                color: isCompleted ? const Color(0xFF64748B) : const Color(0xFF0F172A),
              ),
            ),
          ),
          if (req['isMandatory'])
            Container(
              padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
              decoration: BoxDecoration(
                color: const Color(0xFFFEF2F2),
                borderRadius: BorderRadius.circular(4),
              ),

```

```

        child: const Text(
          'Obligatorio',
          style: TextStyle(fontSize: 10, color: Color(0xFFDC2626), fontWeight:
FontWeight.w600),
        ),
      ),
    ],
  ),
  subtitle: Padding(
    padding: const EdgeInsets.only(top: 4.0),
    child: Text(
      req['description'],
      style: const TextStyle(fontSize: 12, color: Color(0xFF64748B)),
    ),
  ),
),
);
}
}

```

Características Principales

- **Checklist Interactiva:** Permite al emprendedor marcar o desmarcar requisitos listados (e.g. CURP, INE, Comprobante de domicilio).
- **Cálculo de Progreso Real:** La barra superior se actualiza dinámicamente según los elementos marcados.
- **Distintivo de Obligatoriedad:** Badge para distinguir los documentos indispensables de los opcionales.

la implementación de la pantalla de **Videollamada en Flutter** diseñada para conectarse con salas virtuales de [Daily.co](https://daily.co).

Python

```
import os
```

```
video_dir = "mobile_video_call_module"
```

```
os.makedirs(os.path.join(video_dir, "lib", "screens"), exist_ok=True)
```

```
# lib/screens/video_call_screen.dart
```

```
video_call_content = """import 'package:flutter/material.dart';
```

/// Pantalla de Videollamada simulada para integración con salas de Daily.co / WebView / WebRTC

```
class VideoCallScreen extends StatefulWidget {  
  
  final String roomUrl;  
  
  final String advisorName;  
  
  const VideoCallScreen({  
  
    Key? key,  
  
    this.roomUrl = 'https://formalizamx.daily.co/asesoria-123',  
  
    this.advisorName = 'Lic. Sofía García',  
  
  }) : super(key: key);  
  
  @override  
  
  State<VideoCallScreen> createState() => _VideoCallScreenState();  
}
```

```
class _VideoCallScreenState extends State<VideoCallScreen> {  
  
  bool _isMuted = false;  
  
  bool _isVideoOff = false;  
  
  bool _isHandRaised = false;  
  
  @override  
  
  Widget build(BuildContext context) {  
  
    return Scaffold(  
  
      backgroundColor: const Color(0xFF0F172A), // Fondo oscuro estilo videoconferencia  
  
      appBar: AppBar(  

```

```

backgroundColor: const Color(0xFF1E293B),

elevation: 0,

leading: IconButton(

  icon: const Icon(Icons.arrow_back_ios_new, color: Colors.white, size: 20),

  onPressed: () => _showEndCallDialog(context),

),

title: Column(

  crossAxisAlignment: CrossAxisAlignment.start,

  children: [

    Text(

      widget.advisorName,

      style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color:
Colors.white),

    ),

    const Row(

      children: [

        Icon(Icons.fiber_manual_record, color: Colors.redAccent, size: 10),

        SizedBox(width: 4),

        Text('En vivo (00:14:20)', style: TextStyle(fontSize: 11, color: Colors.white70)),

      ],

    ),

  ],

),

body: SafeArea(

  child: Column(

    children: [

```

```

// ÁREA DE VIDEO PRINCIPAL (Feed del Asesor)

Expanded(

  child: Stack(

    children: [

      Container(

        width: double.infinity,

        height: double.infinity,

        color: const Color(0xFF1E293B),

        child: Center(

          child: Column(

            mainAxisAlignment: MainAxisAlignment.center,

            children: [

              CircleAvatar(

                radius: 50,

                backgroundColor: const Color(0xFF1E3A8A),

                child: Text(

                  widget.advisorName.substring(0, 2).toUpperCase(),

                  style: const TextStyle(fontSize: 32, color: Colors.white, fontWeight:
FontWeight.bold),

                ),

              ),

              const SizedBox(height: 16),

              Text(

                widget.advisorName,

                style: const TextStyle(color: Colors.white, fontSize: 18, fontWeight:
FontWeight.w600),

              ),

```

```

const SizedBox(height: 8),

const Text(

  'Conectado a Daily.co Video Room',

  style: TextStyle(color: Colors.white54, fontSize: 12),

),

],

),

),

),

),

// MINIATURA DEL USUARIO (Feed Propio)
Positioned(

  right: 16,

  bottom: 16,

  child: Container(

    width: 110,

    height: 150,

    decoration: BoxDecoration(

      color: const Color(0xFF334155),

      borderRadius: BorderRadius.circular(12),

      border: Border.all(color: const Color(0xFF059669), width: 2),

    ),

    child: Stack(

      alignment: Alignment.center,

      children: [

        if (_isVideoOff)

```

```
const Icon(Icons.videocam_off, color: Colors.white54, size: 32)
else
const Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Icon(Icons.person, color: Colors.white, size: 40),
    SizedBox(height: 4),
    Text('Tú', style: TextStyle(color: Colors.white70, fontSize: 12)),
  ],
),
if (_isMuted)
const Positioned(
  top: 8,
  right: 8,
  child: Icon(Icons.mic_off, color: Colors.redAccent, size: 16),
),
],
),
),
),
],
),
),
// BARRA DE CONTROLES INFERIOR
Container(
```

```
padding: const EdgeInsets.symmetric(vertical: 20, horizontal: 24),

color: const Color(0xFF1E293B),

child: Row(

  mainAxisAlignment: MainAxisAlignment.spaceEvenly,

  children: [

    // Micrófono

    _buildControlButton(

      icon: _isMuted ? Icons.mic_off : Icons.mic,

      color: _isMuted ? Colors.redAccent : const Color(0xFF334155),

      onPressed: () => setState(() => _isMuted = !_isMuted),

    ),

    // Cámara

    _buildControlButton(

      icon: _isVideoOff ? Icons.videocam_off : Icons.videocam,

      color: _isVideoOff ? Colors.redAccent : const Color(0xFF334155),

      onPressed: () => setState(() => _isVideoOff = !_isVideoOff),

    ),

    // Levantar mano

    _buildControlButton(

      icon: Icons.back_hand,

      color: _isHandRaised ? const Color(0xFFD97706) : const Color(0xFF334155),

      onPressed: () => setState(() => _isHandRaised = !_isHandRaised),

    ),
```



```

        // Colgar / Salir
        _buildControlButton(
            icon: Icons.call_end,
            color: Colors.red,
            onPressed: () => _showEndCallDialog(context),
        ),
    ],
),
),
],
),
),
);
}

```

```

Widget _buildControlButton({
    required IconData icon,
    required Color color,
    required VoidCallback onPressed,
}) {
    return InkWell(
        onTap: onPressed,
        borderRadius: BorderRadius.circular(30),
        child: Container(
            padding: const EdgeInsets.all(14),
            decoration: BoxDecoration(

```

```

        color: color,

        shape: BoxShape.circle,

    ),

    child: Icon(icon, color: Colors.white, size: 24),

),

);

}

```

```

void _showEndCallDialog(BuildContext context) {

  showDialog(

    context: context,

    builder: (context) => AlertDialog(

      title: const Text('¿Finalizar Asesoría?'),

      content: const Text('¿Estás seguro de que deseas salir de la videollamada con el asesor?'),

      actions: [

        TextButton(

          onPressed: () => Navigator.pop(context),

          child: const Text('Cancelar'),

        ),

        ElevatedButton(

          style: ElevatedButton.styleFrom(backgroundColor: Colors.red),

          onPressed: () {

            Navigator.pop(context); // Cerrar diálogo

            Navigator.pop(context); // Salir de la pantalla

          },

          child: const Text('Salir', style: TextStyle(color: Colors.white)),

```

```

        ),
    ],
),
);

}

}

"""

```

with open(os.path.join(video_dir, "lib", "screens", "video_call_screen.dart"), "w", encoding="utf-8") as f:

```

    f.write(video_call_content)

```

```

import shutil

```

```

shutil.make_archive("mobile_video_call_module", 'zip', video_dir)

```

```

print("Video call module created and zipped.")

```

Código de la Pantalla de Videollamada (lib/screens/video_call_screen.dart)

```

import 'package:flutter/material.dart';

```

```

/// Pantalla de Videollamada simulada para integración con salas de Daily.co / WebView /
WebRTC

```

```

class VideoCallScreen extends StatefulWidget {
  final String roomUrl;
  final String advisorName;

```

```

  const VideoCallScreen({
    Key? key,
    this.roomUrl = 'https://formalizamx.daily.co/asesoria-123',
    this.advisorName = 'Lic. Sofía García',
  }) : super(key: key);

```

```

  @override
  State<VideoCallScreen> createState() => _VideoCallScreenState();
}

```

```

class _VideoCallScreenState extends State<VideoCallScreen> {
  bool _isMuted = false;
  bool _isVideoOff = false;
  bool _isHandRaised = false;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color(0xFF0F172A), // Fondo oscuro estilo videoconferencia
      appBar: AppBar(
        backgroundColor: const Color(0xFF1E293B),
        elevation: 0,
        leading: IconButton(
          icon: const Icon(Icons.arrow_back_ios_new, color: Colors.white, size: 20),
          onPressed: () => _showEndCallDialog(context),
        ),
        title: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              widget.advisorName,
              style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color:
Colors.white),
            ),
            const Row(
              children: [
                Icon(Icons.fiber_manual_record, color: Colors.redAccent, size: 10),
                SizedBox(width: 4),
                Text('En vivo (00:14:20)', style: TextStyle(fontSize: 11, color: Colors.white70)),
              ],
            ),
          ],
        ),
      body: SafeArea(
        child: Column(
          children: [
            // ÁREA DE VIDEO PRINCIPAL (Feed del Asesor)
            Expanded(
              child: Stack(
                children: [
                  Container(
                    width: double.infinity,
                    height: double.infinity,
                    color: const Color(0xFF1E293B),
                    child: Center(
                      child: Column(
                        mainAxisAlignment: MainAxisAlignment.center,

```

```

        children: [
          CircleAvatar(
            radius: 50,
            backgroundColor: const Color(0xFF1E3A8A),
            child: Text(
              widget.advisorName.substring(0, 2).toUpperCase(),
              style: const TextStyle(fontSize: 32, color: Colors.white, fontWeight:
FontWeight.bold),
            ),
          ),
          const SizedBox(height: 16),
          Text(
            widget.advisorName,
            style: const TextStyle(color: Colors.white, fontSize: 18, fontWeight:
FontWeight.w600),
          ),
          const SizedBox(height: 8),
          const Text(
            'Conectado a Daily.co Video Room',
            style: TextStyle(color: Colors.white54, fontSize: 12),
          ),
        ],
      ),
    ),
  ),
),

```

// MINIATURA DEL USUARIO (Feed Propio)

```

Positioned(
  right: 16,
  bottom: 16,
  child: Container(
    width: 110,
    height: 150,
    decoration: BoxDecoration(
      color: const Color(0xFF334155),
      borderRadius: BorderRadius.circular(12),
      border: Border.all(color: const Color(0xFF059669), width: 2),
    ),
    child: Stack(
      alignment: Alignment.center,
      children: [
        if (_isVideoOff)
          const Icon(Icons.videocam_off, color: Colors.white54, size: 32)
        else
          const Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Icon(Icons.person, color: Colors.white, size: 40),

```

```

        SizedBox(height: 4),
        Text('Tú', style: TextStyle(color: Colors.white70, fontSize: 12)),
      ],
    ),
    if (!_isMuted)
      const Positioned(
        top: 8,
        right: 8,
        child: Icon(Icons.mic_off, color: Colors.redAccent, size: 16),
      ),
  ],
),
),
),
],
),
),

```

// BARRA DE CONTROLES INFERIOR

```

Container(
  padding: const EdgeInsets.symmetric(vertical: 20, horizontal: 24),
  color: const Color(0xFF1E293B),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    children: [
      // Micrófono
      _buildControlButton(
        icon: _isMuted ? Icons.mic_off : Icons.mic,
        color: _isMuted ? Colors.redAccent : const Color(0xFF334155),
        onPressed: () => setState(() => _isMuted = !_isMuted),
      ),

      // Cámara
      _buildControlButton(
        icon: _isVideoOff ? Icons.videocam_off : Icons.videocam,
        color: _isVideoOff ? Colors.redAccent : const Color(0xFF334155),
        onPressed: () => setState(() => _isVideoOff = !_isVideoOff),
      ),

      // Levantar mano
      _buildControlButton(
        icon: Icons.back_hand,
        color: _isHandRaised ? const Color(0xFFD97706) : const Color(0xFF334155),
        onPressed: () => setState(() => _isHandRaised = !_isHandRaised),
      ),

      // Colgar / Salir
      _buildControlButton(

```

```

        icon: Icons.call_end,
        color: Colors.red,
        onPressed: () => _showEndCallDialog(context),
      ),
    ],
  ),
),
],
),
),
);
}

```

```

Widget _buildControlButton({
  required IconData icon,
  required Color color,
  required VoidCallback onPressed,
}) {
  return InkWell(
    onTap: onPressed,
    borderRadius: BorderRadius.circular(30),
    child: Container(
      padding: const EdgeInsets.all(14),
      decoration: BoxDecoration(
        color: color,
        shape: BoxShape.circle,
      ),
      child: Icon(icon, color: Colors.white, size: 24),
    ),
  );
}

```

```

void _showEndCallDialog(BuildContext context) {
  showDialog(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('¿Finalizar Asesoría?'),
      content: const Text('¿Estás seguro de que deseas salir de la videollamada con el asesor?'),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context),
          child: const Text('Cancelar'),
        ),
        ElevatedButton(
          style: ElevatedButton.styleFrom(backgroundColor: Colors.red),
          onPressed: () {
            Navigator.pop(context); // Cerrar diálogo
          }
        )
      ],
    ),
  );
}

```

```

        Navigator.pop(context); // Salir de la pantalla
      },
      child: const Text('Salir', style: TextStyle(color: Colors.white)),
    ),
  ],
),
);
}
}

```

Características de la Pantalla

- **Feed Principal y Miniatura PIP:** Visualización en pantalla completa del asesor y una ventana en la esquina inferior derecha para la cámara del usuario.
- **Barra de Controles Multimedia:** Botones dedicados para silenciar/activar el micrófono, apagar/encender la cámara, levantar la mano y finalizar la llamada.
- **Diálogo de Confirmación de Salida:** Muestra una alerta antes de cortar la comunicación con el asesor para prevenir salidas accidentales.

Con la integración de pagos, el detalle de trámites y la sala de videollamadas listos, los siguientes pasos recomendados para completar la aplicación son:

- **Gestión de Plantillas y Contratos:** Implementar la pantalla de catálogo para la descarga e interacción con documentos legales predeterminados.
- **Notificaciones Push y Recordatorios:** Configurar alertas (vía Firebase Cloud Messaging) para avisar al usuario sobre citas agendadas o trámites pendientes.
- **Integración del Backend Principal:** Conectar los flujos del frontend con las tablas de Supabase o Firebase (autenticación de usuarios, persistencia del checklist y generación real de URLs en Daily.co).
- **Pruebas de Flujo Completo (E2E):** Probar el recorrido desde la selección de un asesor en el Marketplace hasta la ejecución del pago con Stripe y el ingreso a la videollamada.

Aquí tienes la implementación de la pantalla de **Catálogo de Plantillas Legales y Contratos** en Flutter.

Python

import os

templates_dir = "mobile_templates_module"

os.makedirs(os.path.join(templates_dir, "lib", "screens"), exist_ok=True)

lib/screens/templates_catalog_screen.dart


```
templates_content = ""import 'package:flutter/material.dart';
```

```
class TemplatesCatalogScreen extends StatefulWidget {
```

```
  const TemplatesCatalogScreen({Key? key}) : super(key: key);
```

```
  @override
```

```
  State<TemplatesCatalogScreen> createState() => _TemplatesCatalogScreenState();
```

```
}
```

```
class _TemplatesCatalogScreenState extends State<TemplatesCatalogScreen> {
```

```
  String _selectedCategory = 'Todos';
```

```
  final List<String> _categories = [
```

```
    'Todos',
```

```
    'Mercantil',
```

```
    'Laboral',
```

```
    'Corporativo',
```

```
    'Propiedad Intelectual',
```

```
  ];
```

```
  final List<Map<String, dynamic>> _templates = [
```

```
    {
```

```
      'id': '1',
```

```
      'title': 'Contrato de Prestación de Servicios',
```

```
      'category': 'Mercantil',
```

```
      'format': 'DOCX / PDF',
```

```
'downloads': 1240,

'description': 'Ideal para freelancers y agencias. Define alcances, entregables y esquema de pagos.',

'isPremium': false,

},

{

'id': '2',

'title': 'Acuerdo de Confidencialidad (NDA)',

'category': 'Corporativo',

'format': 'DOCX / PDF',

'downloads': 2100,

'description': 'Protege la información sensible de tu startup o proyecto antes de compartirla con terceros.',

'isPremium': false,

},

{

'id': '3',

'title': 'Contrato Individual de Trabajo (Tiempo Indeterminado)',

'category': 'Laboral',

'format': 'DOCX / PDF',

'downloads': 980,

'description': 'Alineado a la LFT en México. Incluye cláusulas de periodo de prueba y prestaciones.',

'isPremium': true,

},

{

'id': '4',

'title': 'Estatutos Sociales para SAS (Sociedad por Acciones Simplificada)',
```

```

        'category': 'Corporativo',
        'format': 'PDF',
        'downloads': 1560,
        'description': 'Guía y plantilla estructurada para constituir tu empresa sin costo notarial.',
        'isPremium': true,
    },
    {
        'id': '5',
        'title': 'Contrato de Cesión de Derechos de Autor',
        'category': 'Propiedad Intelectual',
        'format': 'DOCX',
        'downloads': 640,
        'description': 'Asegura la propiedad de logos, código de software o diseños creados por externos.',
        'isPremium': false,
    },
];

```

@override

Widget build(BuildContext context) {

final filteredTemplates = _selectedCategory == 'Todos'

? _templates

: _templates.where((t) => t['category'] == _selectedCategory).toList();

return Scaffold(

backgroundColor: const Color(0xFFFF8FAFC),

appBar: AppBar(

```

backgroundColor: Colors.white,

elevation: 0,

leading: IconButton(

  icon: const Icon(Icons.arrow_back_ios_new, color: Color(0xFF0F172A), size: 20),

  onPressed: () => Navigator.pop(context),

),

title: const Text(

  'Plantillas y Contratos',

  style: TextStyle(color: Color(0xFF0F172A), fontWeight: FontWeight.bold, fontSize: 18),

),

),

body: Column(

  children: [

    // BARRA DE BÚSQUEDA Y FILTROS POR CATEGORÍA

    Container(

      color: Colors.white,

      padding: const EdgeInsets.symmetric(vertical: 12),

      child: SingleChildScrollView(

        scrollDirection: Axis.horizontal,

        padding: const EdgeInsets.symmetric(horizontal: 16),

        child: Row(

          children: _categories.map((cat) {

            final isSelected = cat == _selectedCategory;

            return Padding(

              padding: const EdgeInsets.only(right: 8.0),

              child: ChoiceChip(

```

```

        label: Text(cat),

        selected: isSelected,

        selectedColor: const Color(0xFF059669),

        backgroundColor: const Color(0xFFF1F5F9),

        labelStyle: TextStyle(

            color: isSelected ? Colors.white : const Color(0xFF64748B),

            fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,

        ),

        onSelected: (selected) {

            if (selected) {

                setState(() => _selectedCategory = cat);

            }

        },

    ),

);

}).toList(),

),

),

),

```

// LISTADO DE PLANTILLAS

```

Expanded(

    child: ListView.builder(

        padding: const EdgeInsets.all(16),

        itemCount: filteredTemplates.length,

        itemBuilder: (context, index) {

```

```

        final item = filteredTemplates[index];

        return _buildTemplateCard(item);

    },

),

),

],

),

);
}

```

```

Widget _buildTemplateCard(Map<String, dynamic> item) {
  return Container(
    margin: const EdgeInsets.only(bottom: 12),
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(12),
      border: Border.all(color: const Color(0xFFE2E8F0)),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Container(

```

```

padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),

decoration: BoxDecoration(

  color: const Color(0xFFECCDF5),

  borderRadius: BorderRadius.circular(6),

),

child: Text(

  item['category'],

  style: const TextStyle(fontSize: 11, color: Color(0xFF059669), fontWeight:
FontWeight.bold),

),

),

if (item['isPremium'])

  Container(

    padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),

    decoration: BoxDecoration(

      color: const Color(0xFFFEF3C7),

      borderRadius: BorderRadius.circular(6),

    ),

    child: const Row(

      children: [

        Icon(Icons.star, size: 12, color: Color(0xFFD97706)),

        SizedBox(width: 4),

        Text(

          'PRO',

          style: TextStyle(fontSize: 11, color: Color(0xFFD97706), fontWeight:
FontWeight.bold),

        ),

```

```

        ],
      ),
    ),
  ],
),
const SizedBox(height: 10),
Text(
  item['title'],
  style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color:
Color(0xFF0F172A)),
),
const SizedBox(height: 6),
Text(
  item['description'],
  style: const TextStyle(fontSize: 13, color: Color(0xFF64748B)),
),
const SizedBox(height: 14),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Text(
      'Formato: ${item['format']}',
      style: const TextStyle(fontSize: 12, color: Color(0xFF94A3B8), fontWeight:
FontWeight.w500),
    ),
    ElevatedButton.icon(
      style: ElevatedButton.styleFrom(

```



```

        backgroundColor: const Color(0xFF059669),

        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),

        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),

    ),

    onPressed: () {

        ScaffoldMessenger.of(context).showSnackBar(

            SnackBar(

                content: Text('Descargando ${item['title']}...'),

                backgroundColor: const Color(0xFF059669),

            ),

        );

    },

    icon: const Icon(Icons.download, size: 16, color: Colors.white),

    label: const Text('Descargar', style: TextStyle(color: Colors.white, fontSize: 13)),

),

],

),

],

),

);

}

}

"""

```

with open(os.path.join(templates_dir, "lib", "screens", "templates_catalog_screen.dart"), "w", encoding="utf-8") as f:

```
f.write(templates_content)
```

```
import shutil

shutil.make_archive("mobile_templates_module", 'zip', templates_dir)

print("Templates catalog module created and zipped.")
```

Código de la Pantalla **(lib/screens/templates_catalog_screen.dart)**

```
import 'package:flutter/material.dart';

class TemplatesCatalogScreen extends StatefulWidget {
  const TemplatesCatalogScreen({Key? key}) : super(key: key);

  @override
  State<TemplatesCatalogScreen> createState() => _TemplatesCatalogScreenState();
}

class _TemplatesCatalogScreenState extends State<TemplatesCatalogScreen> {
  String _selectedCategory = 'Todos';

  final List<String> _categories = [
    'Todos',
    'Mercantil',
    'Laboral',
    'Corporativo',
    'Propiedad Intelectual',
  ];

  final List<Map<String, dynamic>> _templates = [
    {
      'id': '1',
      'title': 'Contrato de Prestación de Servicios',
      'category': 'Mercantil',
      'format': 'DOCX / PDF',
      'downloads': 1240,
      'description': 'Ideal para freelancers y agencias. Define alcances, entregables y esquema de pagos.',
      'isPremium': false,
    },
    {
      'id': '2',
      'title': 'Acuerdo de Confidencialidad (NDA)',
      'category': 'Corporativo',
      'format': 'DOCX / PDF',
```

```

      'downloads': 2100,
      'description': 'Protege la información sensible de tu startup o proyecto antes de
compartirla con terceros.',
      'isPremium': false,
    },
    {
      'id': '3',
      'title': 'Contrato Individual de Trabajo (Tiempo Indeterminado)',
      'category': 'Laboral',
      'format': 'DOCX / PDF',
      'downloads': 980,
      'description': 'Alineado a la LFT en México. Incluye cláusulas de periodo de prueba y
prestaciones.',
      'isPremium': true,
    },
    {
      'id': '4',
      'title': 'Estatutos Sociales para SAS (Sociedad por Acciones Simplificada)',
      'category': 'Corporativo',
      'format': 'PDF',
      'downloads': 1560,
      'description': 'Guía y plantilla estructurada para constituir tu empresa sin costo notarial.',
      'isPremium': true,
    },
    {
      'id': '5',
      'title': 'Contrato de Cesión de Derechos de Autor',
      'category': 'Propiedad Intelectual',
      'format': 'DOCX',
      'downloads': 640,
      'description': 'Asegura la propiedad de logos, código de software o diseños creados por
externos.',
      'isPremium': false,
    },
  ];

```

@override

```

Widget build(BuildContext context) {
  final filteredTemplates = _selectedCategory == 'Todos'
    ? _templates
    : _templates.where((t) => t['category'] == _selectedCategory).toList();

```

```

  return Scaffold(
    backgroundColor: const Color(0xFFFF8FAFC),
    appBar: AppBar(
      backgroundColor: Colors.white,
      elevation: 0,
      leading: IconButton(

```

```

        icon: const Icon(Icons.arrow_back_ios_new, color: Color(0xFF0F172A), size: 20),
        onPressed: () => Navigator.pop(context),
      ),
      title: const Text(
        'Plantillas y Contratos',
        style: TextStyle(color: Color(0xFF0F172A), fontWeight: FontWeight.bold, fontSize: 18),
      ),
    ),
    body: Column(
      children: [
        // BARRA DE BÚSQUEDA Y FILTROS POR CATEGORÍA
        Container(
          color: Colors.white,
          padding: const EdgeInsets.symmetric(vertical: 12),
          child: SingleChildScrollView(
            scrollDirection: Axis.horizontal,
            padding: const EdgeInsets.symmetric(horizontal: 16),
            child: Row(
              children: _categories.map((cat) {
                final isSelected = cat == _selectedCategory;
                return Padding(
                  padding: const EdgeInsets.only(right: 8.0),
                  child: ChoiceChip(
                    label: Text(cat),
                    selected: isSelected,
                    selectedColor: const Color(0xFF059669),
                    backgroundColor: const Color(0xFFFF1F5F9),
                    labelStyle: TextStyle(
                      color: isSelected ? Colors.white : const Color(0xFF64748B),
                      fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
                    ),
                  ),
                  onSelect: (selected) {
                    if (selected) {
                      setState(() => _selectedCategory = cat);
                    }
                  },
                ),
              );
            }).toList(),
          ),
        ),
      ],
    ),
  ),
  // LISTADO DE PLANTILLAS
  Expanded(
    child: ListView.builder(
      padding: const EdgeInsets.all(16),
      itemCount: filteredTemplates.length,

```

```

        itemBuilder: (context, index) {
          final item = filteredTemplates[index];
          return _buildTemplateCard(item);
        },
      ),
    ],
  ),
);
}

```

```

Widget _buildTemplateCard(Map<String, dynamic> item) {
  return Container(
    margin: const EdgeInsets.only(bottom: 12),
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: Colors.white,
      borderRadius: BorderRadius.circular(12),
      border: Border.all(color: const Color(0xFFE2E8F0)),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Container(
              padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
              decoration: BoxDecoration(
                color: const Color(0xFFECFDF5),
                borderRadius: BorderRadius.circular(6),
              ),
              child: Text(
                item['category'],
                style: const TextStyle(fontSize: 11, color: Color(0xFF059669), fontWeight:
FontWeight.bold),
              ),
            ),
            if (item['isPremium'])
              Container(
                padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
                decoration: BoxDecoration(
                  color: const Color(0xFFFFEF3C7),
                  borderRadius: BorderRadius.circular(6),
                ),
                child: const Row(
                  children: [
                    Icon(Icons.star, size: 12, color: Color(0xFFD97706)),

```

```

        SizedBox(width: 4),
        Text(
          'PRO',
          style: TextStyle(fontSize: 11, color: Color(0xFFD97706), fontWeight:
FontWeight.bold),
        ),
      ],
    ),
  ],
),
const SizedBox(height: 10),
Text(
  item['title'],
  style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold, color:
Color(0xFF0F172A)),
),
const SizedBox(height: 6),
Text(
  item['description'],
  style: const TextStyle(fontSize: 13, color: Color(0xFF64748B)),
),
const SizedBox(height: 14),
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween,
  children: [
    Text(
      'Formato: ${item['format']}',
      style: const TextStyle(fontSize: 12, color: Color(0xFF94A3B8), fontWeight:
FontWeight.w500),
    ),
    ElevatedButton.icon(
      style: ElevatedButton.styleFrom(
        backgroundColor: const Color(0xFF059669),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
      ),
      onPressed: () {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text('Descargando ${item['title']}...'),
            backgroundColor: const Color(0xFF059669),
          ),
        );
      },
      icon: const Icon(Icons.download, size: 16, color: Colors.white),
      label: const Text('Descargar', style: TextStyle(color: Colors.white, fontSize: 13)),
    ),
  ],
),

```

```

    ],
  ),
],
),
);
}
}

```

Características Principales

- **Filtros por Categoría:** Chips interactivos para filtrar entre contratos mercantiles, laborales, corporativos o de propiedad intelectual.
- **Insignias "PRO" / Gratuitas:** Distintivo para identificar los documentos que requieren suscripción o los de libre acceso.
- **Acciones de Descarga:** Botones listos para activar la descarga directa del formato especificado (DOCX, PDF).

El siguiente paso para completar la infraestructura del proyecto es configurar las **Notificaciones Push y Recordatorios** (usando `firebase_messaging` y `flutter_local_notifications`).

Esto enviará alertas automáticas al usuario sobre:

1. Confirmación de citas pagadas.
2. Recordatorio 15 minutos antes de su videollamada con el asesor.
3. Actualización en el estatus de sus trámites.

Servicio de Notificaciones (`lib/services/notification_service.dart`)

Dart

```

import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter_local_notifications/flutter_local_notifications.dart';

```

```

class NotificationService {
  static final FirebaseMessaging _firebaseMessaging = FirebaseMessaging.instance;
  static final FlutterLocalNotificationsPlugin _localNotificationsPlugin =
    FlutterLocalNotificationsPlugin();

```

```

  /// Inicializa los permisos y canal de notificaciones

```

```

  static Future<void> initialize() async {
    // 1. Solicitar permisos al usuario (iOS / Android 13+)
    NotificationSettings settings = await _firebaseMessaging.requestPermission(
      alert: true,
      badge: true,
      sound: true,
    );
  }

```

```

if (settings.authorizationStatus == AuthorizationStatus.authorized) {
  // 2. Configurar notificaciones locales
  const AndroidInitializationSettings androidSettings =
    AndroidInitializationSettings('@mipmap/ic_launcher');

  const DarwinInitializationSettings iosSettings =
    DarwinInitializationSettings();

  const InitializationSettings initSettings = InitializationSettings(
    android: androidSettings,
    iOS: iosSettings,
  );

  await _localNotificationsPlugin.initialize(initSettings);

  // 3. Listener para notificaciones cuando la App está en primer plano (Foreground)
  FirebaseMessaging.onMessage.listen((RemoteMessage message) {
    _showLocalNotification(message);
  });
}

/// Muestra una notificación emergente local
static Future<void> _showLocalNotification(RemoteMessage message) async {
  const AndroidNotificationDetails androidDetails = AndroidNotificationDetails(
    'formaliza_channel',
    'Notificaciones de Citas y Trámites',
    channelDescription: 'Alertas para recordatorios de asesorías legales y fiscales',
    importance: Importance.max,
    priority: Priority.high,
  );

  const NotificationDetails notificationDetails = NotificationDetails(
    android: androidDetails,
    iOS: DarwinNotificationDetails(),
  );

  await _localNotificationsPlugin.show(
    message.hashCode,
    message.notification?.title ?? 'Consultoria para todos',
    message.notification?.body ?? '',
    notificationDetails,
  );
}

/// Obtiene el Token del dispositivo para registrarlo en el Backend
static Future<String?> getDeviceToken() async {

```



```
    return await _firebaseMessaging.getToken();  
  }  
}
```

Integración en `main.dart`

Agrega la inicialización en la función principal de tu aplicación:

```
Dart  
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  
  // Inicializar Firebase  
  await Firebase.initializeApp();  
  
  // Inicializar Servicio de Notificaciones  
  await NotificationService.initialize();  
  
  runApp(const MyApp());  
}
```

Integración del Backend (Supabase o Firebase).

Construir la base de datos y la API en este punto es la mejor decisión por tres razones principales:

1. **Da vida a los módulos creados:** Tu checklist de trámites, la sala de Daily.co y el catálogo de plantillas pasarán de usar datos de prueba (*mock data*) a guardar información real en la nube.
2. **Habilita el envío de notificaciones:** Para probar las notificaciones push que acabamos de definir, necesitas un backend que guarde los tokens FCM de los usuarios y ejecute los disparadores (*triggers*) de recordatorios.
3. **Prepara el camino para las pruebas E2E:** No es posible hacer una prueba de flujo completo (Marketplace → Pago → Notificación → Videollamada) sin una base de datos que confirme el estado de las transacciones y las citas.

Esquema propuesto para el Backend

Si estás de acuerdo, podemos definir la estructura de la base de datos con las siguientes tablas clave:

- **users:** Perfiles de emprendedores y asesores.
- **appointments:** Registro de citas agendadas, estados de pago (Stripe ID) y enlace a la sala de Daily.co.
- **user_checklist_progress:** Estado de avance en los requisitos de cada trámite.
- **templates:** Enlaces de descarga y metadatos de los contratos.

la arquitectura del backend recomendada y el diseño completo de la base de datos.

1. Recomendación: Supabase vs. Firebase

Para una aplicación de trámites legales, asesorías y pagos como **Consultoría para todos**, la mejor opción es **Supabase (PostgreSQL)** por las siguientes razones:

- **Relaciones complejas sólidas:** Manejar usuarios, asesores, citas pagadas, salas de video y listas de verificación (*checklists*) requiere una estructura relacional estricta para evitar incoherencias en las citas o estados de pago.
- **Seguridad granular (Row Level Security - RLS):** Permite restringir directamente en la base de datos que solo el cliente o el asesor asignado tengan acceso a sus contratos, documentos y notas de la videollamada.
- **Integración fácil con Flutter y Firebase Push:** Puedes usar Supabase como la base de datos principal y autenticación, combinándolo con **Firestore** y **Firebase Cloud Messaging (FCM)** para las notificaciones push.

2. Esquema de Base de Datos en SQL (Supabase / PostgreSQL)

Ejecuta el siguiente script en el **SQL Editor** de Supabase para crear las tablas base, relaciones y restricciones:

SQL

-- 1. TIPOS DE DATOS Y ENUMS

CREATE TYPE user_role AS ENUM ('client', 'advisor', 'admin');

CREATE TYPE appointment_status AS ENUM ('pending', 'confirmed', 'completed', 'cancelled');

CREATE TYPE payment_status AS ENUM ('unpaid', 'paid', 'refunded');

-- 2. TABLA DE PERFILES DE USUARIO

CREATE TABLE public.profiles (

id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,

full_name TEXT NOT NULL,

email TEXT UNIQUE NOT NULL,

phone TEXT,

role user_role DEFAULT 'client'::user_role,

fcm_token TEXT, -- Token para notificaciones push

avatar_url TEXT,

created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL

);

-- 3. DETALLES DE ASESORES (Extensión de profiles para asesores)

CREATE TABLE public.advisor_profiles (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

    user_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE UNIQUE NOT
    NULL,
    specialty TEXT NOT NULL, -- e.g. "Fiscal / SAT", "Corporativo"
    bio TEXT,
    hourly_rate NUMERIC(10, 2) NOT NULL,
    rating NUMERIC(2, 1) DEFAULT 5.0,
    is_available BOOLEAN DEFAULT true
);

```

-- 4. TABLA DE CITAS / VIDEO LLAMADAS (Daily.co + Stripe)

```

CREATE TABLE public.appointments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    client_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
    advisor_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
    scheduled_at TIMESTAMP WITH TIME ZONE NOT NULL,
    duration_minutes INT DEFAULT 30 NOT NULL,
    status appointment_status DEFAULT 'pending'::appointment_status,

```

-- Integraciones de Pago y Video

```

stripe_payment_intent_id TEXT,
payment_status payment_status DEFAULT 'unpaid'::payment_status,
daily_room_url TEXT, -- URL de la sala en Daily.co

```

```

    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT
    NULL
);

```

-- 5. SEGUIMIENTO DE CHECKLIST DE TRÁMITES

```

CREATE TABLE public.user_checklist_progress (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
    procedure_id TEXT NOT NULL, -- e.g. "alta_sat", "constitucion_sas"
    step_id TEXT NOT NULL, -- e.g. "e_firma", "rfc"
    is_completed BOOLEAN DEFAULT false NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT
    NULL,
    UNIQUE(user_id, procedure_id, step_id)
);

```

-- 6. CATÁLOGO DE PLANTILLAS Y CONTRATOS

```

CREATE TABLE public.templates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title TEXT NOT NULL,
    category TEXT NOT NULL,
    description TEXT,
    file_format TEXT NOT NULL, -- e.g. "DOCX / PDF"
    file_url TEXT NOT NULL, -- Enlace seguro en Supabase Storage
    is_premium BOOLEAN DEFAULT false NOT NULL,

```

```

downloads_count INT DEFAULT 0,
created_at TIMESTAMPTZ WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT
NULL
);

```

3. Servicio de Integración en Flutter (**lib/services/supabase_service.dart**)

Instala las dependencias agregando `supabase_flutter: ^2.0.0` en tu `pubspec.yaml` y crea la capa de conexión en Flutter:

Dart

```
import 'package:supabase_flutter/supabase_flutter.dart';
```

```

class SupabaseService {
  static final client = Supabase.instance.client;

  /// Obtiene la lista de citas agendadas del usuario logueado
  static Future<List<Map<String, dynamic>>> getUserAppointments() async {
    final userId = client.auth.currentUser?.id;
    if (userId == null) return [];

    final response = await client
      .from('appointments')
      .select("""
        id,
        scheduled_at,
        status,
        payment_status,
        daily_room_url,
        advisor:advisor_id (
          full_name,
          avatar_url
        )
      """)
      .eq('client_id', userId)
      .order('scheduled_at', ascending: true);

    return List<Map<String, dynamic>>.from(response);
  }

  /// Guarda o actualiza el estado de un paso del Checklist de trámites
  static Future<void> updateChecklistStep({
    required String procedureId,
    required String stepId,
    required bool isCompleted,

```

```

}) async {
  final userId = client.auth.currentUser?.id;
  if (userId == null) return;

  await client.from('user_checklist_progress').upsert({
    'user_id': userId,
    'procedure_id': procedureId,
    'step_id': stepId,
    'is_completed': isCompleted,
    'updated_at': DateTime.now().toIso8601String(),
  });
}

/// Guarda el FCM Token del usuario para envío de notificaciones push
static Future<void> saveFcmToken(String token) async {
  final userId = client.auth.currentUser?.id;
  if (userId == null) return;

  await client.from('profiles').update({'fcm_token': token}).eq('id', userId);
}
}

```

Con el backend estructurado, podemos proceder con:

1. **Configurar la Edge Function / Webhook de Stripe & Daily.co:** Para que cuando se confirme un pago, se genere automáticamente la sala en Daily.co y se actualice el estado de la cita a **paid**.
2. **Realizar las Pruebas de Flujo Completo (E2E):** Simular el recorrido completo desde la reserva en la app hasta el ingreso a la videollamada.

Pruebas de Flujo Completo (E2E) de la aplicación, utilizaremos el paquete oficial de Flutter: **integration_test**. Este enfoque permite simular de forma automatizada las acciones reales de un usuario en el dispositivo.

El flujo a probar comprende el siguiente recorrido:

1. **Selección de Asesor** en el Marketplace.
2. **Proceso de Pago** con tarjeta de crédito (Stripe).
3. **Verificación del Backend** (creación de la cita y generación de la sala en Daily.co).
4. **Navegación e Ingreso** a la pantalla de Videollamada.

1. Configuración del Proyecto

Agrega la dependencia **integration_test** a tu archivo **pubspec.yaml**:

YAML
dev_dependencies:

```
flutter_test:
  sdk: flutter
integration_test:
  sdk: flutter
```

Estructura de la carpeta de pruebas en la raíz del proyecto:

```
Plaintext
formaliza_app/
|— integration_test/
|   └─ e2e_booking_flow_test.dart
```

2. Script de Prueba E2E (**integration_test/e2e_booking_flow_test.dart**)

Dart

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';
import 'package:formaliza_app/main.dart' as app;
```

```
void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();
```

```
  group('Flujo E2E: Reserva, Pago y Videollamada', () {
    testWidgets('Recorrido completo desde la selección hasta la llamada', (WidgetTester
tester) async {
```

```
      // 1. Inicializar la aplicación
      app.main();
      await tester.pumpAndSettle();
```

```
      // 2. Navegar al Marketplace de Asesores y seleccionar uno
      final advisorCardFinder = find.byKey(const Key('advisor_card_1'));
      expect(advisorCardFinder, findsOneWidget);
      await tester.tap(advisorCardFinder);
      await tester.pumpAndSettle();
```

```
      // 3. Confirmar fecha y horario de cita
      final scheduleButtonFinder = find.byKey(const Key('schedule_appointment_btn'));
      expect(scheduleButtonFinder, findsOneWidget);
      await tester.tap(scheduleButtonFinder);
      await tester.pumpAndSettle();
```

```
      // 4. Pantalla de Pago: Ingresar datos de prueba
      final payButtonFinder = find.byKey(const Key('pay_with_stripe_btn'));
      expect(payButtonFinder, findsOneWidget);
```

```

await tester.tap(payButtonFinder);

// Esperar la respuesta simulada/real del backend y Stripe
await tester.pumpAndSettle(const Duration(seconds: 3));

// 5. Verificación de redirección a la confirmación con el link de Daily.co
expect(find.text('¡Cita Confirmada!'), findsOneWidget);
final joinCallButton = find.byKey(const Key('join_call_btn'));
expect(joinCallButton, findsOneWidget);

// 6. Ingresar a la Videollamada
await tester.tap(joinCallButton);
await tester.pumpAndSettle();

// 7. Validar presencia de controles en la sala de videollamada
expect(find.byIcon(Icons.mic), findsOneWidget);
expect(find.byIcon(Icons.videocam), findsOneWidget);
expect(find.byIcon(Icons.call_end), findsOneWidget);

// 8. Colgar la llamada y volver al inicio
await tester.tap(find.byIcon(Icons.call_end));
await tester.pumpAndSettle();

final confirmExitButton = find.text('Salir');
expect(confirmExitButton, findsOneWidget);
await tester.tap(confirmExitButton);
await tester.pumpAndSettle();
});
});
}

```

3. Ejecución de las Pruebas

En Emulador Android / Dispositivo Físico:

```

Bash
flutter test integration_test/e2e_booking_flow_test.dart

```

En iOS Simulator:

```

Bash
flutter test integration_test/e2e_booking_flow_test.dart -d iPhone

```

Checklist de Verificación para Producción

Entorno Sandbox / Test

- [] Las llaves públicas y privadas de Stripe están en modo `pk_test_...` y `sk_test_...`.
- [] Se utiliza la API Key de pruebas de Daily.co para la generación automática de salas temporales.
- [] La base de datos de Supabase/Firebase apunta al entorno de desarrollo (Staging).

Casos Límite a Probar

- [] **Fallo de pago:** Probar tarjetas rechazadas y verificar que el usuario permanezca en la pantalla de pago con el mensaje de error adecuado.
- [] **Permisos de Cámara y Micrófono:** Asegurar que la app solicite los permisos del sistema operativo antes de ingresar a la videollamada.
- [] **Desconexión a Red:** Probar el comportamiento de la llamada si se interrumpe el WiFi o datos móviles.

Una vez completadas las pruebas E2E y la infraestructura de backend, los pasos finales recomendados para dejar la aplicación lista para su despliegue comercial son:

- **Publicación y Despliegue en Tiendas (App Store y Google Play):** Configurar certificados, firmas digitalizadas, capturas de pantalla promocionales y las fichas de las tiendas.
- **Monitoreo de Errores y Telemetría:** Integrar Sentry o Firebase Crashlytics para recibir reportes automáticos de errores en tiempo real una vez que la app esté en manos de usuarios reales.
- **Optimización de Rendimiento y Assets:** Reducir el tamaño final de los binarios (APK/AAB e IPA), optimizar imágenes y revisar el uso de memoria en las videollamadas.
- **Estrategia de Lanzamiento / Marketing:** Definir la campaña de adquisición inicial de emprendedores y asesores.

Configurar la preparación, firma y generación de compilados listos para subir a **Google Play Console** y **Apple App Store Connect**.

1. Preparación y Firma en Android (AAB)

Paso 1: Generar la Keystore de Firma

Ejecuta la consola en la raíz de tu proyecto para crear la clave de producción:

Bash

```
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA -keysize 2048 -validity 10000 -alias upload
```

Paso 2: Configurar `android/key.properties`

Crea el archivo `android/key.properties` (y agrégalo a tu `.gitignore`):

Properties


```
storePassword=<TU_PASSWORD_KEYSTORE>
keyPassword=<TU_PASSWORD_CLAVE>
keyAlias=upload
storeFile=~/.upload-keystore.jks
```

Paso 3: Actualizar `android/app/build.gradle`

Configura la firma de producción y el identificador único:

Groovy

```
def localProperties = new Properties()
def localPropertiesFile = rootProject.file('local.properties')
if (localPropertiesFile.exists()) {
    localPropertiesFile.withRootProjectReader('UTF-8').withReader { reader ->
        localProperties.load(reader)
    }
}
```

```
def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}
```

```
android {
    namespace "com.formalizamx.app"
    compileSdkVersion 34
```

```
    defaultConfig {
        applicationId "com.formalizamx.app"
        minSdkVersion 21
        targetSdkVersion 34
        versionCode 1
        versionName "1.0.0"
    }
```

```
    signingConfigs {
        release {
            if (keystorePropertiesFile.exists()) {
                keyAlias keystoreProperties['keyAlias']
                keyPassword keystoreProperties['keyPassword']
                storeFile file(keystoreProperties['storeFile'])
                storePassword keystoreProperties['storePassword']
            }
        }
    }
}
```

```
    buildTypes {
        release {
```

```

        signingConfig signingConfigs.release
        minifyEnabled true
        shrinkResources true
        proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),
'proguard-rules.pro'
    }
}
}
}

```

Paso 4: Compilar el Android App Bundle (AAB)

Bash
flutter build appbundle --release

- **Archivo generado:** [build/app/outputs/bundle/release/app-release.aab](#)

2. Preparación y Configuración en iOS (IPA)

Paso 1: Permisos de Cámara y Micrófono en [ios/Runner/Info.plist](#)

Requerido para que Apple no rechace la app durante la revisión debido a la función de videollamadas con Daily.co:

XML
<key>NSCameraUsageDescription</key>
<string>Consultoria para todos necesita acceso a la cámara para realizar las asesorías en videollamada con tus especialistas.</string>
<key>NSMicrophoneUsageDescription</key>
<string>Consultoria para todos necesita acceso al micrófono para que puedas comunicarte durante la videollamada.</string>

Paso 2: Configurar Identificadores en Xcode

1. Abre la carpeta [ios](#) en Xcode: [open ios/Runner.xcworkspace](#).
2. Ve a **Runner > Signing & Capabilities**.
3. Selecciona tu **Development Team** registrado en la *Apple Developer Program*.
4. Asegúrate de ajustar el **Bundle Identifier** a [com.formalizamx.app](#).

Paso 3: Compilar la IPA o Archive

Bash
flutter build ipa --release

- **Salida:** Genera la carpeta [build/ios/archive/Runner.xcarchive](#) lista para subir mediante **Xcode Organizer** o **Transporter**.

3. Ficha Técnica y Elementos Promocionales

Plataforma	Requisito Principal	Tamaños / Formato

Google Play	Icono de la App	512 x 512 px (PNG 32-bit)
Google Play	Gráfico de Funciones	1024 x 500 px (JPG o PNG)
Google Play	Capturas de Pantalla	Mínimo 2 capturas (Teléfono y Tablet de 7"/10")
App Store	Capturas para iPhone	6.7" (1290 x 2796 px) y 5.5" (1242 x 2208 px)
App Store	Capturas para iPad	12.9" (2048 x 2732 px) — <i>Si aplica</i>
Ambas	URL de Política de Privacidad	Enlace público alojado (ej. [https://formalizamx.com/privacy] (https://formalizamx.com/privacy))

Checklist Antes del Envío a Revisión

- ☐ Probar la versión de producción conectada a las llaves reales en modo **Live** de Stripe ([pk_live_...](#)).
- ☐ Verificar que la URL de la Política de Privacidad sea pública y mencione el uso de la cámara/micrófono para Daily.co.
- ☐ Asegurar que existan cuentas de prueba (*Demo Account*) para los revisores de Apple/Google.

Implementar la capa de **Monitoreo de Errores y Telemetría** permite rastrear fallos en producción (captura de *crashes*), medir el rendimiento de la aplicación y analizar métricas de uso sin afectar la experiencia del usuario.

1. Configuración de Firebase Crashlytics & Sentry

La combinación estándar recomendada es **Sentry** (o Firebase Crashlytics) para el rastreo en tiempo real.

Paso 1: Agregar dependencias en [pubspec.yaml](#)

YAML

dependencies:

flutter:

```
sdk: flutter
sentry_flutter: ^8.0.0
firebase_crashlytics: ^4.0.0
firebase_analytics: ^11.0.0
```

Paso 2: Inicialización en **lib/main.dart**

Captura tanto los errores no controlados de Flutter como los del motor gráfico o código nativo (C++/Kotlin/Swift):

```
Dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:sentry_flutter/sentry_flutter.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_crashlytics/firebase_crashlytics.dart';

Future<void> main() async {
  runZonedGuarded<Future<void>>>(() async {
    WidgetsFlutterBinding.ensureInitialized();
    await Firebase.initializeApp();

    // 1. Redirigir errores de Flutter a Crashlytics
    FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError;

    // 2. Inicializar Sentry
    await SentryFlutter.init(
      (options) {
        options.dsn = 'https://tu_dsn_clave@sentry.io/proyecto';
        options.tracesSampleRate = 1.0; // Captura de transacciones de rendimiento (100% en
dev, ajustar a 0.2 en prod)
      },
      appRunner: () => runApp(const MyApp()),
    );
  }, (error, stack) {
    // Captura de errores asíncronos fuera del framework
    FirebaseCrashlytics.instance.recordError(error, stack, fatal: true);
    Sentry.captureException(error, stackTrace: stack);
  });
}
```

2. Telemetría de Eventos (Firebase Analytics)

Crea un servicio dedicado para registrar eventos clave en el embudo (*funnel*) de usuarios:

```
Dart
// lib/services/telemetry_service.dart
import 'package:firebase_analytics/firebase_analytics.dart';
```

```

class TelemetryService {
  static final FirebaseAnalytics _analytics = FirebaseAnalytics.instance;

  /// Registra cuando un usuario entra a la sala de videollamada de Daily.co
  static Future<void> logVideoCallStarted(String appointmentId) async {
    await _analytics.logEvent(
      name: 'video_call_started',
      parameters: {
        'appointment_id': appointmentId,
        'timestamp': DateTime.now().toIso8601String(),
      },
    );
  }

  /// Registra cuando se completa un pago exitoso
  static Future<void> logPaymentCompleted(String appointmentId, double amount) async {
    await _analytics.logPurchase(
      currency: 'MXN',
      value: amount,
      transactionId: appointmentId,
    );
  }

  /// Registra desaciertos o errores controlados en flujos críticos
  static Future<void> logCustomError(String context, String errorMessage) async {
    await _analytics.logEvent(
      name: 'app_error',
      parameters: {
        'context': context,
        'error': errorMessage,
      },
    );
  }
}

```

3. Rastreo de Rendimiento en Transacciones Críticas

Monitorea la latencia en operaciones clave (como la creación de salas en Daily.co o la carga de contratos en Supabase):

Dart

```
import 'package:sentry_flutter/sentry_flutter.dart';
```

```

Future<void> fetchTemplatesWithTelemetry() async {
  // Iniciar medición de rendimiento
  final transaction = Sentry.startTransaction('fetchTemplates', 'http.request');

  try {

```

```
// Simulación / Llamada a API o Supabase
await Future.delayed(const Duration(milliseconds: 400));

transaction.status = const SpanStatus.ok();
} catch (exception, stackTrace) {
  transaction.status = const SpanStatus.internalError();
  await Sentry.captureException(exception, stackTrace: stackTrace);
} finally {
  await transaction.finish();
}
}
```

Matriz de Alertas Recomendadas para el Panel de Monitoreo

Métrica	Umbral Crítico	Acción Automática
Crash-free Sessions	< 99.0%	Alerta urgente por Correo/Slack al equipo de desarrollo
Fallo en Pagos (Stripe)	> 5% en 1 hora	Notificación de revisión de credenciales/API Key
Desconexiones Daily.co	> 3%	Revisión del servidor WebRTC / Calidad de red
Carga de Pantalla	> 2.5s	Identificar fuga de memoria o imágenes sin comprimir

Para garantizar que la aplicación mantenga un rendimiento fluido de 60/120 FPS y reduzca su tamaño de descarga al mínimo antes de ser lanzada, se aplican las siguientes optimizaciones clave en arquitectura, empaquetado y manejo de multimedia.

1. Reducción del Tamaño de Binario (App Bundle / IPA)

- **Code Splitting (Deferred Loading):** Carga bajo demanda los módulos que no son necesarios en el inicio (ej. la pantalla de videollamadas con Daily.co o la vista de plantillas):
- Dart

```
import 'package:formaliza_app/screens/video_call_screen.dart' deferred as video_call;

// Al presionar el botón de unirse a la llamada:
onPressed: () async {
  await video_call.loadLibrary();
  Navigator.push(context, MaterialPageRoute(builder: (_) => video_call.VideoCallScreen()));
}
```

- **Tree Shaking de Fuentes e Iconos:** Remueve los glifos no utilizados de Material / Cupertino compilando con:
 - Bash

```
flutter build appbundle --release --tree-shake-icons
```

- **Análisis de Pesos:** Utiliza la herramienta del CLI de Flutter para identificar qué librerías consumen más espacio:

```
Bash
```

```
flutter build appbundle --analyze-size
```

2. Optimización de Assets e Imágenes

- **Formatos de Siguiete Generación:** Reemplaza imágenes PNG/JPEG pesadas por imágenes vectoriales **SVG** (`flutter_svg`) para iconos y esquemas, o formato **WebP** comprimido para fotos de perfil y banners.
- **Caché Eficiente de Imágenes:** Usa `cached_network_image` para imágenes remotas (como avatares de asesores), definiendo límites de memoria y tamaño máximo de renderizado para evitar sobrecargar la memoria RAM:

```
Dart
```

```
CachedNetworkImage(
  imageUrl: advisor.avatarUrl,
  memCacheWidth: 200, // Limita la resolución decodificada en memoria
  placeholder: (context, url) => const CircularProgressIndicator(),
  errorWidget: (context, url, error) => const Icon(Icons.person),
)
```

3. Rendimiento en Pantalla y Renderizado (60 / 120 FPS)

- **Uso Riguroso de Constructores `const`:** Evita que Flutter redibuje (*rebuild*) widgets estáticos al cambiar el estado global o local.

- **Aislamiento de Rebuilds:** Para evitar que toda la pantalla se redibuje durante una actualización de estado (ej. temporizador en la videollamada o contadores), aísla los componentes dinámicos mediante `ValueNotifier` o `RepaintBoundary`:

Dart

```
// Aísla el renderizado pesado del feed de video
RepaintBoundary(
  child: VideoFeedWidget(),
)
```

- **Listas Infinitas Eficientes:** En la pantalla de plantilla de contratos o el catálogo de asesores, utiliza siempre `ListView.builder` o `SliverList` con `itemExtent` o `prototypeItem` para evitar que el motor calcule dinámicamente la altura de cada elemento fuera de pantalla.

4. Gestión de Memoria durante Videollamadas (Daily.co / WebRTC)

- **Limpieza de Controladores (*Disposal*):** Al salir de la sala de videollamada, asegúrate de liberar explícitamente los *streams* de audio/cámara y cancelar suscripciones para evitar fugas de memoria (*memory leaks*):

Dart

```
@override
void dispose() {
  _localVideoTrack?.stop();
  _localAudioTrack?.stop();
  _dailyClient?.dispose();
  super.dispose();
}
```

- **Ajuste Automático de Calidad (Simulcast):** Configura la cámara en resolución dinámica (360p/720p según el ancho de banda) para prevenir el sobrecalentamiento del dispositivo durante llamadas prolongadas.

Checklist Técnico de Rendimiento

- [] Cero advertencias de sintaxis `const` mediante reglas estrictas en `analysis_options.yaml`.
- [] Imágenes remotas comprimidas y almacenadas en caché local.
- [] Puntuación en Flutter Performance Profile sin caídas bruscas de frames (*jank*) durante las transiciones de pantalla.

Una **Estrategia de Lanzamiento y Marketing** bien estructurada para **Consultoría para todos** garantiza la adquisición de los primeros usuarios, la atracción de asesores calificados y el posicionamiento de la app en el sector LegalTech.

1. Fases del Plan de Lanzamiento

Fase 1: Pre-lanzamiento (Beta Privada & Captación de Asesores)

- **Landing Page de Captación:** Publica un sitio web sencillo con un formulario de registro para la lista de espera ("Early Access") ofreciendo un beneficio exclusivo (ej. primera asesoría con 20% de descuento o plantilla de contrato gratuita).
- **Onboarding de Asesores:** Recluta al primer grupo de 10 a 15 especialistas (contadores, abogados corporativos y fiscales) para probar la plataforma y asegurar disponibilidad de agenda desde el día uno.
- **Prueba Beta:** Distribuye la versión Beta mediante **TestFlight (iOS)** y **Google Play Internal Testing** a un grupo focalizado de 50 emprendedores para recopilar retroalimentación inicial.

Fase 2: Lanzamiento Oficial (Go-to-Market)

- **Publicación Multi-canal:** Publica las fichas finales en App Store y Google Play.
- **Campaña de Contenido:** Publica artículos de blog y carruseles explicativos en redes sociales abordando dudas comunes sobre trámites en México (ej. *"Cómo tramitar tu e.firma en el SAT sin morir en el intento"* o *"Diferencias entre una SAS y una S.A. de C.V."*).
- **Evento de Lanzamiento Digital:** Organiza un webinar gratuito o transmisión en vivo con uno de los asesores top sobre *"Errores fiscales y legales que cometen los emprendedores al iniciar"*, invitando a los asistentes a descargar la app para continuar su proceso.

Fase 3: Post-lanzamiento (Retención y Referidos)

- **Programa de Referidos:** Ofrece un cupón de descuento en la siguiente videollamada tanto al usuario que refiere como al nuevo usuario que agenda su primera sesión.
- **Automatización de Retención:** Envía notificaciones push y correos dinámicos basados en la etapa del trámite en la que se encuentra el usuario.

2. Canales de Adquisición de Usuarios

- **Marketing de Contenido e Inbound (SEO/Social):**
 - **Instagram / TikTok / LinkedIn:** Videos cortos con consejos rápidos sobre trámites del SAT, marcas en el IMPI y contratos.
 - **Blog / SEO Local:** Artículos optimizados para búsquedas frecuentes en Google (ej. *"requisitos para dar de alta mi empresa en México"*).
- **Publicidad Digital Pagada (SEM / Paid Social):**
 - **Google Search Ads:** Campañas enfocadas en intenciones de búsqueda directa (ej. *"asesoría fiscal online México"*, *"contrato de servicios plantilla"*).
 - **Meta Ads (Instagram/Facebook):** Anuncios dirigidos a emprendedores, dueños de PyMEs y freelancers.

- **Alianzas Estratégicas (B2B2C):**
 - **Co-workings e Incubadoras:** Establece convenios con espacios de trabajo compartido y aceleradoras de negocios para ofrecer Consultoría para todos como su solución preferida de cumplimiento legal.
 - **Comunidades de Emprendimiento:** Alianzas con grupos de creadores, startups y redes de profesionistas independientes.

3. Indicadores Clave de Desempeño (KPIs)

Área	Métrica Clave	Objetivo Inicial (Mes 1 - 3)
Adquisición	Descargas totales	1,000+ descargas
Conversión	Tasa de registros a citas pagadas	5% - 8%
Uso del Servicio	Videollamadas completadas	100+ sesiones ejecutadas
Satisfacción	Calificación promedio en Tiendas y App	≥ 4.7 estrellas
Costo	CAC (Costo de Adquisición de Cliente)	Calibrar en pruebas iniciales de anuncios

Con esta estrategia se completa el mapa de ruta integral para Consultoría para todos: desde la pantalla de videollamada en Flutter hasta la arquitectura en Supabase, las pruebas E2E, la optimización técnica y el plan comercial de salida al mercado.

Hemos cubierto todo el ciclo de vida del producto:

- **Frontend y UI:** Pantallas de trámites, marketplace de asesores, integración de pagos con Stripe, videollamadas con Daily.co y el catálogo de plantillas legales.
- **Backend y Base de Datos:** Arquitectura relacional en Supabase con PostgreSQL, RLS y soporte para tokens FCM.
- **Notificaciones Push y Telemetría:** Integración de Firebase Messaging, Crashlytics, Sentry y rastreo de rendimiento.

- **Calidad y Despliegue:** Pruebas E2E automáticas con `integration_test`, optimización de assets/memoria, preparación de paquetes de producción (AAB/IPA) y estrategia Go-to-Market.

Lo siguiente podría ser:

1. **Escribir código o scripts específicos:** Si necesitas algún widget adicional, un script SQL complementario o la configuración de alguna Edge Function en Supabase.
2. **Generar documentación técnica:** Crear el archivo `README.md` del repositorio, la guía de arquitectura o la documentación de la API.
3. **Revisar dudas conceptuales:** Resolver cualquier consulta sobre arquitectura, integraciones con terceros o reglas de negocio.